École doctorale **Mathématiques, sciences de l'information et de l'ingénieur** | ED 269
Université de Strasbourg

UNIVERSITÉ DE STRASBOURG

Université de Strasbourg

ÉCOLE DOCTORALE DE MATHÉMATIQUES, SCIENCES DE L'INFORMATION ET DE L'INGÉNIEUR
Laboratoire des sciences de l'ingénieur, de l'informatique et de l'imagerie (UMR 7357)

THÈSE présentée par

Ugo BATTISTON

soutenue le : 10 décembre 2026

pour obtenir le grade de : **Docteur de l'Université de Strasbourg**
Discipline/Spécialité : **Informatique**

Optimisation polyédrique automatique des noyaux Kokkos : un co-design bibliothèque–compilateur

THÈSE dirigée par:

M. CLAUSS Philippe Professeur, Université de Strasbourg
M. PÉRACHE Marc Directeur de recherche, CEA DAM Île-de-France

RAPPORTEURS :

Mme ANCOURT Corinne Directrice de recherche, École des Mines de Paris
M. THIBAULT Samuel Professeur, Université de Bordeaux

EXAMINATEURS:

Mme BRUNET Elisabeth Maîtresse de conférences, Institut Mines-Télécom
M. ALIAS Christophe Chargé de recherche, ENS Lyon

Table des matières

Table des matières i
Table des figures v
Liste des extraits de code vii
Liste des tableaux ix
1. Introduction 1
1.1. Compilateurs 2
1.2. Bibliothèques de portabilité des performances 3
1.3. Modèle polyédrique 3
1.4. Plan et contributions 4
2. Contexte scientifique 7
2.1. Nids de boucles 7
2.1.1. Boucles parfaitement imbriquées 8
2.1.2. Boucles imparfaitement imbriquées 8
2.2. Le modèle de programmation Kokkos 9
2.2.1. Espaces : exécution et mémoire 9
2.2.2. Abstraction des données 10
2.2.3. Exécution parallèle 11
2.2.3.1. Motifs d'exécution 12
2.2.3.2. Politiques d'exécution 12
2.2.3.2.1. RangePolicy et MDRangePolicy 12
2.2.3.2.2. TeamPolicy 13
2.3. Modèle polyédrique 14
2.3.1. Polyèdre 16
2.3.2. Instruction 17
2.3.3. Domaine d'itération 17
2.3.4. Dépendances de données 19
2.3.5. Ordonnancement 20
2.4. L'écosystème polyédrique 22
2.4.1. Outils polyédriques 23
2.4.1.1. Integer Set Library (ISL) 23

2.4.1.2. Représentation standardisée OpenScop 24
2.4.2. Compilateurs polyédriques 26
2.5. Plongée au cœur de LLVM Polly 27
2.5.1. Architecture et intégration dans la chaîne de compilation 28
2.5.1.1. Préparation du code (`Code Preparation`) 29
2.5.1.2. Détection des SCoPs (`SCoP Detection`) 30
2.5.1.3. Construction des SCoPs (`SCoP Building`) 30
2.5.1.3.1. Délinéarisation des accès mémoire 31
2.5.1.4. Optimisation des SCoPs (`SCoP Optimization`) 33
2.5.1.5. Génération de l'AST (`IslAst`) 33
2.5.1.6. Génération de code (`Code Generation`) 34
2.5.2. Limites de Polly 34
3. État de l'art 37
3.1. Bibliothèques de portabilité des performances 38
3.2. Modèle polyédrique et implémentations 41
3.3. Combiner abstractions de haut niveau et optimisation polyédrique 43
4. Rendre les noyaux de calcul de Kokkos visibles 47
4.1. Motivations 48
4.2. Modifications de Kokkos 49
4.2.1. Option *usePolyOpt* 50
4.2.2. Réécriture du backend et isolation 51
4.2.3. Annotations 54
4.3. Préparation du code et restructuration de l'IR 55
4.3.1. Extraction et exploitation des annotations 56
4.3.2. Simplification et assainissement du CFG 59
4.3.3. Canonicalisation des registres de tableaux 62
4.3.4. Délinéarisation 66
4.3.5. Synchronisation du backend 68
5. Motifs d'exécution complexes 71
5.1. Boucles triangulaires 72
5.1.1. Problèmes liés à la représentation intermédiaire 72
5.1.2. Épluchage de boucles 74

5.1.2.1. Détection des nids de boucles triangulaires 75
5.1.2.2. Application de l'épluchage de boucle 76
5.1.2.3. Régularisation du flot de contrôle 77
5.2. Vision globale des boucles 78
5.2.1. Nouveau `parallel_for` variadique 80
5.2.1.1. Linéarisation du flot de contrôle 82
5.2.2. Nommage statique des tableaux 84
5.2.3. Injection de contraintes 86
5.2.3.1. Grammaire des contraintes 88
5.2.4. Limitations de la vision globale des noyaux 90
6. Ordonnancement et génération hétérogènes 93
6.1. Intégration de Pluto 94
6.1.1. Représentation OpenScop 95
6.1.1.1. Conversion du tuilage 96
6.2. Intégration de PPCG et réhabilitation de Polly-ACC 98
7. Évaluations 103
7.1. Protocole expérimental 103
7.2. Évaluation sur CPU 106
7.2.1. Étude des résultats CPU 106
7.2.2. Gains de localité 110
7.2.3. Gains de parallélisme 112
7.2.4. Régressions 113
7.2.5. Sensibilité au matériel 117
7.2.5.1. Le cas de la factorisation `lu` 117
7.2.5.2. Régressions liées au tuilage fixe 118
7.2.6. Bilan de l'évaluation sur CPU 118
7.3. Évaluation sur GPU 119
7.3.1. Étude des résultats GPU 119
7.3.2. Parallélisation des noyaux à dépendances 121
7.3.3. Exploitation de la hiérarchie mémoire 122
7.3.4. Contre-performances 123
7.3.5. Bilan de l'évaluation sur GPU 123

7.4. Temps de compilation 124
7.5. Comparaison avec Polly sur PolyBench 125
8. Conclusion et perspectives 131
8.1. Contributions 131
8.1.1. Visibilité des noyaux Kokkos 131
8.1.2. Motifs d'exécution complexes 132
8.1.3. Ordonnancement et génération de code hétérogène 132
8.2. Discussion et limites 133
8.3. Perspectives 134
Références 135

Table des figures

Figure 1 Représentation géométrique du domaine d'itération pour l'instruction S_1 18
Figure 2 Exemple d'un nid de boucles simple et de sa représentation ISL correspondante. 24
Figure 3 Exemple d'un nid de boucles simple et de sa représentation OpenScop correspondante. 25
Figure 4 Intégration des passes de Polly au sein de la chaîne d'optimisation *middle-end* de LLVM à la position `before-vectorizer`. (Source : Documentation de LLVM Polly) 28
Figure 5 Chaîne de compilation polyédrique source-à-source. 41
Figure 6 Chaîne de compilation polyédrique au niveau de l'IR. 42
Figure 7 Architecture de l'outil Apollo, séparant la compilation statique et l'optimisation dynamique à l'exécution. 43
Figure 8 Chaîne de compilation proposée, issue du co-design entre Kokkos et Polly. 49
Figure 9 Architecture de sélection des backends dans Kokkos. 52
Figure 10 Élévation du chargement de la borne de boucle dans le pré-en-tête du SCoP. . . . 58
Figure 11 Exemple de graphe de flot de contrôle (CFG) complexifié par des branchements conditionnels et du versionnage de boucles, entravant la détection du SCoP. 61
Figure 12 Exemple de dépendance scalaire sur `a` entre deux *statements* dans le modèle IR de Polly. 63
Figure 13 Comparaison des expressions d'évolution scalaire (SCEV) générées par LLVM pour un tableau tridimensionnel. Lors d'un accès partiel, la simplification algébrique entraîne une perte d'information structurelle que seules nos annotations permettent de restaurer. 67
Figure 14 Comparaison de la sélection du modèle d'exécution entre l'approche globale monolithique de Polly et l'approche fine par noyau de Kokkos. 69
Figure 15 Représentation simplifiée en IR LLVM du noyau **trisolv** (Liste 13). 73
Figure 16 Évolution du graphe de flot de contrôle du noyau **trisolv** suite à la passe d'épluchage statique. 78
Figure 17 Transformation de l'IR illustrant la linéarisation du flot de contrôle et la fusion des noyaux. 84

Figure 18 Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend OpenMP) avec les ordonnanceurs ISL et Pluto sur processeur Dual AMD EPYC 7502 (2×32 cœurs, 64 threads sans hyperthreading, taille de tuile : 32). 107
Figure 19 Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend OpenMP) avec les ordonnanceurs ISL et Pluto sur processeur Intel Core Ultra 7 265 (20 threads sans hyperthreading, taille de tuile : 32). 108
Figure 20 Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend CUDA) avec l'ordonnanceur PPCG sur accélérateur NVIDIA RTX 5090 (32 Go VRAM, hôte Intel Core Ultra 7 265, 20 threads sans hyperthreading). 120
Figure 21 Accélération de Kokkos + Polly (usePolyOpt) par rapport à Polly appliqué à PolyBench/C, avec l'ordonnanceur ISL sur processeur Intel Core Ultra 7 265 (20 threads sans hyperthreading, taille de tuile : 32). 126

Liste des extraits de code

Liste 1 Exemple d'un nid de boucles parfaitement imbriqué de profondeur 2. 8

Liste 2 Exemple d'un nid de boucles imparfaitement imbriqué où les instructions existent à des profondeurs différentes. 9

Liste 3 Exemple d'allocation standard de `Kokkos::View` et de transferts de mémoire hétérogènes. 11

Liste 4 Exemple d'une addition de matrices parallèle utilisant une politique d'exécution `MDRangePolicy`. 13

Liste 5 Exemple de deux nids de boucles simples avec deux instructions. 16

Liste 6 Exemple de code source en C avec un accès multidimensionnel 3D. 31

Liste 7 Représentation de l'évolution scalaire de l'adresse mémoire de `A[i][j][k]` issue de la Liste 6. 31

Liste 8 Exemple de syntaxe déclarative dans Tiramisu, séparant proprement l'algorithme de son ordonnancement d'exécution. 43

Liste 9 Exemple de multiplication matrice-vecteur utilisant l'interface Python de haut niveau PyKokkos. 45

Liste 10 Nouvelle signature de `parallel_for` avec le paramètre `usePolyOpt`. 50

Liste 11 Exemple d'utilisation du paramètre `usePolyOpt` pour activer l'analyse polyédrique sur un noyau Kokkos spécifique. 51

Liste 12 Exemple d'annotation de borne de boucle impliquant une expression arithmétique `n-1`. 57

Liste 13 Exemple de code Kokkos présentant un motif de boucle triangulaire (noyau **trisolv**). 72

Liste 14 Implémentation du noyau `gemver` en Kokkos, illustrant la fragmentation en de multiples appels à `parallel_for`. 79

Liste 15 `parallel_for` variadique. 81

Liste 16 Implémentation du noyau `gemver` en Kokkos, illustrant le regroupement en un unique appel au `parallel_for` variadique. 82

Liste 17 Définition de la structure `ConstExprLabel` permettant l'intégration de chaînes de caractères statiques dans les paramètres de *template* d'une `View`. 85

Liste 18 Implémentation de l'injection statique des chaînes de contraintes (**assumptions**) dans les fonctions de Kokkos. 87
Liste 19 Exemple de code avec des contraintes injectées. 89
Liste 20 Représentation ensembliste et relationnelle d'un domaine et de son ordonnancement sous ISL. 95
Liste 21 Traduction matricielle du domaine et de l'ordonnancement au format OpenScop. 96
Liste 22 Domaine et ordonnancement originaux générés par Polly avant optimisation. 97
Liste 23 Représentation après optimisation par Pluto, avec ajout de dimensions de tuilage (`fk0`, `fk1`, `fk2`) dans le domaine. 98
Liste 24 Ordonnancement converti pour Polly : le domaine initial est restauré et les contraintes de tuilage sont déléguées à la relation d'ordonnancement. 98
Liste 25 Implémentations Kokkos du noyau `symm` : version de référence, parallélisée sur `j` seul, et version analysée par Polly (`usePolyOpt`). 111
Liste 26 Structure simplifiée du code généré par Polly avec l'ordonnanceur ISL pour `symm` : les trois nids principaux, parcourus par tuiles de 32. Les boucles internes aux tuiles sont omises. 111
Liste 27 Structure de l'AST généré par Polly avec l'ordonnanceur Pluto pour `seidel-2d` sur CPU. 113
Liste 28 Extrait de l'AST généré par Polly avec l'ordonnanceur ISL pour `deriche` sur CPU : une passe récursive et une passe de combinaison. 114
Liste 29 Structure de l'AST généré par Polly avec l'ordonnanceur ISL pour `heat-3d` sur CPU. La boucle la plus interne est marquée comme pouvant être parallélisée avec l'annotation `#pragma known-parallel`. 115
Liste 30 Extrait simplifié de l'AST généré par Polly avec l'ordonnanceur Pluto pour `adi` sur CPU : le nid fusionné, à l'intérieur de la boucle temporelle. Les bornes et les indices sont omis, et le paramètre `p_0` a été renommé `N`. 117
Liste 31 Extrait simplifié du code hôte généré par PPCG pour `lu`. 122

Liste des tableaux

Tableau 1 Comparaison syntaxique des boucles parallèles entre différentes bibliothèques C++ de portabilité des performances. 39
Tableau 2 Liste de transformations de dépendances scalaires vers des accès mémoire explicites, appliquées par la passe `ArrayReg2MemPass`. 64
Tableau 3 Caractéristiques des deux plateformes d'évaluation. 104
Tableau 4 Catégories et tailles des 27 noyaux PolyBench évalués. 105
Tableau 5 Moyennes géométriques des accélérations sur CPU par catégorie de benchmark et par ordonnanceur. Entre parenthèses figure le nombre de noyaux accélérés sur le nombre de noyaux évalués. 109
Tableau 6 Moyennes géométriques des accélérations sur GPU par catégorie de benchmark. Entre parenthèses figure le nombre de noyaux accélérés sur le nombre de noyaux évalués. 121
Tableau 7 Temps de compilation relatif de Clang avec Kokkos + Polly (`usePolyOpt`) par rapport à Clang avec Kokkos, mesuré sur la machine Intel. 124

Chapitre 1

Introduction

La puissance de calcul des supercalculateurs n'a cessé d'augmenter au fil des années, stimulée par l'évolution rapide des architectures matérielles. Historiquement, cette progression a été alimentée par la loi de Moore à travers l'augmentation du nombre de transistors, leur miniaturisation et l'élévation de la fréquence d'horloge des processeurs. Toutefois, alors que les contraintes physiques ont mis un frein à la montée en fréquence, l'industrie s'est massivement tournée vers le parallélisme matériel, conduisant à l'adoption généralisée d'architectures pipelinées et vectorielles, de systèmes à mémoire partagée et distribuée, ainsi qu'à l'intégration d'accélérateurs matériels.

L'exploitation de ces architectures hétérogènes a entraîné l'émergence de multiples paradigmes de programmation, rendant le développement de codes scientifiques de haute performance de plus en plus complexe. Bien que des efforts aient été déployés pour déléguer cette complexité aux compilateurs et autres outils automatisés, l'obtention d'une optimisation et d'une parallélisation purement automatiques sur les systèmes modernes multicœurs et basés sur des accélérateurs reste un défi majeur.

Bien que les compilateurs modernes intègrent de nombreuses passes d'optimisation, ils peinent fréquemment à appliquer efficacement des parallélisations avancées et des transformations de boucles. Cette limitation provient principalement du fossé sémantique entre l'intention de haut niveau du programmeur et la représentation intermédiaire de bas niveau comprise par la machine. Pour réduire les coûts de développement, la communauté du calcul haute performance (HPC) a conçu des bibliothèques de portabilité des performances. Celles-ci fournissent des abstractions de haut niveau permettant aux développeurs d'exprimer

explicitement des motifs de calcul optimisés, des stratégies de parallélisation et la gestion de la mémoire, avec la promesse de hautes performances sur diverses architectures, processeurs (CPU) comme accélérateurs graphiques (GPU), à partir d'un code source unique et unifié.

Alors que ces environnements facilitent grandement l'expression du parallélisme, l'optimisation fine des nids de boucles reste un défi majeur. Le modèle polyédrique offre une solution mathématique éprouvée pour l'optimisation des boucles. Cette approche algébrique et géométrique permet l'analyse précise des dépendances de données et l'application de transformations complexes que les compilateurs traditionnels peinent à réaliser. Une solution prometteuse au défi de l'optimisation des boucles réside dans la synergie entre la puissance analytique du modèle polyédrique et les abstractions de haut niveau des bibliothèques de portabilité des performances. En faisant converger ces deux approches, il devient possible d'optimiser et de paralléliser efficacement les codes scientifiques sur des architectures complexes et hétérogènes.

1.1. Compilateurs

Le compilateur sert de pont critique entre les abstractions de haut niveau écrites par le développeur et les instructions de bas niveau exécutées par la machine. En plus de la conversion en un fichier binaire exécutable, le compilateur est responsable de l'application d'optimisations spécifiques. Idéalement, un développeur pourrait s'en remettre entièrement aux heuristiques internes et aux passes d'optimisation du compilateur pour atteindre des performances maximales. Cependant, pour maximiser l'utilisation des ressources matérielles pour les codes HPC intensifs en calcul, cela est rarement suffisant.

Une approche alternative consiste à effectuer des transformations et des optimisations manuelles du code directement au niveau de la source pour aider le compilateur. Bien qu'une optimisation manuelle bien exécutée produise souvent des performances supérieures, elle exige une expertise significative, augmente le temps de développement et dégrade sévèrement la maintenabilité du code. Elle nécessite en effet que le développeur soit à la fois un scientifique du domaine et un expert en optimisation de code de bas niveau. Pour surmonter ce goulot d'étranglement, les modèles de portabilité des performances offrent une alternative permettant d'obtenir des performances sans requérir d'expertise de

développement approfondie. Ils permettent aux développeurs d'exprimer simplement leurs noyaux de calcul, tandis que la bibliothèque gère la parallélisation, le placement des données et le déploiement de ces noyaux pour maximiser les performances sur les cibles matérielles choisies.

1.2. Bibliothèques de portabilité des performances

Les bibliothèques de portabilité des performances offrent aux développeurs des interfaces logicielles pour exprimer le parallélisme et gérer la mémoire sur diverses architectures matérielles. Dans le domaine du HPC, les bibliothèques les plus utilisées sont Kokkos, RAJA, SYCL et le standard de parallélisme C++ (`std::execution::par`). Ces outils permettent le développement d'un code source unique pouvant être compilé pour différentes cibles, avec la promesse de la portabilité des performances.

Cependant, ces bibliothèques n'effectuent pas intrinsèquement d'optimisation automatique du code source. Elles s'appuient sur des optimisations structurelles statiques et un parallélisme explicite sans réaliser d'analyse statique profonde des noyaux de calcul eux-mêmes. Les performances qu'elles délivrent proviennent principalement d'un découpage parallèle robuste et de techniques de placement de bas niveau implémentées par les développeurs de la bibliothèque, plutôt que d'une restructuration agressive des boucles. En employant le modèle polyédrique, des transformations de boucles complexes peuvent être automatiquement appliquées à ces régions de code critiques et coûteuses en calcul pour maximiser l'utilisation du matériel.

1.3. Modèle polyédrique

Le modèle polyédrique est un ensemble de méthodes mathématiques dédié à l'optimisation des nids de boucles. Il a prouvé son efficacité dans l'optimisation de programmes grâce à des implémentations robustes telles que Pluto [Bon+08a], Polly [Gro+11] et PPCG [Ver+13]. En exprimant les boucles d'un programme sous une forme mathématique à l'aide d'ensembles et

de relations, le modèle permet des transformations linéaires complexes qui sont typiquement hors de portée des optimiseurs traditionnels basés sur les arbres syntaxiques abstraits (AST). Parmi ses capacités, il peut automatiquement réordonner, fusionner, diviser, tuiler, vectoriser et paralléliser les boucles.

L'application du modèle polyédrique est limitée aux noyaux de calcul contenant des boucles imbriquées avec des accès aux données affines. Dans l'univers du HPC, ces noyaux spécifiques correspondent souvent aux portions les plus chronophages et critiques d'une application. De manière cruciale, ce sont exactement les mêmes régions de code où les bibliothèques de portabilité des performances offrent leur plus grand avantage, en fournissant une syntaxe simplifiée pour l'exécution parallèle. Cibler ces régions avec des techniques polyédriques permet aux codes HPC d'être significativement améliorés quant aux performances à l'exécution.

1.4. Plan et contributions

Cette thèse introduit et implémente une approche hybride combinant le modèle polyédrique avec la bibliothèque de portabilité des performances Kokkos [ET13, Tro+22]. L'objectif principal est de permettre dans Kokkos d'appliquer les capacités avancées d'optimisation de boucles fournies par le modèle polyédrique.

Le chapitre 2 détaille le contexte scientifique et technique nécessaire pour comprendre les fondations de cette thèse. Il introduit les concepts fondamentaux des nids de boucles, les principes de base de Kokkos, et les fondements mathématiques du modèle polyédrique.

Le chapitre 3 présente l'état de l'art, en passant en revue les recherches existantes sur les optimisations dans Kokkos, les outils polyédriques spécifiques, les développements de l'écosystème autour de Kokkos, et la recherche sur l'optimisation des bibliothèques logicielles Python à l'aide du modèle polyédrique.

Au chapitre 4, la première partie de l'implémentation de l'approche hybride proposée est détaillée. Ce chapitre introduit les modifications apportées à Kokkos pour exposer ses noyaux de calcul à l'optimisation polyédrique via l'extension Polly de LLVM, qui a elle-même été étendue pour traiter les constructions Kokkos.

Le chapitre 5 se penche sur les schémas d'exécution complexes, détaillant comment la vision polyédrique peut aller au-delà d'un noyau Kokkos unique, et comment gérer le cas complexe des boucles triangulaires.

Le chapitre 6 couvre les phases d'ordonnancement et de génération de code hétérogène, incluant l'intégration d'ordonnanceurs externes comme Pluto et la génération de code GPU via PPCG.

Le chapitre 7 présente l'évaluation expérimentale de cette chaîne d'outils. Notre approche est testée sur divers codes scientifiques, ses performances sont comparées à des implémentations Kokkos idiomatiques parallélisées. Ces résultats expérimentaux valident l'efficacité et la viabilité de l'approche hybride proposée et de notre implémentation.

Enfin, le chapitre 8 conclut ce manuscrit et discute des potentielles perspectives de recherche futures.

Chapitre 2

Contexte scientifique

Cette thèse se concentre sur l'application du modèle polyédrique pour optimiser statiquement les nids de boucles exprimés en Kokkos [ET13, Tro+22]. Étant donné que ces travaux font le pont entre les bibliothèques de portabilité des performances de haut niveau et les transformations mathématiques de boucles de bas niveau, une solide compréhension de ces deux domaines est requise.

Ce chapitre fournit les bases théoriques nécessaires pour la suite du manuscrit. Tout d'abord, la section 2.1 présente le concept de nid de boucles. La section 2.2 introduit le modèle de programmation Kokkos, en détaillant ses abstractions de mémoire et d'exécution. La section 2.3 explore la représentation polyédrique et la section 2.4 décrit les outils et compilateurs de l'écosystème polyédrique, avec une attention particulière portée dans la section 2.5 à l'outil Polly qui constitue le point de départ de notre implémentation.

2.1. Nids de boucles

En programmation comme en mathématiques, une boucle `for` est caractérisée par une variable d'itération i (souvent appelée itérateur ou indice), une borne inférieure, une borne supérieure et un pas (stride). Lorsqu'une ou plusieurs boucles sont imbriquées dans une autre boucle, la structure résultante est appelée un **nid de boucles**. Le nombre de boucles imbriquées définit la **profondeur** du nid. À toute étape d'exécution donnée, l'état d'un nid de boucles de profondeur n est identifié de manière unique par son **vecteur d'itération** $\vec{i} = (i_1, i_2, ..., i_n)^T$, qui regroupe les valeurs actuelles de tous les indices des boucles englobantes. À l'intérieur de ces boucles,

une **instruction** représente l'opération ou l'instruction de calcul réelle qui est exécutée à un point d'itération donné.

Ces structures sont largement utilisées dans le HPC et les codes scientifiques, car elles fournissent le mécanisme principal pour parcourir et manipuler de grandes structures de données multidimensionnelles telles que les vecteurs, les matrices et les tenseurs.

2.1.1. Boucles parfaitement imbriquées

Un nid de boucles est considéré comme **parfaitement imbriqué** si toutes les instructions de calcul sont situées exclusivement dans la boucle la plus interne. Il n'y a pas de code exécuté entre les instructions `for` des boucles externes et internes.

La liste 1 illustre une boucle parfaitement imbriquée de profondeur 2 avec une seule instruction `S1`.

```cpp
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        C[i][j] = A[i][j] + B[i][j];  // Instruction S1 (Profondeur 2)
    }
}
```

Liste 1. – Exemple d'un nid de boucles parfaitement imbriqué de profondeur 2.

2.1.2. Boucles imparfaitement imbriquées

En pratique, de nombreux algorithmes scientifiques ne peuvent pas être écrits sous forme de boucles parfaitement imbriquées. Nous généralisons le concept aux **boucles imparfaitement imbriquées**, qui se produisent lorsque des instructions de calcul existent à différents niveaux d'imbrication. En d'autres termes, les instructions sont intercalées entre les boucles. De plus, plusieurs boucles de même profondeur peuvent apparaître dans le nid.

La liste 2 présente un nid de boucles imparfait où une instruction d'initialisation `S1` est exécutée à l'intérieur de la boucle externe `i`, mais à l'extérieur de la boucle interne `j` qui contient la seconde instruction `S2`.

```cpp
for (int i = 0; i < N; i++) {
    row_sum[i] = 0;                 // Instruction S1 (Profondeur 1)
    for (int j = 0; j < M; j++) {
        row_sum[i] += A[i][j];      // Instruction S2 (Profondeur 2)
    }
}
```

Liste 2. – Exemple d'un nid de boucles imparfaitement imbriqué où les instructions existent à des profondeurs différentes.

Bien que les nids de boucles imparfaits soient naturels à écrire pour les développeurs, ils posent un défi important pour les bibliothèques de parallélisation comme Kokkos, présentées dans la section 2.2. L'extraction du parallélisme et l'optimisation de la localité des données dans ces nids imparfaits nécessitent des transformations de code complexes. Cette complexité mathématique est précisément ce que le modèle polyédrique, présenté à la section 2.3 suivante, vise à résoudre.

2.2. Le modèle de programmation Kokkos

Initialement développé pour abstraire le parallélisme matériel, le modèle de programmation Kokkos [Tro+22] s'est imposé comme une bibliothèque de référence dans le HPC et la simulation scientifique. Il fournit une interface C++ unifiée pour la programmation parallèle hétérogène, avec un accent principal sur la portabilité des performances. En offrant des abstractions de haut niveau pour la gestion de la mémoire et le contrôle de l'exécution, Kokkos simplifie le processus de développement tout en voulant garantir l'obtention de performances, quelle que soit l'architecture cible.

2.2.1. Espaces : exécution et mémoire

Pour cibler diverses architectures matérielles, Kokkos introduit le concept d'espaces pour abstraire les unités d'exécution complexes et les hiérarchies de mémoire des supercalculateurs modernes. Un nœud HPC typique est hétérogène, couplant souvent un processeur hôte (CPU) avec un ou plusieurs accélérateurs (GPU), chacun pouvant posséder sa propre mémoire physique distincte. Kokkos gère cette hétérogénéité en découplant clairement l'endroit où le code s'exécute de l'endroit où résident les données :

- **Les espaces d'exécution** définissent où les noyaux de calcul sont exécutés. Ils associent les opérations parallèles à un matériel spécifique via une implémentation dédiée du modèle de programmation. Par exemple, `Kokkos::Serial` ou `Kokkos::OpenMP` dictent que le code s'exécutera sur le CPU hôte, tandis que `Kokkos::Cuda` ou `Kokkos::HIP` ciblent les accélérateurs GPU.
- **Les espaces mémoire** définissent où les données sont physiquement allouées. Ils abstraient la hiérarchie de la mémoire de la machine cible. Des exemples courants incluent `Kokkos::HostSpace` pour la mémoire principale de la machine hôte, `Kokkos::CudaSpace` pour la mémoire du périphérique GPU CUDA, ou `Kokkos::CudaUVMSpace` pour la mémoire virtuelle unifiée (UVM).

Ce concept d'espaces est étroitement lié aux abstractions de données et aux motifs d'exécution parallèle de Kokkos, qui sont abordés dans les sections suivantes.

2.2.2. Abstraction des données

Pour simplifier la gestion des données à travers des architectures hétérogènes, Kokkos fournit une abstraction de données fondamentale : la structure `Kokkos::View`. Cette structure de données C++ patron agit comme un tableau multidimensionnel léger à comptage de références. Une `Kokkos::View` encapsule le pointeur de données, ses dimensions, l'espace mémoire cible et sa disposition en mémoire, abstrayant efficacement les allocations matérielles.

Une caractéristique cruciale pour la portabilité des performances est la disposition en mémoire. Étant donné que les CPU et les GPU gèrent les accès mémoire différemment, Kokkos définit des traits de disposition spécifiques :

- `Kokkos::LayoutRight` (row-major) : Optimise la localité spatiale et l'utilisation des lignes de cache pour les CPU.
- `Kokkos::LayoutLeft` (column-major) : Assure la coalescence des accès mémoire, ce qui est critique pour les performances des GPU.

Par défaut, Kokkos sélectionne automatiquement la disposition optimale à la compilation en fonction de l'espace d'exécution, bien qu'elle puisse être spécifiée explicitement par le programmeur.

La liste 3 montre un flux de travail de gestion de mémoire hétérogène basique. À la ligne 2, une vue 2D de flottants est allouée directement dans la mémoire du GPU (`Kokkos::CudaSpace`), en utilisant explicitement une disposition `Kokkos::LayoutLeft`. La ligne 4 démontre la création d'une vue miroir accessible par l'hôte (`Kokkos::create_mirror_view`). Cette fonction alloue un tableau équivalent dans l'espace mémoire de l'hôte, permettant au CPU de manipuler les données en toute sécurité.

L'opérateur parenthèse surchargé `operator()` fournit une syntaxe d'accès multidimensionnelle intuitive, gérant automatiquement la linéarisation complexe des indices (ligne 7). Enfin, la synchronisation explicite des données entre les espaces mémoire de l'hôte et du périphérique est effectuée à l'aide de `Kokkos::deep_copy` (ligne 9).

```cpp
1  // ...
2  Kokkos::View<double **, Kokkos::CudaSpace, Kokkos::LayoutLeft>
   tab_device("tab_device", N, N);
3
4  auto tab_host = Kokkos::create_mirror_view(tab_device);
5
6  for(int i = 0; i < N; i++)
7      tab_host(i, i) = static_cast<double>(i);
8
9  Kokkos::deep_copy(tab_device, tab_host);
10 // ...
```

Liste 3. – Exemple d'allocation standard de `Kokkos::View` et de transferts de mémoire hétérogènes.

Kokkos fournit plusieurs types de vues spécialisées pour des cas d'utilisation avancés (par exemple : `DualView`, `DynRankView`, `OffsetView`). Cependant, dans le cadre de cette première implémentation, cette thèse se concentre exclusivement sur les vues standards (`Kokkos::View`). Ce choix permet de cibler la structure de données fondamentale et la plus répandue de l'écosystème Kokkos, tout en garantissant la compatibilité avec le modèle polyédrique.

2.2.3. Exécution parallèle

Dans le modèle de programmation Kokkos, le lancement d'un noyau de calcul nécessite de combiner un motif d'exécution, une politique d'exécution et le corps du noyau (généralement défini via une fonction lambda C++ ou un foncteur). Cette séparation des concepts permet

aux développeurs d'exprimer la sémantique de leur algorithme indépendamment de leur placement sur le matériel.

2.2.3.1. Motifs d'exécution

Pour abstraire les modèles de programmation parallèle spécifiques au matériel (ex: OpenMP, CUDA), Kokkos fournit trois motifs d'exécution parallèle fondamentaux. Un motif d'exécution dicte la sémantique de l'opération :

- `Kokkos::parallel_for` : Applique une opération indépendante sur un espace d'itération. Il représente les boucles standards de parallélisme de données où aucune dépendance n'existe entre les itérations.
- `Kokkos::parallel_reduce` : Calcule une réduction (ex: somme, min, max, ou réductions personnalisées) sur un espace d'itération. Kokkos gère automatiquement les accès concurrents spécifiques à l'architecture.
- `Kokkos::parallel_scan` : Effectue une somme préfixe (scan) sur un espace d'itérations, fournissant une brique de base parallèle.

2.2.3.2. Politiques d'exécution

Alors que le motif définit quelle opération est effectuée, la politique d'exécution définit comment et où elle est exécutée. Elle définit le domaine d'itérations, donne des paramètres d'ordonnancement et spécifie l'espace d'exécution cible.

2.2.3.2.1. RangePolicy et MDRangePolicy

`Kokkos::RangePolicy` et `Kokkos::MDRangePolicy` décrivent les espaces d'itérations pour les boucles unidimensionnelles et les boucles parfaitement imbriquées à n dimensions. La politique `MDRangePolicy` applique automatiquement un tuilage spécifique à l'architecture pour maximiser la localité du cache ou la coalescence des accès mémoire. Nous verrons au chapitre 4 que ce tuilage doit être désactivé pour laisser libre cours au tuilage automatique du modèle polyédrique.

2.2.3.2.2. TeamPolicy

Pour des algorithmes plus complexes, les espaces d'itérations unidimensionnels ou multidimensionnels et l'ordonnancement de base sont souvent insuffisants. `Kokkos::TeamPolicy` résout ce problème en exposant le parallélisme hiérarchique, ce qui est utile pour exploiter des topologies matérielles spécifiques, en particulier sur les GPU. Il divise logiquement l'espace d'itérations en un tableau unidimensionnel de *Ligues*, où chaque ligue est constituée de plusieurs *Équipes* de threads. Lors d'une projection sur un GPU, une ligue correspond typiquement à une grille de blocs de threads, tandis qu'une équipe correspond à un bloc individuel de threads. Sur un CPU, une ligue peut correspondre aux cœurs physiques disponibles, avec des équipes utilisant des threads matériels ou des unités vectorielles. Avec cette approche plus avancée, les développeurs obtiennent un contrôle fin sur les ressources matérielles, tel que l'exploitation de la mémoire partagée gérée par l'utilisateur sur les GPU ou la gestion explicite de la mémoire cache sur les CPU, au détriment de la simplicité de développement.

Pour illustrer comment Kokkos expose des espaces d'itération multidimensionnels, la liste 4 montre une addition de matrices 2D implémentée avec une politique `Kokkos::MDRangePolicy`. Dans cet exemple, la politique définit explicitement un domaine d'itérations bidimensionnel (indiqué par `Kokkos::Rank<2>`) allant de $(0, 0)$ à (N, M). Le troisième argument de `Kokkos::parallel_for` est la fonction lambda, qui décrit le noyau de calcul lui-même. En tant qu'arguments, cette fonction lambda prend les indices d'itération i et j pour effectuer les opérations élément par élément sur les vues multidimensionnelles.

```cpp
Kokkos::View<double**> A("A", N, M);
Kokkos::View<double**> B("B", N, M);
Kokkos::View<double**> C("C", N, M);

Kokkos::parallel_for("MatrixAddition",
    Kokkos::MDRangePolicy<Kokkos::Rank<2>>({0, 0}, {N, M}),
    KOKKOS_LAMBDA(const int i, const int j) {
        C(i, j) = A(i, j) + B(i, j);
    }
);
```

Liste 4. – Exemple d'une addition de matrices parallèle utilisant une politique d'exécution `MDRangePolicy`.

2.3. Modèle polyédrique

Le modèle polyédrique [Bas12, FL11] est un outil mathématique permettant l'analyse et la transformation précises des nids de boucles. Contrairement aux compilateurs traditionnels qui appliquent des transformations locales directement sur une représentation intermédiaire (IR) ou un arbre de syntaxe abstrait, le modèle polyédrique s'appuie entièrement sur des abstractions algébriques et géométriques. Il garantit la correction sémantique des transformations appliquées. Pour représenter les nids de boucles, le modèle utilise une représentation géométrique en polyèdres des domaines d'itération, des accès mémoire, des dépendances de données et des ordonnancements d'instructions, tous exprimés par des ensembles et des relations mathématiques.

Définition 1 (Ensemble). Un *ensemble* (set) est une collection de coordonnées dans un espace unique E de dimension k. Dans le contexte du modèle polyédrique, un ensemble peut être formellement considéré comme un cas particulier de relation où l'espace d'entrée est de dimension zéro.

Un ensemble peut être représenté comme un polyèdre paramétrique restreignant les coordonnées valides :

$$S(\vec{p}) = \left\{ \vec{x} \in E \;\middle|\; A \cdot \begin{pmatrix} \vec{x} \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{1}$$

où :

- $\vec{p}$ est un vecteur de N paramètres,
- $\vec{x} \in E$ est un vecteur de coordonnées,
- A est une matrice d'entiers de taille $m \times (k + N + 1)$ qui encode les m contraintes affines définissant les frontières de l'ensemble.

Définition 2 (Relation). Une *relation* $R : E \to F$ est une correspondance depuis un ensemble de coordonnées d'entrée dans un espace d'entrée E de dimension k vers un ensemble de coordonnées de sortie dans un espace de sortie F de dimension l.

L'élément $x \in E$ est dit en relation avec $y \in F$ si $(x, y) \in R$. On le note couramment $x\ R\ y$ ou $R\ x\ y$. Une relation peut également être représentée comme un polyèdre paramétrique :

$$R(\vec{p}) = \left\{ \vec{x}_{\text{in}} \to \vec{x}_{\text{out}} \in E \times F \;\middle|\; A \cdot \begin{pmatrix} \vec{x}_{\text{out}} \\ \vec{x}_{\text{in}} \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{2}$$

où :

- $\vec{p}$ est un vecteur de N paramètres,
- $\vec{x}_{\text{in}} \in E$ est une coordonnée d'entrée,
- $\vec{x}_{\text{out}} \in F$ est une coordonnée de sortie,
- A est une matrice d'entiers de taille $m \times (k + l + N + 1)$ qui encode les m contraintes affines de la relation.

En modélisant les nids de boucles sous cette forme, le modèle polyédrique raisonne exclusivement sur des objets mathématiques, offrant une plus grande liberté pour appliquer des transformations complexes en toute sécurité. À l'inverse, les représentations traditionnelles en Abstract Syntax Tree (AST) ou en Intermediate Representation (IR) se prêtent bien aux passes de compilation standards, mais leur manque d'abstraction mathématique limite drastiquement l'application d'optimisations structurelles avancées sur les boucles. Cette rigueur formelle restreint néanmoins le domaine d'application de l'approche polyédrique : elle ne peut cibler que les nids de boucles dont les bornes et les accès mémoire sont affines, formellement regroupés sous l'appellation de *Static Control Parts* (SCoPs).

Définition 3 (Partie à contrôle statique (SCoP)). Un Static Control Part (SCoP) est une région de code sans appels de fonctions ou arithmétique de pointeurs, où les bornes des boucles, les conditions et les indices des tableaux sont soit constants, soit des fonctions affines des itérateurs des boucles englobantes et des paramètres globaux.

Au sein d'un SCoP, chaque itération de boucle peut être représentée comme un point entier à l'intérieur d'un polyèdre convexe de dimension d, où d correspond à la profondeur maximale de la boucle. Les dépendances de données sont modélisées comme des relations affines entre les vecteurs d'itération des instructions source et cible. Par conséquent, optimiser un segment de code équivaut à appliquer des transformations affines à ces espaces d'itérations

pour exposer le parallélisme ou améliorer la localité des données. De telles transformations incluent la réorganisation d'instructions, le tuilage de boucles, la torsion, et la fusion ou fission de boucles.

Pour illustrer ces concepts tout au long de cette section, la liste 5 présente un exemple simple d'un SCoP comprenant deux instructions, S_1 et S_2, imbriquées dans deux nids de boucles différents.

```cpp
for (int i = 0; i < N; i++)
  for (int j = 0; j < M; j++)
    C[i][j] = A[i][j] + B[i][j];      // Instruction S1

for (int i = 1; i < N; i++)
  for (int j = 0; j < M; j++)
    C[i][j] = C[i][j] + C[i - 1][j];  // Instruction S2
```

Liste 5. – Exemple de deux nids de boucles simples avec deux instructions.

2.3.1. Polyèdre

Définition 4 (Polyèdre/polytope rationnel). Un *polyèdre de dimension* d rationnel $\mathcal{P}$ est un sous-ensemble de $\mathbb{Q}^d$ qui peut être défini par un système de $n \in \mathbb{N}^+$ inégalités affines :

$$\mathcal{P} = \left\{ \vec{x} \in \mathbb{Q}^d \mid A \cdot \vec{x} + a \geq \vec{0} \right\} \tag{3}$$

où A est une matrice d'entiers de taille $n \times d$ et $a \in \mathbb{Z}^n$. Cette formulation est connue sous le nom de *représentation implicite* d'un polyèdre.

Étant donné qu'un polyèdre peut s'étendre indéfiniment dans certaines directions, un polyèdre convexe strictement borné est spécifiquement appelé un *polytope*.

Dans la grande majorité des applications du monde réel, les bornes des boucles ne sont pas explicitement définies par des constantes statiques. Pour résoudre ce problème, des paramètres symboliques sont introduits pour représenter les bornes dynamiques et les tailles des données. Par conséquent, le modèle polyédrique étend les polytopes standards en polytopes paramétriques.

Définition 5 (Polytope paramétrique). Un *polytope de dimension* d paramétrique $\mathcal{P}(\vec{p})$ est un polyèdre paramétrique borné défini par :

$$\mathcal{P}(\vec{p}) = \left\{ \vec{x} \in \mathbb{Q}^d \mid A \cdot \begin{pmatrix} \vec{x} \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{4}$$

où $\vec{p}$ est le *vecteur* p symbolique des paramètres, et A est une matrice d'entiers de taille $m \times (d + p + 1)$ encodant les m contraintes.

Notons que ces contraintes peuvent exprimer à la fois des inégalités et des égalités. Par exemple, la paire d'inégalités $x_i \geq 0$ et $-x_i \geq 0$ est naturellement utilisée pour représenter l'égalité stricte $x_i = 0$.

2.3.2. Instruction

Définition 6 (Instance d'instruction). Une *instance d'instruction* est une exécution spécifique d'une instruction donnée S lors d'une itération particulière de ses k boucles englobantes. Chaque instance d'instruction est identifiée de manière unique par les valeurs des itérateurs de ses boucles englobantes au moment de l'exécution.

En reprenant la liste 5, le SCoP présente deux instructions distinctes imbriquées dans deux nids de boucles. Dans ce contexte, une instance de l'instruction S_1 ou S_2 se produit pour chaque combinaison valide des itérateurs i et j.

2.3.3. Domaine d'itération

Une instruction est associée à un ensemble d'instances d'instruction bornées à l'intérieur d'un polytope. Chaque instance individuelle est caractérisée de manière unique par un vecteur composé des valeurs des itérateurs de ses boucles englobantes.

Définition 7 (Vecteur d'itération). Un *vecteur d'itération* $\vec{x}$ est un vecteur colonne de dimension k qui représente les valeurs des k itérateurs de boucle caractérisant une exécution spécifique d'une instruction S. Il est défini comme :

$$\vec{x} = \begin{pmatrix} x_1 \\ x_2 \\ \cdots \\ x_k \end{pmatrix} \tag{5}$$

où x_i est l'indice de la boucle à la profondeur i, et k est la profondeur totale du nid de boucles englobant l'instruction.

L'ensemble exhaustif de tous les vecteurs d'itération pour lesquels une instruction donnée est exécutée est appelé le domaine d'itérations de cette instruction.

Définition 8 (Domaine d'itérations). Le domaine d'itérations d'une instruction S est l'ensemble de tous les vecteurs d'itération $\vec{x}$ possibles pour lesquels l'instruction S est exécutée. Il peut être représenté comme un polyèdre convexe défini par un système d'inégalités affines. Cet ensemble peut être représenté par les points entiers d'un polytope paramétrique :

$$\mathcal{D}_S(\vec{p}) = \left\{ \vec{x} \in \mathbb{Z}^k \mid A \cdot \begin{pmatrix} \vec{x} \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{6}$$

où $\vec{p}$ est un vecteur de N paramètres, $\vec{x}$ est un vecteur d'itération de dimension k, et A est une matrice d'entiers de taille $m \times (k + N + 1)$ qui encode les m contraintes du polytope.

La figure 1 illustre le domaine d'itération de l'instruction S_1 dérivé du SCoP de la liste 5. Le domaine d'itération de S_1 est représenté géométriquement comme un polyèdre convexe dans l'espace bidimensionnel défini par les indices de boucle i et j. Chaque point entier à l'intérieur de ce polyèdre correspond à une instance unique de l'instruction, tandis que les limites du polyèdre sont strictement déterminées par les bornes des boucles et par toute instruction conditionnelle présente dans le code.

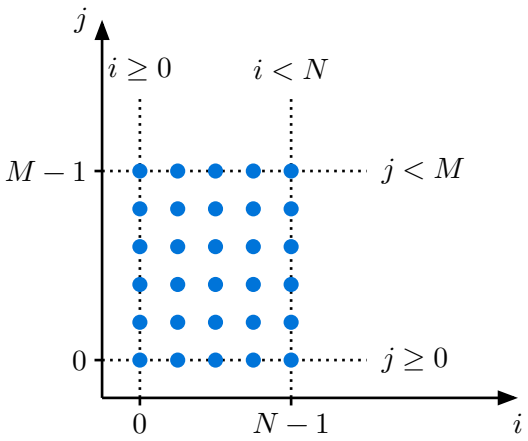


Figure 1. – Représentation géométrique du domaine d'itération pour l'instruction S_1.

2.3.4. Dépendances de données

Pour que le modèle polyédrique génère des transformations valides produisant exactement les mêmes résultats que le code original, il doit respecter strictement les dépendances de données initiales du programme. Les dépendances de données agissent comme des contraintes qui restreignent l'ordre d'exécution légal des instances d'instruction. Elles sont introduites lorsque plusieurs instances d'instruction accèdent aux mêmes emplacements mémoire.

Définition 9 (Dépendance de données). Deux instances d'instruction $S_1(\vec{x}_1)$ et $S_2(\vec{x}_2)$ sont dites dépendantes si les deux instances accèdent exactement au même emplacement mémoire, qu'au moins l'un des accès est une opération d'écriture, et qu'une instance s'exécute avant l'autre dans l'ordre original du programme.

Il existe trois principaux types de dépendances de données qui restreignent le réordonnancement des instructions :

- **Lecture-Après-Écriture ou dépendance de flot :** Une instruction source écrit dans un emplacement mémoire qui est ensuite lu par une instruction cible.
- **Écriture-Après-Lecture ou anti-dépendance :** Une instruction source lit un emplacement mémoire avant qu'il ne soit écrasé par une instruction cible.
- **Écriture-Après-Écriture ou dépendance de sortie :** Une instruction source écrit dans un emplacement mémoire qui est ensuite écrasé par une instruction cible.

Le modèle polyédrique représente géométriquement ces dépendances de données comme des relations affines entre les vecteurs d'itération des instructions source et cible. L'analyse de flot de données proposée par Feautrier [Fea91] permet de calculer exactement ces relations pour les programmes à contrôle statique.

Définition 10 (Relation de dépendance). Les dépendances entre les instances d'une instruction source S et d'une instruction cible T peuvent être représentées comme des relations entre des vecteurs d'itération. Un couple de points entiers dans le polyèdre associé à la relation représente une dépendance entre les vecteurs d'itération source et cible correspondants. Cette relation peut être représentée par le polytope paramétrique suivant :

$$\delta_{S,T}(\vec{p}) = \left\{ \vec{x}_S \rightarrow \vec{x}_T \mid R_{S,T} \cdot \begin{pmatrix} \vec{x}_S \\ \vec{x}_T \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{7}$$

où $R_{S,T}$ est une matrice d'entiers de taille $m \times (k + l + N + 1)$, avec m le nombre de contraintes, $k = \dim(\vec{x}_S)$ la profondeur de l'instruction source, $l = \dim(\vec{x}_T)$ la profondeur de l'instruction cible et $N = \dim(\vec{p})$ le nombre de paramètres.

En reprenant l'exemple de la liste 5, nous pouvons observer une dépendance de flot sur l'instruction S_2 à travers les itérations de la boucle externe. Spécifiquement, l'instruction S_2 à l'itération (i', j') lit la valeur `C[i' - 1][j']`, qui a été précédemment écrite par S_2 à l'itération (i, j) où $i = i' - 1$ et $j = j'$. Cette dépendance de flot portée par la boucle peut être formellement exprimée en utilisant notre représentation matricielle :

$$\begin{aligned} \delta_{S_2,S_2}(\vec{p}) &= \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \rightarrow \begin{pmatrix} i' \\ j' \end{pmatrix} \;\middle|\; i = i' - 1 \text{ et } j = j' \right\} \\ &= \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \rightarrow \begin{pmatrix} i' \\ j' \end{pmatrix} \;\middle|\; \begin{pmatrix} 1 & 0 & -1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & -1 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} i \\ j \\ i' \\ j' \\ N \\ M \\ 1 \end{pmatrix} = \vec{0} \right\} \end{aligned} \tag{8}$$

2.3.5. Ordonnancement

Un ordonnancement dicte l'ordre d'exécution des instances d'instruction au sein de l'espace d'itération. Dans le modèle polyédrique, cet ordonnancement est représenté comme une relation affine associant au vecteur d'itération d'une instruction un vecteur de date logique multidimensionnelle. Le calcul automatique de tels ordonnancements affines respectant les dépendances a été formalisé par Feautrier, d'abord pour des dates unidimensionnelles [Fea92a] puis multidimensionnelles [Fea92b].

Définition 11 (Relation d'ordonnancement). Étant donné une instruction S, une *relation d'ordonnancement* θ_S détermine l'ordre d'exécution de ses instances. Pour ce faire, elle associe à chaque instance $\vec{x}$ d'une instruction S un *temps d'exécution logique* (ou *date logique*) $\vec{t}$:

$$\theta_S(\vec{p}) = \left\{ \vec{x} \to \vec{t} \;\middle|\; T \cdot \begin{pmatrix} \vec{x} \\ \vec{t} \\ \vec{p} \\ 1 \end{pmatrix} \geq \vec{0} \right\} \tag{9}$$

où $\vec{p}$ est un vecteur de N paramètres, $\vec{x}$ est un vecteur d'itération de dimension k de l'instruction S, $\vec{t}$ est un vecteur d'ordonnancement logique de dimension d, et T est une matrice d'entiers de taille $m \times (k + d + N + 1)$ qui encode les m contraintes affines du polyèdre.

L'abstraction d'ordonnancement permet aux compilateurs de raisonner sur le temps en utilisant des dates logiques multidimensionnelles plutôt que des ordres d'exécution linéaires explicites. Pour comparer l'ordre d'exécution de deux instances d'instruction sur la base de leurs dates logiques, le modèle s'appuie sur l'ordre lexicographique.

Définition 12 (Ordre lexicographique). Étant donné deux vecteurs d'itération $\vec{x}$ et $\vec{x}'$ de même dimension, l'ordre lexicographique $\preceq$ est défini par :

$$\begin{pmatrix} x_1 \\ x_2 \\ \dots \\ x_n \end{pmatrix} \preceq \begin{pmatrix} x'_1 \\ x'_2 \\ \dots \\ x'_n \end{pmatrix} \iff \exists k, 1 \leq k \leq n : \left(\forall j : 1 \leq j < k \mid x_j = x'_j\right) \wedge x_k \leq x'_k$$

$$\iff \begin{cases} x_1 \leq x'_1 \\ \text{ou} \\ x_1 = x'_1 \wedge x_2 \leq x'_2 \\ \text{ou} \\ \dots \\ \text{ou} \\ (\forall i \in [\![1, n[\![,\ \ x_i = x'_i) \wedge x_n \leq x'_n \end{cases} \tag{10}$$

En revenant à notre exemple de la liste 5, l'ordonnancement d'exécution original projette le domaine d'itération 2D vers un espace de temps logique 3D. La première dimension (t_0) encode l'ordre lexical des instructions, tandis que les dimensions restantes encodent les itérateurs. Les relations d'ordonnancement correspondantes sont :

$$\theta_{S_1}(\vec{p}) = \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \to \begin{pmatrix} t_0 \\ t_1 \\ t_2 \end{pmatrix} \;\middle|\; t_0 = 0 \text{ et } t_1 = i \text{ et } t_2 = j \right\}$$

$$= \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \to \begin{pmatrix} t_0 \\ t_1 \\ t_2 \end{pmatrix} \;\middle|\; \begin{pmatrix} 0 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} i \\ j \\ t_0 \\ t_1 \\ t_2 \\ N \\ M \\ 1 \end{pmatrix} = \vec{0} \right\} \tag{11}$$

$$\theta_{S_2}(\vec{p}) = \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \to \begin{pmatrix} t_0 \\ t_1 \\ t_2 \end{pmatrix} \;\middle|\; t_0 = 1 \text{ et } t_1 = i \text{ et } t_2 = j \right\}$$

$$= \left\{ \begin{pmatrix} i \\ j \end{pmatrix} \to \begin{pmatrix} t_0 \\ t_1 \\ t_2 \end{pmatrix} \;\middle|\; \begin{pmatrix} 0 & 0 & -1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & -1 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} i \\ j \\ t_0 \\ t_1 \\ t_2 \\ N \\ M \\ 1 \end{pmatrix} = \vec{0} \right\} \tag{12}$$

En transformant le domaine spatial 2D en un domaine temporel 3D, l'ordonnancement isole l'ordre d'exécution. Ici, S_1 est toujours exécutée avant S_2 à toute itération donnée car sa dimension temporelle externe est statiquement plus petite ($t_0 = 0$ pour S_1, tandis que $t_0 = 1$ pour S_2). Au sein de leurs blocs respectifs, les deux instructions sont exécutées séquentiellement en suivant le vecteur d'itération original (i, j) puisque t_1 et t_2 reflètent exactement les itérateurs de la boucle.

2.4. L'écosystème polyédrique

Bien que les abstractions mathématiques du modèle polyédrique offrent un cadre de travail puissant pour l'optimisation des boucles, l'application automatique de ces transformations à des programmes du monde réel nécessite des infrastructures logicielles robustes. Au cours des dernières décennies, la communauté de la compilation a développé un écosystème riche

pour manipuler les représentations polyédriques, analyser les dépendances et générer du code optimisé.

Cette section explore les composants principaux de cet écosystème, en les catégorisant en outils polyédriques et en compilateurs.

2.4.1. Outils polyédriques

L'application du modèle polyédrique repose fortement sur des bibliothèques mathématiques capables de résoudre des systèmes complexes d'inégalités affines en nombres entiers.

2.4.1.1. Integer Set Library (ISL)

Pour manipuler les polyèdres, la bibliothèque *Integer Set Library* (ISL), développée par Sven Verdoolaege [Ver10], est largement utilisée dans la communauté polyédrique. ISL est une bibliothèque en C conçue pour manipuler des ensembles et des relations de points entiers bornés par des contraintes affines. Elle manipule deux structures principales :

- **Sets :** Utilisés pour représenter les domaines d'itération.
- **Maps :** Utilisées pour représenter les fonctions d'accès, les dépendances et les ordonnancements en associant les éléments d'un ensemble à ceux d'un autre.

Integer Set Library (ISL) fournit des implémentations hautement optimisées pour les opérations polyédriques essentielles, y compris l'intersection, l'union, la différence d'ensembles et le calcul de minimums ou maximums lexicographiques.

Au-delà des opérations d'ensembles de base, l'une des fonctionnalités d'ISL est son **moteur d'ordonnancement** intégré. Basée sur une variante de l'algorithme du compilateur polyédrique Pluto [Bon+08b], ISL peut calculer automatiquement des ordonnancements affines qui respectent toutes les dépendances de données tout en améliorant la localité des données et en exposant du parallélisme.

Enfin, une fois qu'un ordonnancement a été calculé, ISL dispose d'un générateur d'AST, successeur de CLooG [Bas04], l'outil historique de génération de code polyédrique. Ce composant traduit la représentation polyédrique transformée en une structure classique de nids de boucles. Parce qu'elle encapsule la totalité de la chaîne de compilation mathématique,

ISL sert de moteur fondamental pour la quasi-totalité des compilateurs polyédriques modernes. La figure 2 illustre comment un nid de boucles C standard est modélisé mathématiquement à l'aide de la syntaxe des Sets et des Maps d'ISL.

```c
for (int i = 0; i < N; i++)
  for (int j = 0; j < M; j++)
    A[i][j] = B[i] + C[j]; // S1
```

(a) Code source en entrée.

```
// Domaine de l'ensemble d'itération
[N, M] -> { S1[i, j] : 0 <= i < N and 0 <= j < M }

// Ordonnancement d'exécution (Map)
[N, M] -> { S1[i, j] -> [i, j] }

// Accès mémoire (Maps)
[N, M] -> { S1[i, j] -> A[i, j] } // Écriture
[N, M] -> { S1[i, j] -> B[i] }    // Lecture
[N, M] -> { S1[i, j] -> C[j] }    // Lecture
```

(b) Représentation ISL du domaine d'itérations, de l'ordonnancement et des fonctions d'accès pour le code donné en exemple.

Figure 2. – Exemple d'un nid de boucles simple et de sa représentation ISL correspondante.

2.4.1.2. Représentation standardisée OpenScop

OpenScop [Bas] est une spécification ouverte conçue pour garantir l'interopérabilité en permettant à différents outils polyédriques d'échanger une représentation standardisée d'un SCoP. Elle modélise les systèmes mathématiques d'inégalités affines (domaines, fonctions d'accès et ordonnancements) à l'aide d'un format matriciel structuré, où les colonnes correspondent aux itérateurs de boucles, aux paramètres globaux et aux termes constants, et où les lignes représentent les contraintes affines. La figure 3a montre un nid de boucles simple, tandis que sa représentation OpenScop correspondante est détaillée à la figure 3b.

```c
for (int i = 0; i < N; i++)
  for (int j = 0; j < M; j++)
    A[i][j] = 0;               // Instruction S1
```

(a) Code source en entrée.

```
<OpenScop>
...
DOMAIN
6 6 2 0 0 2
# e/i|  i    j |  N    M |  1
    1    1    0    0    0    0    ## i >= 0
    1   -1    0    1    0   -1    ## -i+N-1 >= 0
    1    0    0    1    0   -1    ## N-1 >= 0
    1    0    1    0    0    0    ## j >= 0
    1    0   -1    0    1   -1    ## -j+M-1 >= 0
    1    0    0    0    1   -1    ## M-1 >= 0
# ------------------------------------------------  1.2 Scattering
SCATTERING
5 11 5 2 0 2
# e/i| c1   c2   c3   c4   c5 |  i    j |  N    M |  1
    0   -1    0    0    0    0    0    0    0    0    0    ## c1 == 0
    0    0   -1    0    0    0    1    0    0    0    0    ## c2 == i
    0    0    0   -1    0    0    0    0    0    0    0    ## c3 == 0
    0    0    0    0   -1    0    0    1    0    0    0    ## c4 == j
    0    0    0    0    0   -1    0    0    0    0    0    ## c5 == 0
# ------------------------------------------------  1.3 Access
WRITE
3 9 3 2 0 2
# e/i| Arr  [1]  [2]|  i    j |  N    M |  1
    0   -1    0    0    0    0    0    0    5    ## Arr == A
    0    0   -1    0    1    0    0    0    0    ## [1] == i
    0    0    0   -1    0    1    0    0    0    ## [2] == j
...
</OpenScop>
```

(b) Représentation OpenScop du domaine d'itérations, de l'ordonnancement (scattering) et des fonctions d'accès pour le code donné en exemple.

Figure 3. – Exemple d'un nid de boucles simple et de sa représentation OpenScop correspondante.

Pour faciliter l'intégration, elle est accompagnée de l'OpenScop Library (OSL), une API C utilisée pour générer, lire et manipuler facilement cette représentation. De plus, l'architecture extensible d'OpenScop prend en charge diverses extensions spécifiques aux outils, permettant aux compilateurs d'intégrer des métadonnées personnalisées sans rompre la compatibilité.

2.4.2. Compilateurs polyédriques

La communauté de recherche a développé divers compilateurs pour automatiser les optimisations de boucles. Bien qu'ils partagent tous la même base théorique, ils ciblent différents niveaux de la pile de compilation et diverses architectures matérielles. Parmi les compilateurs et outils polyédriques les plus notables, on trouve :

- **Pluto [Bon+08a] :** Un compilateur C source-à-source renommé pour son algorithme d'ordonnancement, qui calcule automatiquement les transformations affines pour maximiser simultanément la localité des données et exposer le parallélisme pour les CPU multicœurs.
- **PPCG (Polyhedral Parallel Code Generator) [Ver+13] :** Un compilateur source-à-source conçu spécifiquement pour les architectures hétérogènes, transformant les nids de boucles C séquentiels en code CUDA ou OpenCL optimisé pour l'exécution sur GPU.
- **Apollo (Automatic speculative POLyhedral Loop Optimizer) [Caa+17] :** Un outil qui étend le modèle statique traditionnel en appliquant des transformations dynamiquement à l'exécution, permettant l'optimisation des nids de boucles avec des accès mémoire non résolus statiquement ou un flux de contrôle dépendant des données.
- **Polygeist [Mos21] :** Un *frontend* C/C++ moderne et une infrastructure d'optimisation construite au-dessus de MLIR (Multi-Level Intermediate Representation), qui exploite le dialecte affine pour effectuer des transformations polyédriques au sein d'une chaîne de compilation LLVM.
- **LLVM Polly [Gro+11] :** Un optimiseur de boucles intégré dans l'infrastructure du compilateur LLVM qui opère directement sur la représentation intermédiaire, s'abstrayant du langage source pour effectuer des optimisations avancées.

Bien que chacun de ces outils optimise avec plus ou moins de succès les performances des codes transformés, les travaux présentés dans cette thèse s'appuient principalement sur l'infrastructure LLVM, en utilisant Polly pour intercepter et optimiser les codes Kokkos. Par conséquent, la section suivante propose une plongée approfondie dans l'architecture et la chaîne de compilation de LLVM et de son extension Polly.

2.5. Plongée au cœur de LLVM Polly

Polly [Gro11, GGL12] est un outil d'analyse et d'optimisation polyédrique de boucles de bas niveau, intégré dans l'optimiseur *middle-end* de LLVM. Un compilateur moderne tel que LLVM [LA04] s'organise en trois étages : le *frontend* (Clang pour le C++) analyse le code source et le traduit en IR, le *middle-end* applique sur cette IR les optimisations indépendantes de la machine cible et le *backend* génère le code machine. Polly peut être invoqué nativement via le *frontend* du compilateur Clang (en utilisant des arguments de ligne de commande comme `-O3 -mllvm -polly`). Un avantage stratégique clé de Polly est sa dépendance à l'IR de LLVM. En opérant strictement au niveau de l'IR, Polly est complètement découplé du langage source, lui permettant d'optimiser les boucles indépendamment de la syntaxe du code source original.

Cette architecture basée sur l'IR est particulièrement avantageuse lors du ciblage de bibliothèques parallèles modernes de haut niveau telles que Kokkos. Kokkos s'appuie fortement sur des fonctionnalités C++ avancées, incluant la métaprogrammation par templates, les fonctions lambda et les abstractions d'objets complexes, qui sont impossibles à analyser et à parser pour les compilateurs polyédriques source-à-source traditionnels. En se positionnant dans le *middle-end*, Polly n'intercepte le code qu'une fois que le *frontend* Clang a complètement instancié les templates et résolu les abstractions de haut niveau. Polly contourne ainsi la complexité syntaxique du code source C++ original.

Cependant, Polly ne peut analyser efficacement qu'une représentation « nettoyée » et canonicalisée, c'est-à-dire réécrite sous une forme normalisée où chaque construction (boucles, calculs d'indices, accès mémoire) suit un motif unique reconnu par ses passes d'analyse, et où les nids de boucles multidimensionnels et les accès mémoire sont explicitement exposés. Cette forme n'est pas toujours obtenue : les abstractions de Kokkos laissent dans l'IR des traces (indirections, appels de fonctions, accès linéarisés) qui masquent la structure des boucles et empêchent sa détection. Obtenir cette représentation canonique constitue précisément l'un des enjeux de cette thèse.

2.5.1. Architecture et intégration dans la chaîne de compilation

Au sein de l'infrastructure du compilateur LLVM, les optimisations sont appliquées comme une séquence de passes orchestrées par le gestionnaire de passes. Polly s'intègre nativement dans cette chaîne de compilation *middle-end*, et son point exact d'exécution peut être contrôlé via l'argument de ligne de commande `-polly-position`.

Par défaut et utilisé lors de tous les travaux expérimentaux présentés dans cette thèse, Polly est inséré à la position `before-vectorizer`. Ce placement spécifique est hautement stratégique. Avant même que Polly n'inspecte le code, l'IR a déjà été fortement optimisée et canonicalisée par les passes standards de LLVM. Des passes telles que `mem2reg`, qui promeut les allocations mémoire en registres Static Single Assignment (SSA), `simplifycfg`, qui nettoie le graphe de flot de contrôle, ou encore l'*inlining* des fonctions ont déjà supprimé le surcoût lié à l'abstraction C++ de haut niveau. Par conséquent, Polly opère sur des structures de boucles propres et normalisées et fournit sa sortie hautement optimisée et parallélisable directement au vectoriseur natif de LLVM.

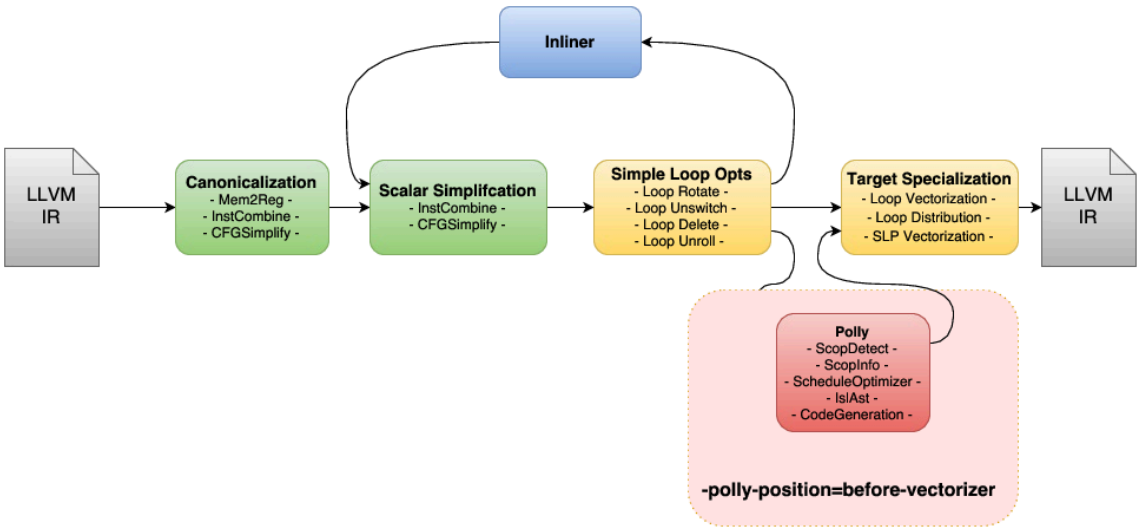


Figure 4. – Intégration des passes de Polly au sein de la chaîne d'optimisation *middle-end* de LLVM à la position `before-vectorizer`. (Source : Documentation de LLVM Polly)

Une fois invoqué, Polly exécute sa propre chaîne de compilation interne spécialisée. Ce système présente le flux de travail polyédrique en une séquence stricte de passes LLVM séquentielles, comme illustré par la figure 4 :

- **`CodePreparation` :** Prépare l'IR en scindant le bloc d'entrée afin d'isoler les allocations mémoire locales (`alloca`) des opérations arithmétiques et des accès aux données, préparant ainsi le code à l'analyse et à la future génération de code.
- **`ScopDetect` :** Analyse le graphe de flot de contrôle pour identifier les régions valides à Entrée Unique et Sortie Unique (Single-Entry Single-Exit, SESE) qui sont appropriées pour la représentation polyédrique dans Polly.
- **`ScopInfo` :** Extrait les instructions de l'IR des régions valides et les traduit en représentations mathématiques ISL (domaines, accès, dépendances de données, et ordonnancement original).
- **`ScheduleOptimizer` :** Invoque le moteur d'ordonnancement intégré d'ISL pour calculer des transformations affines qui améliorent la localité des données et exposent du parallélisme.
- **`IslAst` :** Génère un nouvel AST ISL représentant la structure du nid de boucles ordonnancé de manière optimale.
- **`CodeGeneration` :** Parcourt l'AST ISL pour construire l'IR LLVM finale et optimisée, en y intégrant des vérifications à l'exécution (*runtime checks*) pour l'analyse d'aliasing et des bornes de tableaux lorsque cela est nécessaire.

Les mécanismes détaillés de chacune de ces passes internes sont présentés dans les sous-sections suivantes.

2.5.1.1. Préparation du code (`Code Preparation`)

Avant d'identifier les régions polyédriques, Polly exécute une étape préparatoire via la passe `CodePreparation`. Dans son fonctionnement natif, son rôle est de scinder le bloc d'entrée de la fonction afin d'isoler les allocations mémoire locales (`alloca`) des opérations arithmétiques et des accès aux données. Cette séparation structurelle évite que les instructions d'allocation de pile ne polluent les blocs de calcul et réserve l'espace nécessaire à l'insertion de nouvelles allocations lors de la phase de régénération du code.

2.5.1.2. Détection des SCoPs (`SCoP Detection`)

À la suite des transformations préparatoires, la passe `ScopDetect` est chargée d'identifier les segments de l'IR de LLVM qui peuvent être légalement optimisés en utilisant le modèle polyédrique. Polly opère sur le graphe de flot de contrôle (CFG) pour isoler les régions SESE maximales.

Pour qu'une région SESE soit validée comme un SCoP, Polly applique des critères d'acceptation stricts. Il s'appuie fortement sur l'analyse d'évolution scalaire (Scalar Evolution, SCEV) de LLVM pour inspecter les variables d'induction des boucles, les bornes et les branchements conditionnels. La région n'est acceptée que si `ScopDetect` peut prouver de manière formelle que toutes les bornes des boucles et les conditions de flot de contrôle sont purement des expressions affines.

Cette passe effectue une vérification rigoureuse de la légalité et de la sécurité. La région SESE est immédiatement rejetée si elle contient des instructions avec des effets de bord inconnus (comme des appels de fonctions externes ou non inlinés), des indices de tableaux non affines, ou un aliasing de pointeurs complexe qui ne peut pas être résolu statiquement.

2.5.1.3. Construction des SCoPs (`SCoP Building`)

Une fois qu'une région SESE valide est détectée avec succès, la passe `ScopInfo` est exécutée pour traduire l'IR de LLVM vers les abstractions mathématiques du modèle polyédrique. Ce processus implique de traduire les structures de code en objets ISL.

Pour chaque bloc de base au sein de la région, `ScopInfo` définit une instruction mathématique. Ensuite, il construit le **domaine d'itération** exact en traduisant les contraintes affines des boucles englobantes obtenues grâce à la passe SCEV (*Scalar Evolution*) en structure ISL. De même, les instructions mémoire (telles que `load` et `store`) sont converties en relations d'accès ISL, associant l'exécution logique d'une instruction à des adresses mémoire spécifiques.

Cette phase extrait également l'ordonnancement original du programme non modifié et effectue une analyse des dépendances de données. À la fin de cette passe, Polly a construit une représentation mathématique complète du nid de boucles, entièrement détachée de l'IR de LLVM, prête à être optimisée par l'ordonnanceur polyédrique.

Toutefois, pour garantir la validité de cette représentation et capturer des dépendances de données exactes, Polly doit préalablement reconstruire la structure originelle des accès mémoire : c'est l'étape de délinéarisation.

2.5.1.3.1. Délinéarisation des accès mémoire

L'un des défis majeurs lors de l'extraction des SCoPs réside dans la traduction des instructions mémoire de bas niveau de l'IR de LLVM vers des structures de tableaux multidimensionnels. Ce processus, appelé délinéarisation, est absolument nécessaire pour pouvoir exprimer des fonctions d'accès affines et effectuer les transformations polyédriques associées.

En effet, lors de la génération de l'IR, les accès aux tableaux multidimensionnels sont linéarisés en une arithmétique de pointeurs unidimensionnelle. La liste 6 montre un exemple où l'accès au tableau tridimensionnel `A[i][j][k]` est compilé avec un décalage mémoire calculé par l'expression mathématique $i \times M \times P + j \times P + k$.

```cpp
void init(int N, int M, int P, double *A) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < M; j++)
            for (int k = 0; k < P; k++)
                A[i * M * P + j * P + k] = 0.0;
}
```

Liste 6. – Exemple de code source en C avec un accès multidimensionnel 3D.

Si les tailles des dimensions M et P sont dynamiques, cette expression $i \times M \times P + j \times P + k$ devient non affine car elle implique la multiplication de variables d'induction par des paramètres, ce qui briserait les conditions de validité du modèle polyédrique.

Pour pallier ce problème, Polly s'appuie sur un algorithme, détaillé dans [Gro+15], qui se base sur l'analyse d'évolution scalaire SCEV. L'algorithme analyse l'expression de l'adresse mémoire calculée et factorise les pas d'accès. Dans notre exemple, il observe que l'adresse avance d'un pas de $M \times P \times 8$ à chaque itération de la boucle externe i, d'un pas de $P \times 8$ pour la boucle intermédiaire j, et d'un pas de 8 pour la boucle interne k (8 correspondant à la taille en octets d'un type `double`).

```llvm
{{{ %A, +, (8*%m*%p)}<%for.i>, +, (8*%p)}<%for.j>, +, 8}<%for.k>
```

Liste 7. – Représentation de l'évolution scalaire de l'adresse mémoire de `A[i][j][k]` issue de la liste 6.

Comme l'illustre la liste 7, cette représentation compacte met en évidence les pas d'accès associés à chaque boucle. Pour reconstruire les accès multidimensionnels, l'algorithme de délinéarisation procède à des opérations successives de division euclidienne sur le polynôme de l'adresse mémoire. Ces divisions utilisent les tailles identifiées pour chaque dimension (appelées *terms*), en remontant de la dimension la plus interne vers la plus externe. Dans notre exemple, les *terms* successifs extraits sont 8, P, et M.

Soit S l'expression de l'adresse mémoire linéarisée, qui correspond au polynôme capturé par la SCEV :

$$S(i,j,k) = i \times M \times P \times 8 + j \times P \times 8 + k \times 8 \tag{13}$$

La séparation des dimensions s'effectue alors étape par étape :

Itération 1 (Extraction de la taille de l'élément, Term 8) :

La première étape consiste à extraire la taille du type de donnée en divisant le polynôme global par 8 :

- Quotient : $Q_1 = \left\lfloor \frac{S}{8} \right\rfloor = \left\lfloor \frac{i \times M \times P \times 8 + j \times P \times 8 + k \times 8}{8} \right\rfloor = i \times M \times P + j \times P + k$

Itération 2 (Dimension interne k, Term P) :

Le nouveau polynôme Q_1 est ensuite divisé par la dimension suivante (P) pour en déduire l'indice le plus interne (k) :

- Indice : $\text{Index}_0 = Q_1 \bmod P = (i \times M \times P + j \times P + k) \bmod P = k$
- Quotient : $Q_2 = \left\lfloor \frac{Q_1}{P} \right\rfloor = \left\lfloor \frac{i \times M \times P + j \times P + k}{P} \right\rfloor = i \times M + j$

Itération 3 (Dimension intermédiaire j, Term M) :

Enfin, on divise le quotient restant Q_2 par la dernière dimension (M) pour isoler les indices restants :

- Indice : $\text{Index}_1 = Q_2 \bmod M = (i \times M + j) \bmod M = j$
- Quotient (Dimension externe i) : $\text{Index}_2 = \left\lfloor \frac{Q_2}{M} \right\rfloor = \left\lfloor \frac{i \times M + j}{M} \right\rfloor = i$

Grâce à cette méthode itérative, Polly réussit à séparer les tailles des différentes dimensions et à extraire les indices d'accès (i, j, k). Cette étape de reconstruction est primordiale car elle restaure la vue multidimensionnelle et affine des tableaux, indispensable au modèle

polyédrique. Toutefois, nous montrons à la section 4.3.4 qu'une telle délinéarisation échoue (ou est insuffisante) lorsque l'accès s'effectue avec un indice constant nul (0).

2.5.1.4. Optimisation des SCoPs (`SCoP Optimization`)

Avec la représentation mathématique entièrement construite, la passe `ScheduleOptimizer` délègue la charge principale de l'optimisation au moteur d'ordonnancement intégré d'ISL. L'objectif premier du solveur est de calculer un nouvel ordonnancement qui redéfinit l'ordre d'exécution des instances d'instructions. L'ordonnanceur recherche des transformations qui minimisent la distance de réutilisation des accès mémoire et exposent du parallélisme. Le moteur garantit mathématiquement l'équivalence sémantique du programme : tout ordonnancement calculé doit strictement respecter les dépendances de données exactes extraites lors de la phase précédente.

La sortie de cette passe est un arbre d'ordonnancement optimisé. En isolant les dimensions de boucles qui sont complètement dépourvues de dépendances cycliques portées par la boucle, le solveur prouve mathématiquement l'absence de dépendances de données. Ce parallélisme exposé guide directement les phases ultérieures de génération du code pour appliquer en toute sécurité la vectorisation ou le multithreading.

2.5.1.5. Génération de l'AST (`IslAst`)

En prenant l'arbre d'ordonnancement nouvellement optimisé comme entrée, la passe `IslAst` génère, à l'aide du générateur d'AST d'ISL [GVC15], un AST qui représente logiquement la structure du nid de boucles transformé. À ce stade, la passe analyse en profondeur les propriétés mathématiques du nouvel ordonnancement pour intégrer des directives d'exécution dans les nœuds de l'AST.

Spécifiquement, si le solveur ISL a prouvé mathématiquement que certaines dimensions de boucles sont complètement dépourvues de dépendances portées par la boucle, les nœuds correspondants de l'AST sont explicitement annotés comme étant parallèles. De même, les boucles internes sont annotées comme vectorisables. Ces annotations sémantiques sont fondamentales, car elles servent de directives pour guider la phase de génération de code,

permettant d'activer le multithreading et les instructions Single Instruction Multiple Data (SIMD).

2.5.1.6. Génération de code (`Code Generation`)

La passe `CodeGeneration` est chargée de reconstruire l'IR LLVM finale et optimisée à partir du nouvel AST ISL généré. En parcourant les nœuds de l'AST, cette passe utilise l'`IRBuilder` de LLVM pour construire les structures de boucles correspondantes et le graphe de flot de contrôle. Si les nœuds de l'AST parcourus portent les annotations parallèles ou vectorielles intégrées lors de la phase précédente, le générateur de code les traduit en appels appropriés de la bibliothèque d'exécution OpenMP [Rag11] ou en directives SIMD.

Plutôt que de générer les instructions de calcul à partir de zéro, le générateur réutilise intelligemment le code d'origine. Il copie les anciennes instructions de l'IR de LLVM, en tant qu'opérations mathématiques et logiques de traitement de données sauvegardées lors de la passe `ScopInfo`, et les injecte dans les corps des nouvelles boucles construites. Pendant ce processus de copie, les instructions mémoire (`load` et `store`) sont mises à jour. Les indices de leurs tableaux et l'arithmétique des pointeurs sont réécrits avec les nouvelles fonctions d'accès affines et les nouvelles variables d'induction de boucle.

Enfin, pour garantir une correction sémantique absolue, Polly emploie un mécanisme de versionnage de boucle. Étant donné que l'analyse statique ne peut pas toujours prouver définitivement l'absence d'aliasing de pointeurs ou d'accès hors limites à la compilation, le générateur de code construit si nécessaire un bloc de vérifications de sécurité à l'exécution. Le nid de boucles original et non modifié est préservé dans l'IR en tant que chemin de secours. À l'exécution, si ces vérifications de sécurité échouent, l'exécution bifurque dynamiquement vers le code original, s'assurant que les transformations polyédriques ne compromettent jamais la validité du programme.

2.5.2. Limites de Polly

Bien que LLVM Polly fournisse un mécanisme pour l'optimisation des boucles, son choix architectural d'opérer exclusivement au niveau de l'IR introduit plusieurs limites inhérentes.

Ces défis sont particulièrement marqués lors de l'analyse de code C++ fortement abstrait comme avec Kokkos.

- **Le fossé sémantique et la reconstruction de l'information :** Opérer sur l'IR de LLVM signifie que toutes les constructions du langage de haut niveau ont été abaissées et aplanies. Pour appliquer le modèle polyédrique, Polly doit reconstituer artificiellement la structure originale du programme à partir d'instructions de bas niveau. Cela implique la récupération des structures de tableaux multidimensionnels, la reconstruction des hiérarchies de boucles, et le regroupement logique des instructions en déclarations mathématiques. Si le code C++ original s'appuie sur des abstractions complexes, ce processus de reconstruction devient hautement fragile, provoquant l'échec de Polly à reconnaître des SCoPs valides.
- **Manque de prise en charge des GPU :** Polly est conçu pour optimiser la localité des données et le parallélisme pour les CPU multicœurs (via OpenMP et la vectorisation SIMD). Bien que des extensions expérimentales comme Polly-ACC [GH16] aient été historiquement développées pour générer du code GPU, elles ne sont plus activement maintenues dans les versions récentes de l'infrastructure LLVM. Par conséquent, Polly manque nativement de la capacité robuste à générer du code GPU optimisé pour les architectures hétérogènes modernes.

Ces deux limites, structurelles et matérielles, mettent en évidence l'incompatibilité entre l'approche descendante classique de Polly et les exigences des bibliothèques de portabilité des performances :

D'une part, le choix d'opérer exclusivement au niveau d'une représentation intermédiaire de bas niveau crée un fossé sémantique majeur face aux abstractions C++ modernes.

D'autre part, le fait que la génération de code de Polly soit nativement restreinte aux seuls processeurs multicœurs (via OpenMP) entre en contradiction directe avec le modèle d'exécution de Kokkos, conçu pour cibler des architectures hétérogènes et exploiter les accélérateurs matériels (GPU).

Ainsi, bien que le modèle polyédrique offre des garanties formelles puissantes pour améliorer la localité des données et paralléliser les boucles, son utilisation directe se heurte

à un verrou double : une incapacité à percevoir la structure des calculs issus de Kokkos au niveau de l'IR, et une incapacité à générer du code pour les accélérateurs hétérogènes ciblés. Surmonter ce double verrou en conciliant l'expressivité de Kokkos avec les capacités d'optimisation de Polly constitue le cœur de cette thèse. Le chapitre 3 dresse un panorama des solutions proposées dans la littérature pour rapprocher ces deux mondes, avant de détailler notre approche de co-design dans les chapitres suivants.

Chapitre 3

État de l'art

L'évolution de la complexité matérielle des supercalculateurs modernes a conduit à une augmentation massive à la fois du nombre de cœurs disponibles et de leur puissance de calcul. Ces architectures multicœurs sont fréquemment couplées à des accélérateurs matériels spécialisés, faisant du développement de codes scientifiques un véritable défi. Pour répondre à cette problématique, les développeurs ont inventé différentes approches de programmation. Dans cette thèse, nous en étudierons deux : les bibliothèques de portabilité des performances de haut niveau, conçues pour abstraire le matériel et maintenir un code source unique pour de multiples architectures, et les compilateurs spécialisés dédiés à l'optimisation de code.

D'une part, les bibliothèques de portabilité des performances de haut niveau visent à fournir aux développeurs une interface unifiée pour gérer le parallélisme et la mémoire. D'autre part, les compilateurs spécialisés, tels que les compilateurs polyédriques, se concentrent sur la rigueur mathématique de l'optimisation des boucles et l'exposition d'un parallélisme à grain fin à travers des transformations avancées basées sur le code source ou sur une IR.

Bien que ces deux domaines coexistent, les faire interopérer de manière transparente reste un défi majeur pour la communauté de la compilation. Extraire un modèle mathématique à partir d'un code fortement abstrait par les structures C++ de haut niveau de ces bibliothèques est extrêmement difficile, car le compilateur perd des informations sémantiques critiques au cours du processus d'abaissement. À l'inverse, les bibliothèques elles-mêmes manquent de l'infrastructure interne nécessaire pour effectuer des analyses statiques complexes et des transformations structurelles automatiques de boucles.

Ce chapitre passe en revue la littérature existante autour de ces deux écosystèmes. Il explore d'abord à la section 3.1 comment les bibliothèques de portabilité des performances atteignent des performances indépendamment de l'architecture cible. Il analyse ensuite l'évolution des compilateurs polyédriques, des outils source-à-source jusqu'aux infrastructures IR modernes, en mettant en évidence leurs limitations inhérentes lorsqu'ils sont confrontés à de fortes abstractions (section 3.2). Enfin, la section 3.3 explore les approches hybrides et les langages dédiés qui tentent de combler ce fossé, démontrant la nécessité de la nouvelle approche proposée dans cette thèse.

3.1. Bibliothèques de portabilité des performances

Les développeurs de calcul haute performance en C++ s'appuient sur des bibliothèques de portabilité des performances comme Kokkos (section 2.2), RAJA [HK14], ou sur des standards émergents comme SYCL [Mar26] et le standard de parallélisme C++ (`std::execution::par`) [ISO17]. Tous ces outils partagent une philosophie commune : la séparation stricte entre l'expression de l'algorithme et son modèle d'exécution. Pour y parvenir, ils s'appuient fortement sur les fonctionnalités modernes du C++, en particulier les expressions lambda et les foncteurs, pour encapsuler les noyaux de calcul. Ils utilisent également des espaces d'exécution et des espaces mémoire pour gérer la distribution des calculs et la localité des données à travers des architectures hétérogènes. Pour illustrer cela, le tableau 1 compare les différentes façons d'écrire le même code (une multiplication matrice-vecteur) en utilisant ces différentes bibliothèques.

Bibliothèque	Exemple de syntaxe (multiplication matrice-vecteur)

Kokkos	C++
	`void matVecMult_kokkos(int N, int M, View<double**> A,`
	`  View<double*> x, View<double*> y) {`
	`  auto policy = Kokkos::RangePolicy<>(0, N);`
	
	`  Kokkos::parallel_for(policy, KOKKOS_LAMBDA(const int i) {`
	`  double sum = 0.0f;`
	`  for (int j = 0; j < M; ++j)`
	`  sum += A(i, j) * x(j);`
	`  y(i) = sum;`
	`  });`
	`}`

RAJA	C++
	`void matVecMult_raja(int N, int M, const double* A,`
	`  const double* x, double* y) {`
	`  using ExecPolicy = RAJA::omp_parallel_for_exec;`
	
	`  RAJA::forall<ExecPolicy>(RAJA::RangeSegment(0, N), [=](int i) {`
	`  double sum = 0.0f;`
	`  for (int j = 0; j < M; ++j)`
	`  sum += A[i * M + j] * x[j];`
	`  y[i] = sum;`
	`  });`
	`}`

SYCL	C++
	`void matVecMult_sycl(sycl::queue& q, int N, int M,`
	`  const double* A, const double* x,`
	`  double* y) {`
	`  auto policy = sycl::range<1>(N);`
	
	`  q.parallel_for(policy, [=](sycl::id<1> idx) {`
	`  int i = idx[0];`
	`  double sum = 0.0f;`
	`  for (int j = 0; j < M; ++j)`
	`  sum += A[i * M + j] * x[j];`
	`  y[i] = sum;`
	`  }).wait();`
	`}`

Bibliothèque	Exemple de syntaxe (multiplication matrice-vecteur)

std::execution::par	C++
	`void matVecMult_std(int N, int M, const double* A,`
	`  const double* x, double* y) {`
	`  std::vector<int> rows(N);`
	`  std::iota(rows.begin(), rows.end(), 0);`
	
	`  std::for_each(std::execution::par_unseq, rows.begin(), rows.end(),`
	`  [=](int i) {`
	`  double sum = 0.0f;`
	`  for (int j = 0; j < M; ++j)`
	`  sum += A[i * M + j] * x[j];`
	`  y[i] = sum;`
	`  });`
	`}`

Tableau 1. – Comparaison syntaxique des boucles parallèles entre différentes bibliothèques C++ de portabilité des performances.

Bien que l'objectif principal de ces environnements soit la portabilité, ils offrent tout de même certaines capacités d'optimisation structurelle. Par exemple, le tuilage de boucles est disponible dans Kokkos, RAJA et SYCL. Cela permet aux développeurs d'appliquer un tuilage statique sur les nids de boucles, ce qui améliore la localité des données dans les caches de l'architecture cible. Kokkos dispose d'un algorithme pour choisir automatiquement des tailles de tuiles efficaces en fonction de l'architecture, mais celui-ci est très limité en raison du manque d'informations statiques récoltées au sein de Kokkos. Des travaux ont été réalisés dans Kokkos pour ajouter `kokkos-tools` [Ham+18], une extension qui permet de surveiller et de tracer le code, mais aussi d'optimiser les noyaux à l'aide d'outils externes comme Apex [Huc22] ou Apollo [Bec+17] afin de spécialiser le code en quête de performances encore plus élevées.

Cependant, la limitation fondamentale de ces bibliothèques réside dans leur nature déclarative. Elles agissent principalement comme des moteurs de placement : ils distribuent aveuglément les itérations de boucles sur les threads CPU ou les blocs GPU, en faisant confiance au code écrit par le développeur. Ces bibliothèques ne possèdent pas de moteur d'analyse statique interne capable d'analyser les dépendances de données du noyau de calcul.

Par conséquent, il leur est techniquement et mathématiquement impossible de restructurer le code automatiquement. Les transformations complexes qui modifient l'ordre d'exécution des itérations de boucles (ex: fission, fusion) sont hors de portée de ces outils.

En d'autres termes, si le développeur écrit un nid de boucles structurellement sous-optimal, la bibliothèque le parallélisera fidèlement, mais de manière sous-optimale. L'optimisation du code reste donc la seule responsabilité du développeur, ce qui limite les performances maximales potentielles pouvant être atteintes de manière automatique.

3.2. Modèle polyédrique et implémentations

Il existe différentes implémentations de compilateurs et d'outils basés sur le modèle polyédrique. Historiquement, ces outils s'appuyaient sur des approches source-à-source et étaient limités à l'analyse d'un sous-ensemble du langage C. L'un des compilateurs les plus renommés pour la qualité de son ordonnanceur est Pluto [Bon+08c]. Il permet la compilation source-à-source de code C en explorant un vaste espace de transformations comme le tuilage en diamant. Un autre outil de référence est PPCG (*Polyhedral Parallel Code Generator*) [Ver+13], un compilateur source-à-source conçu pour générer du code GPU optimisé à partir de code C séquentiel. Ces approches purement textuelles, représentées à la figure 5, facilitent l'extraction du modèle : les accès mémoire, tels que les accès aux tableaux multidimensionnels (par exemple, A[i][j]), sont directement visibles sous forme d'indices, ce qui simplifie grandement l'analyse mathématique.

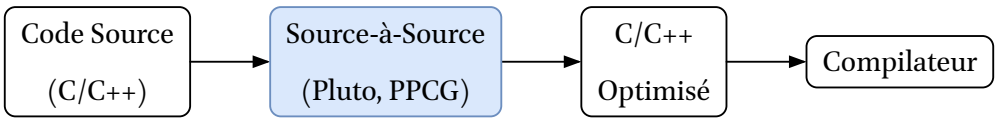


Figure 5. – Chaîne de compilation polyédrique source-à-source.

Bien que ces outils permettent d'atteindre d'excellentes performances pour des nids de boucles transformés, leur champ d'application reste très limité. S'appuyant sur des parseurs rudimentaires, ils n'ont absolument pas été conçus pour analyser des codes complexes provenant de bibliothèques de portabilité des performances, qui font un usage intensif de la métaprogrammation et des structures C++ modernes.

Pour surmonter la complexité liée à l'analyse syntaxique des langages de haut niveau, la communauté de la compilation polyédrique a adopté une nouvelle approche : abaisser le niveau d'analyse. En s'appuyant sur des représentations intermédiaires (IR) telles que celle de

LLVM, les outils polyédriques parviennent à s'abstraire du langage source (C, C++, Fortran) et à contourner la complexité de son analyse syntaxique et sémantique.

Graphite [Sjö+09] a été l'un des premiers compilateurs polyédriques largement adoptés à utiliser cette méthode, s'intégrant directement dans l'IR de GCC [Mer03]. Au sein de l'écosystème LLVM, l'outil Polly [Gro+11] utilise l'IR LLVM pour reconstruire et appliquer le modèle polyédrique. Plus récemment, des outils comme Polygeist [Mos21] s'appuient sur MLIR [Lat+21], une représentation IR LLVM de plus haut niveau qui permet de conserver certaines informations structurelles (telles que la sémantique des boucles `for`) sans avoir à les reconstruire à partir d'un graphe de flot de contrôle plus proche de la machine. Cette chaîne de compilation moderne au niveau de l'IR est représentée à la figure 6.

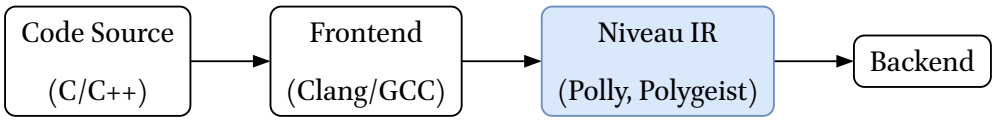


Figure 6. – Chaîne de compilation polyédrique au niveau de l'IR.

Pourtant, malgré leur application au niveau de l'IR, les outils comme Polly ou Polygeist échouent à optimiser les codes générés via des bibliothèques telles que Kokkos. Cette limitation découle du fossé sémantique. L'architecture de ces bibliothèques, conçue pour offrir la meilleure portabilité et généralisation possible à l'utilisateur, s'appuie en interne sur un réseau complexe d'expressions lambda, de foncteurs et d'arithmétique de pointeurs. Lors de la compilation, ces abstractions masquent la linéarité des accès mémoire, génèrent des incertitudes liées à l'aliasing, et brisent catégoriquement les heuristiques nécessaires à la construction du modèle polyédrique.

Une autre approche novatrice pour l'optimisation de code est celle utilisée par Apollo [Caa+17]. Elle consiste à s'appuyer sur l'IR de LLVM pour appliquer des transformations polyédriques, mais se base sur le comportement à l'exécution du programme pour déterminer si une région est polyédrique ou non. Cette méthode s'affranchit de la complexité de l'analyse statique de l'IR, qui peut s'avérer ardue pour déduire la structure et les accès mémoire du programme. La figure 7 illustre le fonctionnement du logiciel Apollo. Toutefois, cette approche engendre un surcoût non négligeable : elle requiert un système d'instrumentation et d'analyse des dépendances de données à l'exécution, ainsi qu'une compilation juste-à-temps (JIT) pour appliquer les transformations.

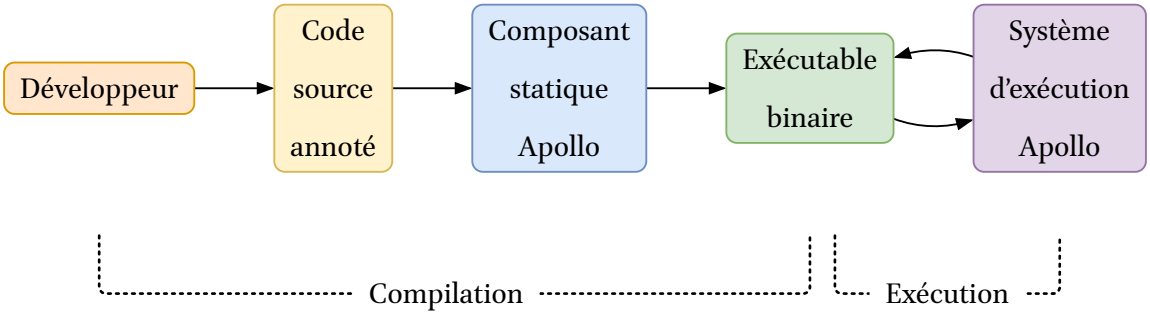


Figure 7. – Architecture de l'outil Apollo, séparant la compilation statique et l'optimisation dynamique à l'exécution.

3.3. Combiner abstractions de haut niveau et optimisation polyédrique

Une autre approche pour atténuer les problèmes précédemment exposés consiste à élever le niveau d'abstraction sémantique en utilisant des langages de plus haut niveau tels que Python ou des langages dédiés (DSL), ou encore des bibliothèques offrant une plus grande expressivité. Ainsi, la sémantique mathématique des opérations devient beaucoup plus évidente à interpréter pour les outils de compilation, ce qui réduit considérablement le fossé sémantique entre le code source et l'application du modèle polyédrique.

Tiramisu [Bag+18] illustre parfaitement cette dynamique. Il s'agit d'une infrastructure logicielle C++ fonctionnant, dans sa conception, comme un langage dédié pour le calcul haute performance, comme illustré en liste 8. L'utilisateur déclare formellement les calculs, les tailles des données, le domaine d'itération, ainsi que les transformations mathématiques à appliquer (tuilage, déroulage, parallélisation). En imposant cette déclaration explicite, l'outil peut appliquer des transformations polyédriques et générer un code C++ hautement optimisé sans que le compilateur n'ait à deviner la structure du programme.

```C++
void matVecMul_tiramisu(int N, int M) {
    tiramisu::init("matVecMult");

    tiramisu::var i("i", 0, N);
    tiramisu::var j("j", 0, M);

    tiramisu::input A("A", {i, j}, tiramisu::p_float64);
```

```
8       tiramisu::input x("x", {j}, tiramisu::p_float64);
9
10      tiramisu::computation y_init("y_init", {i}, tiramisu::expr(0.0));
11
12      tiramisu::computation y_update("y_update", {i, j}, y_init(i) + A(i, j) *
        x(j));
13
14      y_init.then(y_update, i);
15
16      y_init.parallelize(i);
17      y_update.parallelize(i);
18
19      tiramisu::buffer b_A("b_A", {tiramisu::expr(N), tiramisu::expr(M)},
        tiramisu::p_float64, tiramisu::a_input);
20      tiramisu::buffer b_x("b_x", {tiramisu::expr(M)}, tiramisu::p_float64,
        tiramisu::a_input);
21      tiramisu::buffer b_y("b_y", {tiramisu::expr(N)}, tiramisu::p_float64,
        tiramisu::a_output);
22
23      A.store_in(&b_A);
24      x.store_in(&b_x);
25
26      y_init.store_in(&b_y, {i});
27      y_update.store_in(&b_y, {i});
28
29      tiramisu::codegen({&b_A, &b_x, &b_y}, "matvec_mult.o");
30  }
```

Liste 8. – Exemple de syntaxe déclarative dans Tiramisu, séparant proprement l'algorithme de son ordonnancement d'exécution.

Suivant cette même logique d'abstraction, d'autres travaux se sont tournés vers l'écosystème Python pour s'affranchir de la gestion complexe de la mémoire en C++. Par exemple, [Ram+18] a démontré l'efficacité de l'application du modèle polyédrique directement sur les opérations NumPy [Har+20]. De même, des initiatives telles que PyKokkos [Al +21] offrent des interfaces Python de haut niveau. Ces couches d'abstraction permettent de capturer les opérations mathématiques de manière hautement abstraite. Avoir accès à cette information sémantique préservée est extrêmement bénéfique pour extraire le modèle polyédrique, tout en continuant à masquer la complexité matérielle. Un exemple de cette syntaxe de haut niveau est fourni en liste 9.

```python
import pykokkos as pk

@pk.workunit
def matVecMult_kernel(i: int, M: int,
                      A: pk.View2D[pk.double],
                      x: pk.View1D[pk.double],
                      y: pk.View1D[pk.double]):
    sum_val: pk.double = 0.0
    for j in range(M):
        sum_val += A[i, j] * x[j]
    y[i] = sum_val

def matvec_mult_pykokkos(N: int, M: int, A: pk.View2D, x: pk.View1D, y: pk.View1D):
    pk.parallel_for(N, matVecMult_kernel, M=M, A=A, x=x, y=y)
```

Liste 9. – Exemple de multiplication matrice-vecteur utilisant l'interface Python de haut niveau PyKokkos.

Bien que ces approches apportent des solutions très efficaces aux problèmes de performances, elles imposent un coût d'entrée prohibitif pour le monde du HPC. En effet, elles exigent des scientifiques qu'ils réécrivent intégralement leurs codes de simulation historiques et massifs dans de nouveaux langages ou via des API hautement spécifiques.

Pour contourner cette barrière de réécriture, certains travaux ont exploré l'approche inverse : utiliser les bibliothèques de portabilité comme cibles de compilation. Par exemple, les auteurs de [Bas+20] ont utilisé des outils polyédriques pour analyser du code C séquentiel afin de générer automatiquement du code Kokkos. Cette stratégie combine l'optimisation mathématique effectuée en amont avec la portabilité matérielle garantie par Kokkos en aval.

Cependant, bien que cette méthode soit pertinente pour la modernisation des anciens codes, elle ne résout absolument pas le problème central de l'écosystème actuel : optimiser des codes déjà écrits nativement en Kokkos. À ce jour, il n'existe aucun outil capable d'ingérer le code source Kokkos, de l'analyser mathématiquement, et de restructurer ses boucles de l'intérieur de manière transparente. C'est pour combler ce vide scientifique que cette thèse propose une intégration du modèle polyédrique, capable d'opérer directement sous les abstractions de Kokkos.

Chapitre 4

Rendre les noyaux de calcul de Kokkos visibles

Dans ce chapitre, nous présentons la première contribution majeure de cette thèse : une chaîne de compilation qui comble le fossé sémantique entre la bibliothèque de portabilité des performances Kokkos et l'optimiseur polyédrique Polly. Nous y détaillons l'ensemble de notre approche de bout en bout.

Tout d'abord, nous exposons les motivations scientifiques et techniques qui justifient ce travail, ainsi que les principes de la stratégie de co-design mise en place (section 4.1). Ensuite, nous détaillons les modifications apportées à Kokkos : l'introduction du paramètre de template `usePolyOpt` pour cibler l'optimisation à grain fin, la réécriture et l'isolation des *backends* pour fournir des nids de boucles *épurés*, et l'injection d'annotations sémantiques dans l'IR LLVM (section 4.2). Puis, nous décrivons les passes de restructuration et d'analyse intégrées dans Polly (section 4.3). Celles-ci incluent l'extraction des annotations (section 4.3.1), la simplification et l'*assainissement* du flot de contrôle (section 4.3.2), la *canonicalisation* des registres de tableaux (section 4.3.3), ainsi que la *délinéarisation* des accès mémoire (section 4.3.4). Enfin, nous présentons le mécanisme de synchronisation du *backend*, permettant à Polly d'adapter sa génération de code aux différents espaces d'exécution matériels ciblés par les noyaux Kokkos (section 4.3.5).

4.1. Motivations

Comme observé dans l'état de l'art du chapitre 3, les techniques C++ modernes utilisées par Kokkos (métaprogrammation, lambdas, foncteurs, arithmétique de pointeurs internes) créent un fossé sémantique entre le code source et la représentation intermédiaire (IR) de LLVM. Lorsque le code Kokkos est abaissé au niveau de l'IR, où opère l'outil d'analyse et d'optimisation Polly, la structure originale simple des nids de boucles multidimensionnels est délaissée au profit de structures complexes nécessaires au fonctionnement de Kokkos. Le compilateur ne perçoit alors que des pointeurs représentant les bornes de boucles et des accès mémoire via des pointeurs unidimensionnels complexes. Manquant d'informations et craignant des risques d'*aliasing*, Polly se retrouve dans l'incapacité de réaliser ses analyses pour créer une représentation polyédrique et appliquer ses optimisations.

L'objectif de ce travail est donc d'offrir aux développeurs Kokkos l'accès aux optimisations polyédriques avancées (telles que la fusion/fission de boucles) sur du code Kokkos, sans pour autant briser ses promesses de performance et de portabilité. Le développeur doit pouvoir continuer à écrire ses noyaux de calcul de manière agnostique à l'architecture avec une intervention minimale.

Pour atteindre cet objectif, nous proposons une stratégie de co-design entre la bibliothèque et le compilateur (figure 8). Puisque le compilateur Polly ne peut pas deviner l'intention de haut niveau du développeur qui est masquée par la complexité du code, et que Kokkos n'est pas conçu pour effectuer des transformations mathématiques complexes, les deux doivent coopérer. Notre approche consiste à instrumenter Kokkos pour qu'il injecte des indices sémantiques via des annotations dans le code généré afin de produire un code clair et simple à optimiser, et à adapter Polly afin qu'il soit capable de lire ces informations depuis l'IR. Ainsi, le compilateur pourra reconstruire un modèle polyédrique complet et appliquer des transformations statiques, tout en tirant parti de la connaissance de la structure initiale fournie par Kokkos.

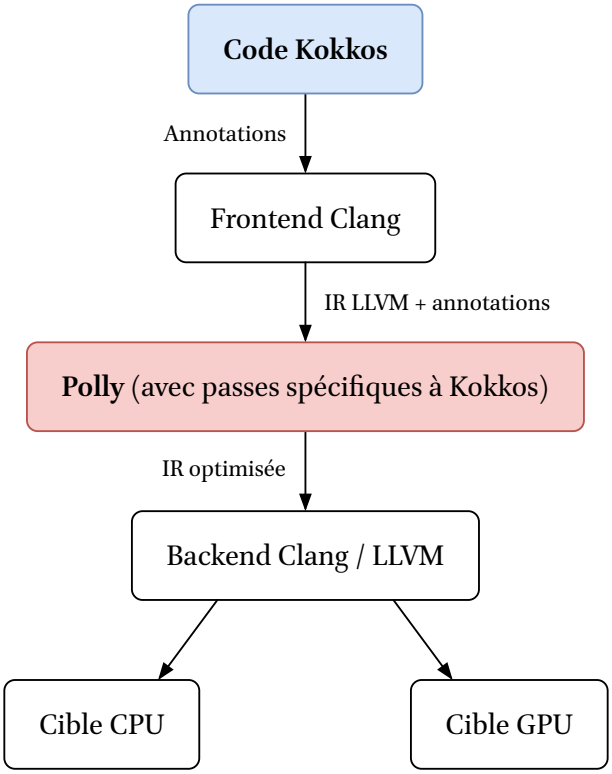


Figure 8. – Chaîne de compilation proposée, issue du co-design entre Kokkos et Polly.

Comme l'illustre la figure 8, cette chaîne de compilation s'articule autour de deux volets complémentaires : d'une part, l'instrumentation en amont de Kokkos pour injecter et préserver les informations sémantiques jusqu'à l'IR LLVM, ainsi que la modification des backends Kokkos pour transmettre le code le plus clair possible à Polly, et d'autre part, l'intégration de passes spécialisées au sein de Polly pour exploiter ces annotations et émettre du code adapté tant aux CPU qu'aux GPU. Les sections suivantes détaillent la mise en œuvre de ces deux piliers, en débutant par les modifications apportées à Kokkos.

4.2. Modifications de Kokkos

Le co-design entre Kokkos et Polly implique d'instrumenter Kokkos afin qu'il injecte des informations sémantiques supplémentaires (telles que les bornes de boucles, des informations sur les tableaux ou encore le backend utilisé) lors de la génération du code. Ces informations, préservées sous forme d'annotations jusqu'à la représentation intermédiaire de LLVM, sont ensuite exploitées par Polly pour reconstruire un SCoP complet et précis.

Outre ce système d'annotations, les modifications apportées à Kokkos s'appuient sur des mécanismes de métaprogrammation. Ces derniers permettent d'introduire une option de configuration de compilation à granularité fine (à l'échelle du noyau) et d'adapter les backends afin de délimiter le code destiné à être analysé par Polly. L'objectif est d'isoler les boucles et de produire une représentation structurellement simplifiée représentable par Polly, optimisant ainsi la détection des SCoPs, tout en garantissant que le reste du programme conserve son comportement initial.

4.2.1. Option *usePolyOpt*

Par défaut, Polly analyse l'ensemble de l'IR d'un module à la recherche de SCoPs. Cette approche globale manque de granularité, en augmentant inutilement les temps de compilation, en risquant de transformer des parties de l'application qui ne le nécessitent pas et en augmentant implicitement le risque d'erreur.

Afin de retrouver un niveau de contrôle fin, similaire à celui offert nativement par Kokkos, où le développeur structure l'exécution noyau par noyau, et afin de garantir que les applications Kokkos existantes restent strictement inchangées si les optimisations polyédriques ne sont pas activées, nous introduisons le paramètre optionnel de template `usePolyOpt`. La liste 10 présente ainsi la nouvelle signature de `parallel_for` modifiée à cet effet.

```C++
template <bool usePolyOpt = false,
          class ExecPolicy,
          class FunctorType,
          class Enable = std::enable_if_t<is_execution_policy<ExecPolicy>::value>
          >
inline void parallel_for(const std::string& str, const ExecPolicy& policy,
                         const FunctorType& functor);
```

Liste 10. – Nouvelle signature de `parallel_for` avec le paramètre `usePolyOpt`.

Cette option s'ajoute simplement lors de l'appel à la fonction `parallel_for` (voir liste 11). Grâce à la métaprogrammation C++, elle permet de déclencher statiquement le chemin de génération de code « annoté et isolé » uniquement pour les noyaux ciblés. Si l'option est omise, le comportement de compilation standard de Kokkos est conservé sans aucune pénalité sur la portabilité ou les performances.

```cpp
Kokkos::parallel_for<Kokkos::usePolyOpt>(policy,
                                         KOKKOS_LAMBDA(long i, long j) {
    B(i, j) = A(i, j) + A(i, j - 1) + A(i, j + 1) +
                        A(i - 1, j) + A(i + 1, j);
  });
```

Liste 11. – Exemple d'utilisation du paramètre `usePolyOpt` pour activer l'analyse polyédrique sur un noyau Kokkos spécifique.

4.2.2. Réécriture du backend et isolation

Pour cibler un backend spécifique, Kokkos s'appuie sur une interface de métaprogrammation structurée autour d'une définition générique de la fonction `parallel_for` (déclarée dans la liste 10). Cette fonction est surchargée par chaque backend afin d'adapter l'exécution aux différentes architectures matérielles. Lors de la compilation, Kokkos redirige automatiquement le code vers le backend approprié en fonction de la politique d'exécution et des types de données templates spécifiés par l'utilisateur. La figure 9 illustre cette architecture de sélection et de redirection du code.

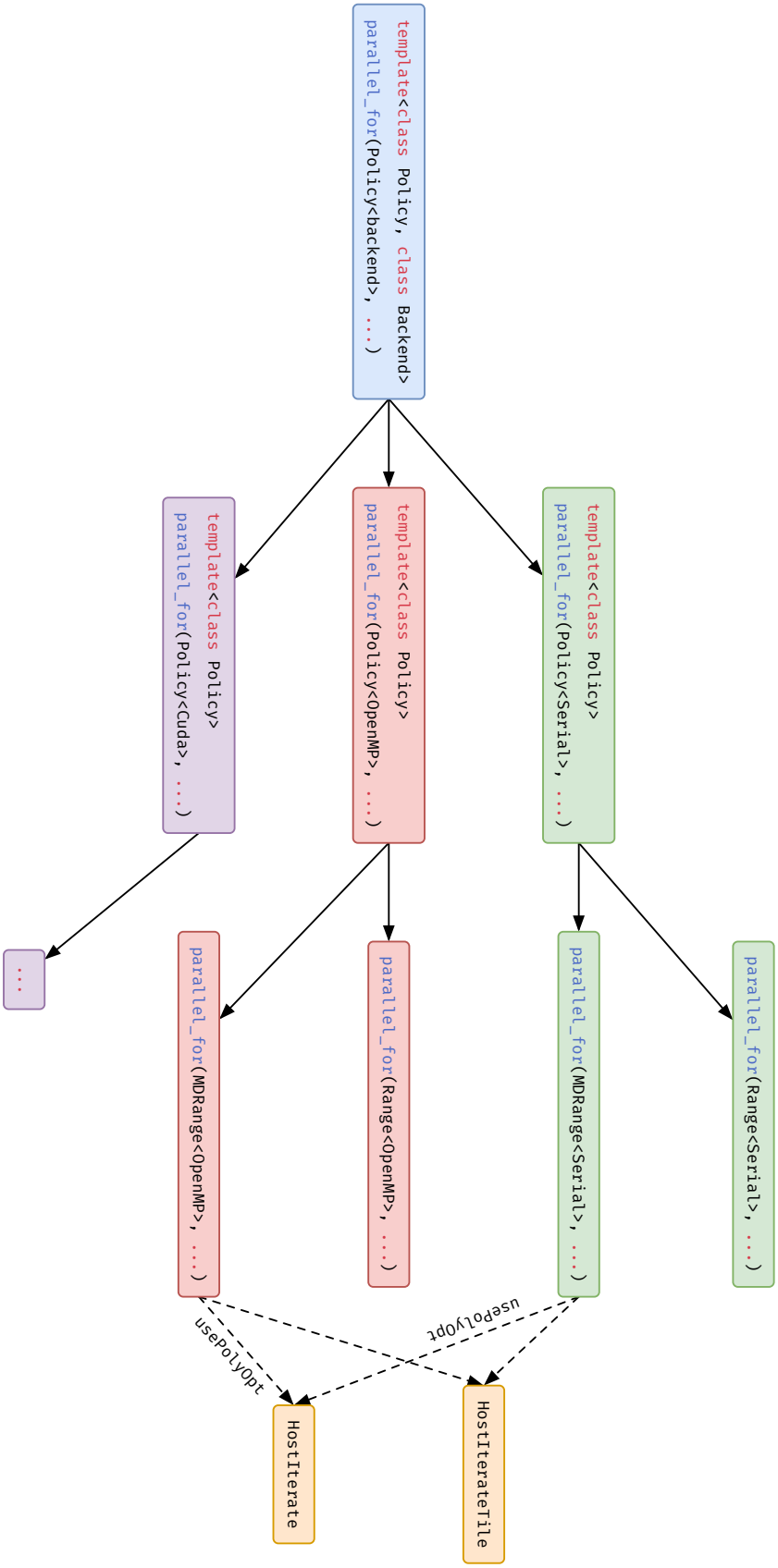


Figure 9. – Architecture de sélection des backends dans Kokkos.

L'implémentation de ces backends étant masquée pour l'utilisateur, elle offre l'opportunité d'adapter le code généré de manière transparente. Cette couche d'abstraction permet de réécrire la structure des noyaux en supprimant la complexité introduite par les optimisations matérielles natives ou les pragmas, permettant ainsi de produire un code source intermédiaire plus accessible pour l'analyse polyédrique par Polly.

Lors de l'utilisation de `MDRangePolicy` pour itérer sur des espaces multidimensionnels, les backends standards de Kokkos appliquent des mécanismes statiques de découpage en blocs pour appliquer un tuilage. Bien que ces transformations statiques soient essentielles pour exposer le parallélisme et optimiser l'exécution sur les architectures cibles, elles saturent le code généré de niveaux de boucles supplémentaires et d'une arithmétique d'indexation complexe dans l'IR. Cela rend opaques les SCoPs que Polly tente de reconstruire.

Or, le modèle polyédrique est conçu pour effectuer lui-même de telles transformations. Afin de promouvoir la détection de SCoPs en exposant à Polly des nids de boucles canoniques et réguliers, la politique `MDRangePolicy` a été réécrite pour désactiver le tuilage de Kokkos lorsqu'elle est utilisée dans notre pipeline. Concrètement, une variante de la structure responsable du tuilage, nommée `HostIterate`, a été ajoutée et remplace `HostIterateTile` lors de l'utilisation du paramètre `usePolyOpt` (voir figure 9). Cela garantit un code épuré pour Polly, tout en préservant le fonctionnement par défaut de Kokkos pour le reste de l'application, et sans nécessiter la moindre intervention du développeur.

Par ailleurs, un noyau Kokkos n'est pas une simple boucle isolée : il est accessible à travers une pile de fonctions imbriquées souvent inlinées à la compilation. Une fois ces fonctions inlinées, le corps de la boucle se retrouve « noyé » dans une vaste région de l'IR contenant des portions de code qui ne doivent pas être transformées par des techniques polyédriques. Notre solution est d'associer un attribut `noinline` à la fonction « feuille » de cette pile d'appels, où se situe l'appel à la fonction de calcul. Nous marquons aussi la fonction avec un attribut dédié `findscop`. Ainsi, Polly identifie sans ambiguïté les noyaux issus du paramètre `usePolyOpt`. De plus, limiter le spectre d'analyse de Polly uniquement aux fonctions portant cet attribut permet d'accélérer la compilation et de garantir que le reste de l'application Kokkos n'est pas impacté par d'éventuelles transformations polyédriques.

4.2.3. Annotations

Afin de préserver la sémantique de haut niveau tout au long du processus d'abaissement vers la représentation intermédiaire, il est nécessaire d'intégrer des informations directement au sein du code de Kokkos. En effet, à travers ses multiples couches d'abstraction, certaines informations explicites dans le code source (telles que les bornes de boucles, les dimensions ou encore les fonctions d'accès aux tableaux) sont perdues, masquées par des structures complexes ou par des calculs d'adresses. Cette perte d'information contraint le compilateur à adopter une approche conservatrice, limitant drastiquement les transformations applicables.

Afin de pallier ce problème de manière transparente, notre approche exploite la fonction intrinsèque suivante du compilateur LLVM :

```C++
1 typeof(expr) __builtin_annotation (typeof(expr) e, const char *string);
```

Cette construction associe une chaîne de caractères `string` (faisant office d'étiquette sémantique) à une expression `e`, générant ainsi un nœud `llvm.annotate` au sein de l'IR. Elle établit ainsi un pont sémantique robuste entre les abstractions C++ de haut niveau et la représentation de bas niveau.

L'injection de ces annotations est réalisée directement au sein des différents backends de Kokkos, mais aussi dans les méthodes d'accès aux données multidimensionnelles. Ainsi, le processus demeure totalement transparent pour l'utilisateur final, dont le code applicatif reste inchangé.

Ces annotations sont notamment employées pour garantir la détection des SCoPs et appliquer de manière sécurisée les nouvelles passes de restructuration (section 4.3) spécifiques à Kokkos au sein de Polly. Elles jouent également un rôle fondamental pour assurer la délinéarisation des tableaux multidimensionnels, dont les détails de mise en œuvre et le fonctionnement sont exposés à la section 4.3.4.

4.3. Préparation du code et restructuration de l'IR

Notre approche de co-design nécessite également d'adapter Polly afin de lui permettre non seulement d'interpréter les informations injectées par Kokkos, mais également d'appliquer des transformations basées sur des heuristiques spécifiques à Kokkos. En effet, la réécriture des backends de la bibliothèque et la présence d'annotations et de leur interprétation directe s'avèrent insuffisantes : l'IR LLVM générée à partir du code Kokkos reste trop opaque à cause de constructions complexes (telles que des bornes de boucles lues dynamiquement depuis la mémoire) que le compilateur peine à identifier comme invariantes. Afin de préparer le code pour l'analyse polyédrique, plusieurs passes de restructuration dédiées doivent être exécutées en amont. Celles-ci garantissent l'obtention d'une représentation intermédiaire sémantiquement équivalente, mais dont la structure épurée expose clairement les informations nécessaires à Polly pour la construction du SCoP.

Au sein de la chaîne de compilation, l'exécution des transformations de restructuration nécessite d'établir un point d'ancrage précis dans la séquence des passes de Polly. Dans son fonctionnement par défaut, le pipeline de Polly débute par une étape préparatoire `CodePreparation` dont le rôle est d'isoler les allocations mémoire locales (`alloca`) des opérations arithmétiques et des accès aux données, afin de simplifier les analyses subséquentes. Nous mettons à profit cette phase précoce pour ajouter, juste après cette passe initiale, notre propre suite de passes de prétraitement.

À ce stade du pipeline, l'IR LLVM résultant de l'abaissement des modèles C++ de Kokkos doit faire l'objet de transformations. Elles relèvent d'un double objectif :

- D'une part, extraire et exploiter les annotations sémantiques injectées par les backends de Kokkos, afin d'exposer clairement à Polly les informations permettant de construire la modélisation polyédrique (section 4.3.1) ;
- D'autre part, assainir la topologie du graphe de flot de contrôle en éliminant les branchements inutiles de la structure de boucle issue de Kokkos (section 4.3.2).

4.3.1. Extraction et exploitation des annotations

Afin d'établir une passerelle sémantique entre les abstractions C++ de haut niveau et le modèle polyédrique, notre approche repose sur l'extraction systématique des annotations injectées au sein des backends de Kokkos via les intrinsèques `__builtin_annotation`. Ce mécanisme, totalement transparent pour l'utilisateur, permet à Polly de récupérer les informations fondamentales pour la modélisation polyédrique :

- Les bornes de l'espace d'itérations des boucles ;
- Les pointeurs de base des structures de données (`View`) ;
- Les dimensions spatiales des tableaux multidimensionnels.

L'accès à ces informations constitue une condition préalable absolue pour permettre au compilateur de construire un domaine d'itérations et de délinéariser fidèlement les accès mémoire.

Résilience des métadonnées face aux optimisations de bas niveau. Cette extraction soulève un défi : la **résilience** des métadonnées tout au long de la chaîne LLVM. En effet, entre l'émission initiale de l'IR par le compilateur frontend Clang et l'exécution des passes polyédriques de Polly, le code intermédiaire traverse une multitude de passes d'optimisation (combinaison d'instructions, élimination de sous-expressions communes, propagation de constantes, simplification du flot de contrôle).

Lorsqu'une annotation est émise dans l'IR par le frontend Clang, elle est systématiquement placée à la suite de l'instruction ciblée. Au cours des passes d'optimisation préalables, bien que les appels à l'intrinsèque `llvm.annotate` soient ignorés et préservés par le compilateur, ces derniers sont susceptibles de référencer des valeurs ayant été altérées ou supprimées principalement pour le marquage des tableaux constitué d'au moins trois 3 annotations distinctes.

Une considération supplémentaire réside dans le fait que le système d'annotations cible les variables de boucle originelles, mais pas nécessairement les expressions dérivées qui en découlent. À titre d'illustration, la liste 12 présente un fragment de code exploitant les fonctionnalités détaillées à la section 5.2.3. La boucle interne `j` s'appuie sur la même variable `n` que la boucle externe `i`, mais au travers de l'expression arithmétique `n - 1`. L'annotation

explicite de `n` porte ainsi exclusivement sur `n`, et non sur le registre temporaire résultant du calcul de `n - 1`. Par conséquent, afin de préserver la validité du SCoP et d'en déduire correctement les bornes polyédriques, le mécanisme d'analyse des annotations doit être capable de remonter la chaîne de dépendances depuis l'utilisation de la borne dérivée jusqu'à la variable annotée initiale.

```cpp
auto policy = Kokkos::RangePolicy(0, n);
Kokkos::parallel_for<Kokkos::usePolyOpt, "p0.u0 == n">(
  policy,
  KOKKOS_LAMBDA(int i) {
    for(int j = 1; j < KOKKOS_LOOP_BOUND(n) - 1; j++) {
      A(i, j) = B(i, j);
    }
  }
);
```

Liste 12. – Exemple d'annotation de borne de boucle impliquant une expression arithmétique `n-1`.

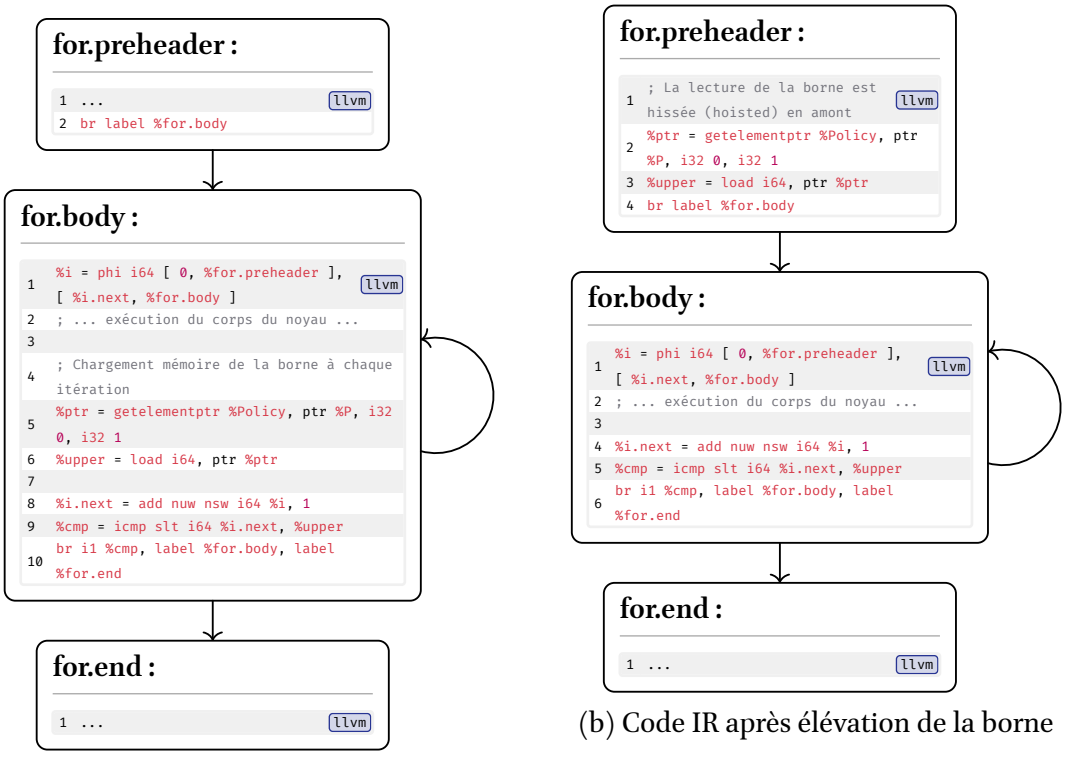
Élévation des bornes de boucles et invariants. Le modèle d'exécution de Kokkos s'appuie sur des politiques de parcours (`RangePolicy`, `MDRangePolicy`) qui, au lieu de manipuler de simples variables scalaires, encapsulent les bornes de boucles dans des structures de données sous forme de tableaux. Lors de l'abaissement vers l'IR LLVM, l'évaluation de ces bornes se matérialise souvent par des chargements mémoire répétés (`getelementptr` suivi de `load`) qui peuvent se situer à l'intérieur même du nid de boucles, bien que les valeurs obtenues soient invariantes.

Cette situation est hautement pénalisante à deux égards :

1. **Pénalité de performance :** À chaque itération, le processeur exécute une lecture mémoire redondante pour recharger une valeur de borne que le programmeur sait invariante sur l'ensemble de la boucle. L'analyse d'alias conservatrice de LLVM ne parvenant pas à prouver qu'aucune écriture dans le corps de boucle ne modifie la valeur de la borne, la passe standard de déplacement de code invariant (**Loop Invariant Code Motion**, LICM) échoue à hisser cette instruction hors de la boucle faute de pouvoir exclure un aliasing.
2. **Incompatibilité avec le modèle polyédrique :** La théorie polyédrique impose que les bornes régissant le domaine d'itérations soient des paramètres invariants définis

strictement en amont de chaque boucle du nid. La présence d'un chargement dynamique au sein du corps de boucle disqualifie immédiatement le nid de boucles lors de la phase de détection de SCoP.

L'extraction des annotations relatives aux bornes permet à notre passe de transformation de l'IR d'identifier sans ambiguïté la nature invariante de ces accès mémoire, et d'en forcer l'élévation (**hoisting**) hors du nid de boucles (figure 10). Une fois les instructions de chargement de bornes de boucles déplacées juste après le calcul invariant de leur adresse en amont de la boucle, ces bornes sont reconnues par Polly comme de véritables paramètres invariants, satisfaisant ainsi pleinement les prérequis de la modélisation polyédrique.



(a) Code IR avant la passe de hoisting.

(b) Code IR après élévation de la borne invariante.

Figure 10. – Élévation du chargement de la borne de boucle dans le pré-en-tête du SCoP.

Autres annotations contextuelles. L'étiquette ajoutée grâce à la fonction intrinsèque `__builtin_annotation` permet non seulement de déterminer la nature de l'annotation, mais elle peut également être exploitée pour extraire des métadonnées d'une plus fine granularité. Ainsi, chaque dimension des tableaux utilisés dans le noyau est annotée par une chaîne de caractères définissant le type d'accès (ex. `"dim"`), tout en précisant le niveau d'imbrication correspondant

(ex. `"dim 1"`). Concernant les bornes de boucles, un mécanisme analogue est employé afin de relier les bornes (inférieure et supérieure) de la politique d'exécution utilisée à la profondeur de la boucle au sein du nid de boucles.

Le chapitre 5 détaillera l'utilisation de ces annotations pour étendre les capacités de détection de Polly. Au-delà des métadonnées relatives aux tableaux et aux bornes de boucles, des annotations additionnelles s'avèrent indispensables pour garantir une utilisation transparente de Polly dans Kokkos et pour modéliser des schémas d'exécution plus complexes. Ces annotations supplémentaires, ainsi que les sections qui leur sont consacrées, sont détaillées ci-dessous.

- Le **backend d'exécution cible :** transmission de l'architecture matérielle assignée au noyau Kokkos (`Serial`, `OpenMP`, `Cuda`), permettant à Polly de sélectionner la stratégie de génération de code adéquate à l'échelle de chaque SCoP individuel (section 4.3.5);
- **Le nommage statique des tableaux :** l'assignation d'un identifiant statique aux structures `Kokkos::View` permet de rétablir l'identité commune de pointeurs distincts. Cette étape est cruciale lorsque ces pointeurs sont capturés par des fonctions *lambda* différentes dans le cadre d'une fusion inter-noyaux (section 5.2);
- Les **contraintes d'assomption** : formulation textuelle de relations algébriques (égalités, inégalités) entre paramètres de boucles, injectées sous forme d'hypothèses dans le domaine polyédrique (section 5.2.3).

4.3.2. Simplification et assainissement du CFG

Bien que l'extraction des annotations sémantiques fournisse les informations de haut niveau indispensables, elles peuvent ne pas suffire à elles seules pour garantir une analyse polyédrique réussie.

En effet, pour qu'une région de code puisse être formellement reconnue comme un SCoP, l'IR LLVM doit impérativement satisfaire des critères de régularité stricts : des bornes de boucles et des conditions de branchement exprimées sous forme de fonctions affines des itérateurs et paramètres, un flot de contrôle statique, des accès mémoire déterministes et explicites, ainsi que l'absence d'aliasing.

Afin de garantir le respect de ces contraintes, nous avons conçu et intégré une suite de passes de restructuration s'exécutant directement lors de la phase de `CodePreparation` de Polly. Lors des phases d'optimisation préliminaires de LLVM, les boucles sont réorganisées (via des passes comme `loop-rotate`) afin d'adopter une structure de type `do-while` facilitant les optimisations. Cette transformation canonique impose l'insertion de branchements conditionnels en amont, visant à vérifier dynamiquement si une boucle possède au minimum une itération avant d'autoriser l'entrée dans le corps de la boucle. Cette vérification est redondante avec celle déjà présente dans les *backends* modifiés pour l'application du modèle polyédrique.

Ainsi, au niveau de la représentation intermédiaire, deux constructions idiomatiques entravent particulièrement la détection des SCoPs : les branchements qui encadrent préventivement les boucles, ainsi que les mécanismes de versionnage de boucles.

Bien que Polly ait été conçu pour gérer certaines de ces structures complexes, comme par exemple la figure 11, le code intermédiaire généré par Kokkos empêche la détection systématique des SCoPs. Les analyses d'évolution scalaire deviennent excessivement conservatrices, rompant ainsi la validité du SCoP. Il s'avère donc indispensable d'introduire des passes de restructuration dédiées afin d'assainir et de simplifier le CFG en amont pour garantir la détection systématique des SCoPs.

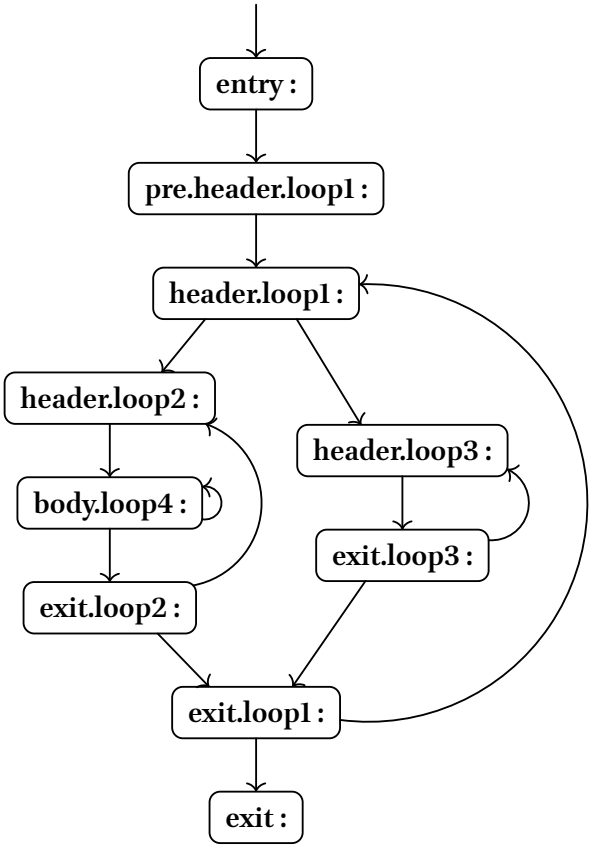


Figure 11. – Exemple de graphe de flot de contrôle (CFG) complexifié par des branchements conditionnels et du versionnage de boucles, entravant la détection du SCoP.

Élimination des vérifications conditionnelles de bornes. Grâce aux modifications apportées aux backends de Kokkos, il est désormais garanti que l'appel du noyau de calcul s'effectue systématiquement avec un espace d'itérations non vide lors de l'exécution. Il devient dès lors possible de supprimer en toute sécurité ces vérifications conditionnelles de bornes, qui surchargent inutilement le graphe de flot de contrôle.

Suppression du versionnage de boucles. De manière analogue, le versionnage de boucles introduit par LLVM génère des variantes d'exécution redondantes, protégées par des gardes. Puisque notre approche délègue le réordonnancement et la parallélisation au moteur polyédrique, ce versionnage prématuré démultiplie inutilement les arêtes du CFG. Une passe dédiée est ainsi chargée de les éliminer, ramenant le nid de boucles à une forme unique, simplifiée et analysable par Polly.

Par ailleurs, Polly insère automatiquement un mécanisme de garde dynamique afin de vérifier à l'exécution la validité des hypothèses formulées lors de l'analyse statique. Ce contrôle

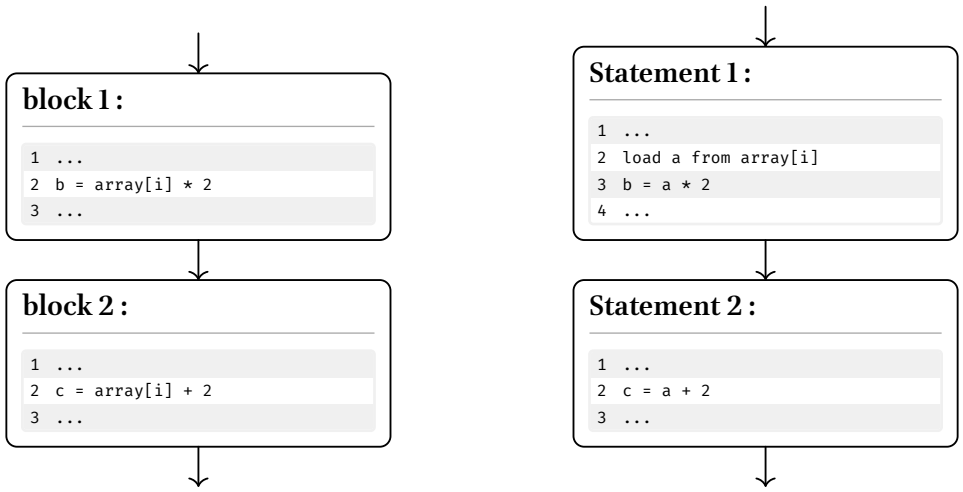
s'assure notamment de la cohérence entre le domaine d'itération et les paramètres utilisés. Ces vérifications conditionnent le lancement du code transformé : si les contraintes du modèle polyédrique ne sont pas respectées, l'exécution bascule vers la version originelle du code, garantissant ainsi la préservation de la sémantique du programme.

4.3.3. Canonicalisation des registres de tableaux

L'analyse et la transformation de la représentation intermédiaire LLVM soulèvent des problématiques que l'on ne rencontre pas avec les outils opérant de source à source. Au niveau de l'IR, les registres employés pour stocker les données obéissent à la forme SSA. Cette propriété impose l'immuabilité et l'assignation unique de chaque registre, par opposition au code source où une même variable peut être réassignée de multiples fois. Dans le contexte de l'IR, cette contrainte engendre non seulement une multiplication des registres définissant une même donnée, mais introduit également des dépendances entre les blocs de base par l'intermédiaire de nœuds PHI.

La figure 12 illustre ce phénomène. La figure 12a présente le graphe de flot de contrôle d'un code source effectuant des accès consécutifs à la même zone mémoire `array[i]`, sans aucune dépendance scalaire entre les instructions. À l'inverse, la figure 12b expose la représentation intermédiaire LLVM équivalente. L'accès mémoire `array[i]` au sein du bloc `block 1` y est remplacé par une instruction de chargement (`load`) dont le résultat est assigné à la variable SSA `%a`. Cette variable `%a` est ensuite consommée non seulement dans le `block 1`, mais également dans le `block 2`.

Lors de son analyse, Polly modélise ces deux blocs sous la forme de deux *statements* distincts. De ce fait, Polly introduit artificiellement une dépendance scalaire : pour que le *statement* représentant le `block 2` puisse être évalué, il est impératif que le premier *statement* (associé au `block 1`) ait été exécuté au préalable, puisqu'il héberge l'instruction de chargement.



(a) Accès mémoire directs sans dépendance scalaire entre *statements*.

(b) Dépendance scalaire sur le registre `a` partagé entre les deux *statements*.

Figure 12. – Exemple de dépendance scalaire sur `a` entre deux *statements* dans le modèle IR de Polly.

La passe `ForwardOpTree` de Polly a pour objectif de rompre ces dépendances entre *statements* afin d'élargir l'espace de recherche de l'ordonnanceur. Son mécanisme repose sur la remontée d'arbres d'opérandes : au lieu de calculer une valeur scalaire dans un *statement* producteur puis de la transmettre à un *statement* consommateur via un registre ou un nœud PHI, la passe identifie l'arbre d'instructions générant cette valeur et le duplique directement au sein du *statement* consommateur. Dans l'exemple précédent (figure 12b), la passe remonte l'arbre d'opérations de `%a` depuis le `block 2` vers le `block 1`. Si elle parvient à prouver qu'il n'existe aucun risque d'altérer la sémantique du programme, elle effectuera une copie de l'instruction `load a from array[i]` dans le `block 2`, brisant ainsi la dépendance.

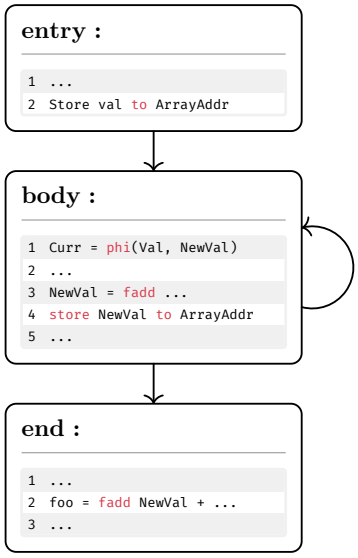
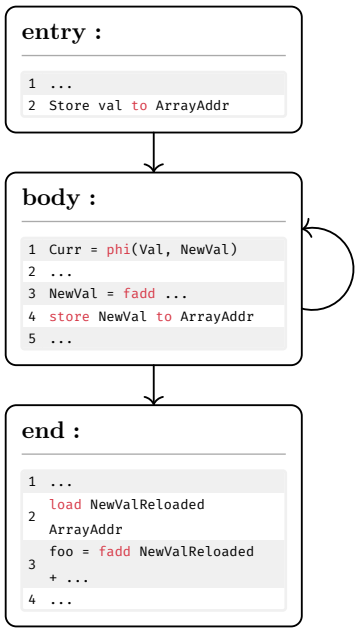
Néanmoins, la portée de la passe `ForwardOpTree` demeure limitée. En effet, les transformations LLVM sont soumises à des contraintes très strictes afin de préserver absolument la sémantique d'origine du programme. Ces analyses conservatrices brident l'application de la passe, principalement pour des raisons de risques d'aliasing mémoire ou de dépendances cycliques inter-itérations. Cette prudence entrave, dans certains cas complexes, la capacité de l'ordonnanceur à optimiser les boucles efficacement.

Afin de pallier ces limitations, nous avons conçu et implémenté la passe `ArrayReg2MemPass`. Son objectif est de rétrograder les registres optimisés sous la forme SSA des tableaux préalablement annotés vers des accès mémoire explicites. L'idée est de briser les dépendances

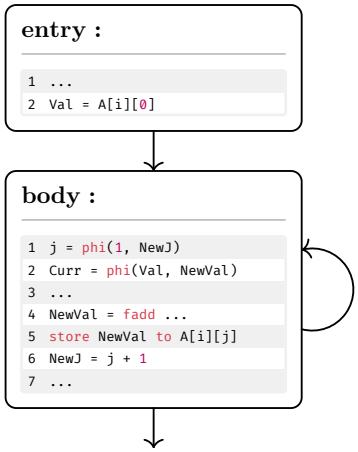
scalaires directement au niveau de l'IR afin de rétablir une structure sémantique plus proche de celle du code source original. Cette passe cible spécifiquement quatre types de dépendance au sein des boucles :

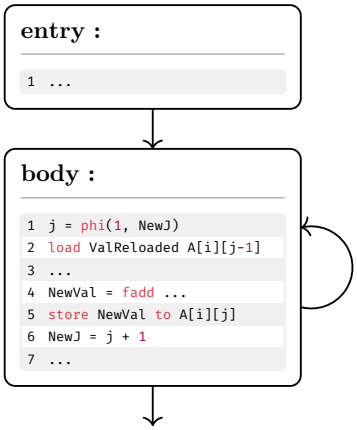
- la dépendance scalaire sortante d'une boucle;
- la dépendance scalaire inter-itérations;
- la dépendance scalaire intra-bloc qui se décline en deux variantes : la lecture réutilisée (une valeur chargée puis utilisée plusieurs fois) et les écritures multiples (une valeur écrite à plusieurs adresses distinctes);
- la dépendance scalaire étendue.

Le tableau 2 présente ces différents types de structures de dépendances ainsi que l'impact de l'application de notre passe sur l'IR.

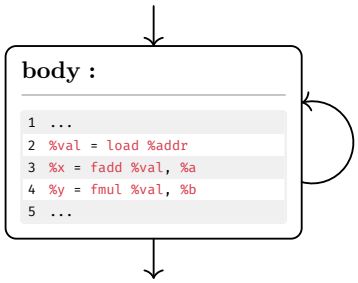
Description du type de dépendance	**Pseudo IR avant**	**Pseudo IR après**
Dépendance scalaire sortante de la boucle : dépendance liée à la valeur de `NewVal`, calculée au sein de la boucle mais utilisée après son exécution.		

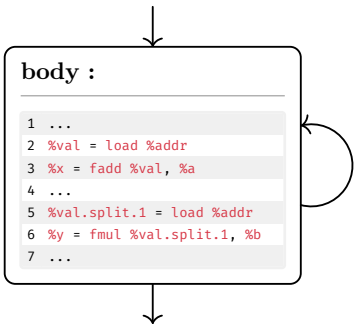
Dépendance scalaire inter-itérations :
à chaque itération du corps de la boucle `body`, une nouvelle valeur `NewVal` est calculée pour être exploitée lors de l'itération suivante.



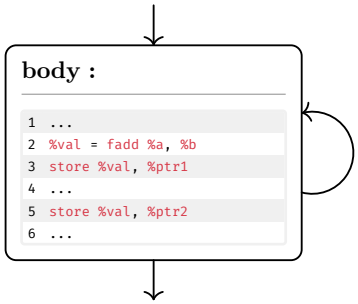


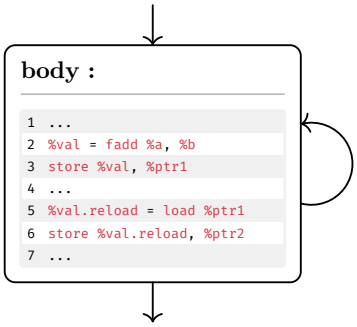
Dépendance scalaire intra-bloc (lecture réutilisée) :
la valeur `val` est chargée et utilisée à de multiples reprises au sein du même bloc. Cette configuration induit une limitation pour l'ordonnanceur : les instructions appartenant au même statement, elles sont indissociables et ne peuvent être réordonnancées indépendamment.





Dépendance scalaire intra-bloc (écritures multiples) :
la valeur `val` est écrite à plusieurs reprises à des adresses distinctes, générant des dépendances inter-instructions scalaires si le bloc est scindé en plusieurs statements.





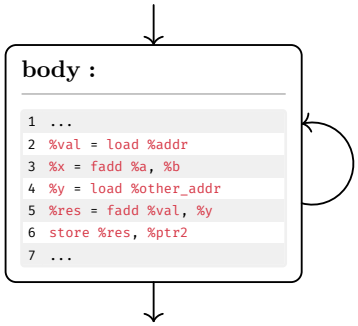
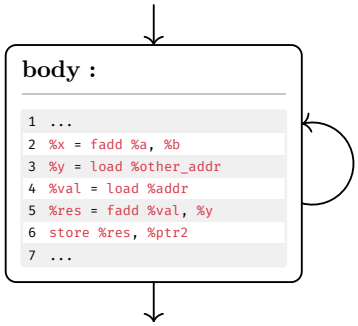
Dépendance scalaire étendue : le chargement de `%val` est dissocié de son utilisation. Lors du découpage des blocs LLVM en statements par Polly, l'instruction de chargement mémoire se retrouve dans un statement distinct de celui où la valeur est consommée. Cette séparation spatiale engendre une dépendance scalaire entre ces deux statements.		

Tableau 2. – Liste de transformations de dépendances scalaires vers des accès mémoire explicites, appliquées par la passe `ArrayReg2MemPass`.

En définitive, l'application de la passe `ArrayReg2MemPass` s'avère essentielle pour lever les verrous structurels induits par les abstractions de Kokkos. En remplaçant ces dépendances scalaires artificielles par des accès mémoire explicites, cette transformation affranchit l'ordonnanceur polyédrique de contraintes de séquencement superflues. Elle rend ainsi possible l'optimisation de codes qui, autrement, auraient été bridés par la présence de fausses dépendances.

4.3.4. Délinéarisation

Afin de construire le domaine d'exécution, le modèle polyédrique doit impérativement reconstruire une représentation multidimensionnelle exacte des accès mémoire, exprimée sous forme de fonctions affines des itérateurs sur chaque dimension, à partir des calculs de pointeurs linéarisés, présents sous forme polynomiale dans l'IR. Polly se base pour cela sur l'analyse d'évolution scalaire (**Scalar Evolution**, SCEV) de LLVM, qui modélise l'évolution des adresses d'accès via une chaîne récursive d'additions (**AddRec**) indexée par les compteurs de boucle. La délinéarisation repose ensuite sur une division itérative de l'expression d'accès par les tailles des dimensions successives du tableau (section 2.5.1.3.1).

Dans le cas d'un accès complet impliquant l'ensemble des dimensions (par exemple `A[i][j][k]`, pour des éléments de 8 octets), la SCEV produit une expression polynomiale imbriquée (voir figure 13a). L'algorithme standard de délinéarisation décompose ensuite ce polynôme via des divisions successives par les tailles respectives des dimensions afin de recouvrer les indices affines canoniques `[i][j][k]`.

Cependant, cette heuristique algébrique présente une faille structurelle majeure lorsqu'elle est confrontée à des accès dont certains indices sont statiquement évalués à zéro (par exemple, `A[0][j][k]`). Dans une telle configuration, l'indice de la dimension externe étant nul (`i = 0`), le terme polynomial contenant la taille de la dimension (`%n`) s'annule lors des simplifications algébriques opérées par LLVM (voir figure 13b). L'expression SCEV résultante ne contenant plus cette taille intermédiaire, le solveur s'avère incapable de la déduire par factorisation. Cette perte d'information génère des accès mémoire ambigus : un même pointeur peut alors être interprété avec un nombre variable de dimensions selon le motif d'accès étudié.

```llvm
; Expression SCEV générée pour un accès complet A[i][j][k] :
{{{ %A, +, (8 * %n * %o) }<%for.i>, +, (8 * %o) }<%for.j>, +, 8 }<%for.k>
```

(a) Expression SCEV pour un accès complet (`A[i][j][k]`). Les tailles de dimensions `%n` et `%o` sont présentes et peuvent être déduites algébriquement.

```llvm
; Expression SCEV générée pour un accès partiel A[0][j][k] :
{{ %A, +, (8 * %o) }<%for.j>, +, 8 }<%for.k>
```

(b) Expression SCEV pour un accès partiel (`A[0][j][k]`). La disparition du terme contenant `%n` provoque l'échec de la délinéarisation standard.

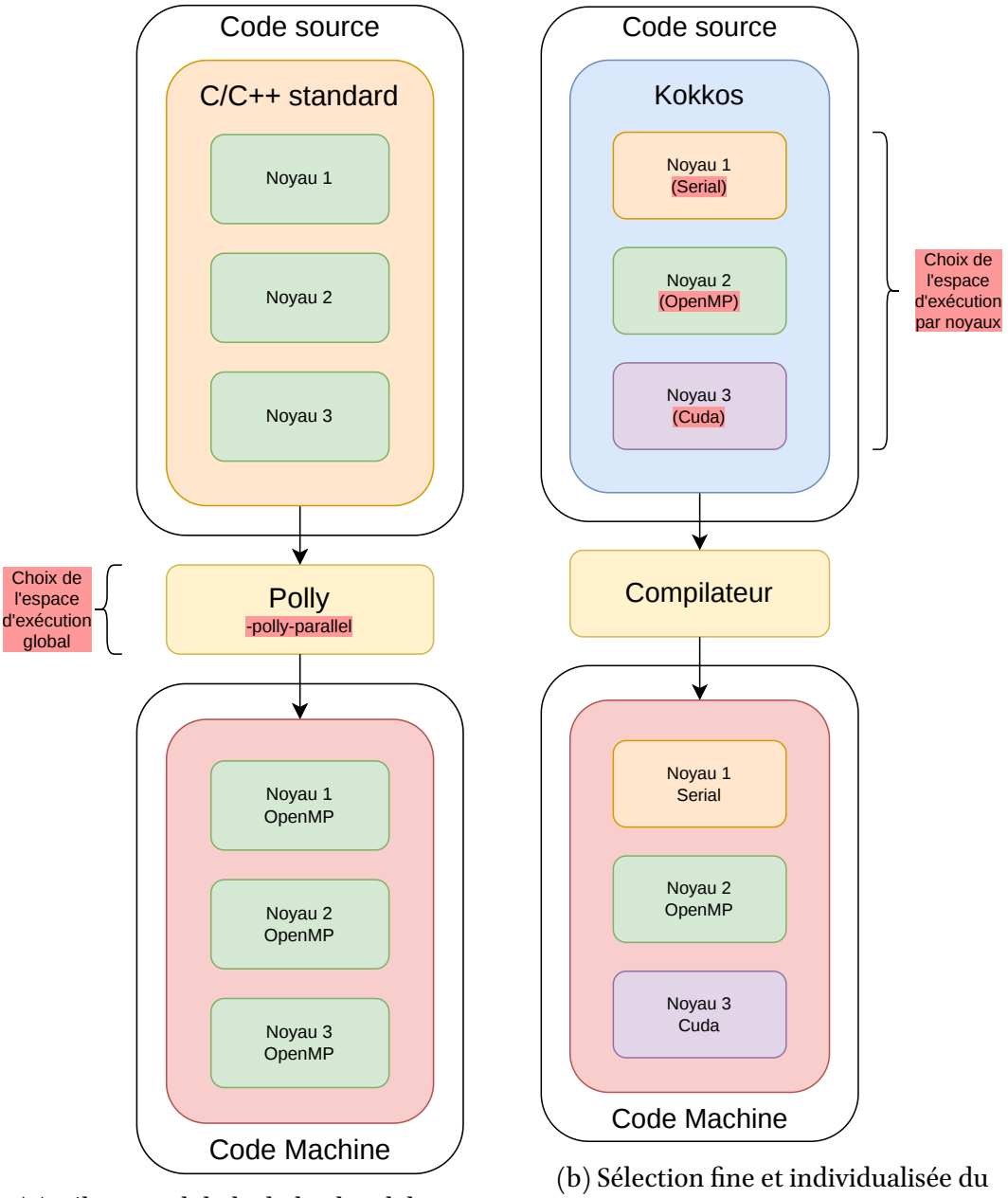
Figure 13. – Comparaison des expressions d'évolution scalaire (SCEV) générées par LLVM pour un tableau tridimensionnel. Lors d'un accès partiel, la simplification algébrique entraîne une perte d'information structurelle que seules nos annotations permettent de restaurer.

Le recours aux annotations sémantiques, introduites lors de la génération des noyaux Kokkos, lève définitivement ce verrou. En communiquant explicitement au compilateur la taille exacte de chaque dimension (par exemple `%m`, `%n` et `%o`) sans dépendre de la reconstruction depuis l'IR, nous rendons ce processus de délinéarisation entièrement déterministe et fiable. Cette

approche présente également un second avantage majeur pour le développeur : lors de l'analyse du SCoP, les accès mémoire reconstruits correspondent fidèlement à la sémantique multidimensionnelle du code source original, facilitant ainsi la traçabilité.

4.3.5. Synchronisation du backend

Dans son fonctionnement standard, le choix de la stratégie de génération de code par Polly est dicté par une option de compilation globale (telle que `-polly-parallel`). Comme l'illustre la figure 14a, cette approche force l'application uniforme du même modèle d'exécution à l'intégralité du module LLVM, en présumant l'usage d'un CPU comme architecture cible unique. Cette vision monolithique entre en conflit direct avec le modèle de Kokkos, où une même application peut répartir l'exécution de ses noyaux de calcul sur des espaces d'exécution hétérogènes (faisant par exemple cohabiter des calculs séquentiels sur CPU et des calculs parallèles sur GPU), chaque noyau ciblant son propre backend (figure 14b). Notre objectif est donc de répliquer ce fonctionnement au sein de Polly afin de synchroniser la génération de code de l'optimiseur avec les choix de backends définis dans le code source Kokkos.



(a) Sélection globale du backend dans Polly standard via une option de compilation unique.

(b) Sélection fine et individualisée du backend par noyaux Kokkos.

Figure 14. – Comparaison de la sélection du modèle d'exécution entre l'approche globale monolithique de Polly et l'approche fine par noyau de Kokkos.

Afin de s'affranchir de cette contrainte globale, l'indicateur `-polly-parallel` étant désormais ignoré par notre chaîne de compilation, un nouveau mécanisme de contrôle à grain fin a été introduit. Par l'intermédiaire du système d'annotations décrit à la section 4.3.1, c'est

désormais chaque noyau Kokkos qui informe explicitement le compilateur de son architecture matérielle cible et qui définit si le SCoP correspondant doit être exécuté en parallèle. Notre implémentation permet actuellement de cibler trois environnements d'exécution distincts : **Serial** (exécution séquentielle), **OpenMP** et **CUDA**. La génération de code pour l'architecture CUDA, qui n'existe pas nativement au sein de Polly, a nécessité une intégration spécifique qui est détaillée à la section 6.2.

Grâce à ces informations contextuelles, Polly acquiert la capacité d'isoler et d'adapter sa stratégie de transformation pour chaque SCoP de manière indépendante. Contrairement au comportement natif de Polly, cette synchronisation autorise la cohabitation, au sein d'un même binaire, de noyaux spécifiquement optimisés pour des architectures distinctes. Ce couplage préserve ainsi l'un des engagements fondamentaux de Kokkos : l'intégrité et la portabilité des performances.

Néanmoins, si cette infrastructure permet d'optimiser efficacement des noyaux réguliers et isolés, le traitement d'algorithmes plus complexes soulève de nouveaux défis pour l'optimisation polyédrique. D'une part, Polly doit être capable de traiter les nids de boucles triangulaires. D'autre part, lorsque les calculs sont fragmentés en de multiples appels consécutifs à la fonction `parallel_for`, le compilateur ne conserve qu'une vision locale de chaque portion du noyau, ce qui entrave l'application de transformations inter-noyaux. Nous détaillons au chapitre suivant (chapitre 5) les extensions conçues pour surmonter ces verrous et étendre notre approche à ces motifs d'exécution complexes.

Chapitre 5

Motifs d'exécution complexes

Le chapitre 4 a présenté la mise en place du pipeline de compilation, permettant d'extraire et de modéliser les informations structurelles nécessaires à l'application de transformations polyédriques sur des noyaux de calcul simples écrits avec Kokkos. Toutefois, de nombreux noyaux de calcul présentent des schémas d'exécution plus complexes qui résistent à cette première approche. D'une part, les boucles triangulaires, courantes dans le calcul scientifique, échouent à être correctement détectées et modélisées par Polly en raison de l'introduction de branchements irréguliers lors de leur abaissement dans la représentation intermédiaire. D'autre part, les algorithmes nécessitant plusieurs étapes de calcul imposent d'enchaîner de multiples appels consécutifs à la fonction `parallel_for`. Cette fragmentation structurelle empêche le compilateur d'acquérir une vision globale du noyau de calcul, restreignant ainsi l'application de transformations inter-noyaux telles que la fusion de boucles.

Ce chapitre présente nos solutions pour étendre l'applicabilité du modèle polyédrique à ces motifs complexes. Dans un premier temps, la section 5.1 détaille les défis liés aux nids de boucles triangulaires et introduit une technique d'épluchage statique de boucle pour régulariser le flot de contrôle. Dans un second temps, la section 5.2 aborde la problématique de la fragmentation des noyaux et décrit les mécanismes développés pour réunifier les noyaux de calcul en un seul `parallel_for` ainsi que la représentation intermédiaire associée. Enfin, nous introduisons une méthode permettant de transmettre des informations sémantiques explicites sous forme de contraintes, afin de consolider l'analyse inter-noyaux.

5.1. Boucles triangulaires

Les boucles triangulaires constituent un motif d'exécution fondamental et récurrent dans les applications de calcul scientifique, notamment dans les implémentations d'algorithmes d'algèbre linéaire. Elles se caractérisent par des boucles imbriquées dont les bornes d'itération internes dépendent directement des variables d'induction des boucles englobantes. Par exemple, la liste 13 illustre le noyau de résolution de système triangulaire (**trisolv**) implémenté en Kokkos. Dans ce noyau, la boucle interne sur l'indice `j` a pour borne supérieure l'itérateur `i` de la boucle externe. Ainsi, l'espace d'itération interne évolue dynamiquement à chaque étape de la boucle externe, décrivant un espace de forme triangulaire.

```cpp
void function(Kokkos::View<"L", double **> L,
              Kokkos::View<"x", double *> x,
              Kokkos::View<"b", double *> b,
              size_t n) {
  auto policy = Kokkos::RangePolicy<OpenMP>(0, n);

  Kokkos::parallel_for<usePolyOpt>(
      policy, KOKKOS_LAMBDA(const size_t i) {
        x(i) = b(i);                        // Stmt1
        for (size_t j = 0; j < i; j++)
          x(i) -= L(i, j) * x(j);           // Stmt2
        x(i) = x(i) / L(i, i);              // Stmt3
      });
}
```

Liste 13. – Exemple de code Kokkos présentant un motif de boucle triangulaire (noyau **trisolv**).

Bien que ce motif soit sémantiquement simple, son analyse par les compilateurs s'avère complexe. En associant ces boucles aux abstractions de Kokkos, la linéarité apparente des accès mémoire se perd au niveau de la représentation intermédiaire, empêchant directement la détection de telles régions par les outils polyédriques.

5.1.1. Problèmes liés à la représentation intermédiaire

L'implémentation de ces boucles triangulaires pose un défi majeur lors de l'abaissement en représentation intermédiaire LLVM vis-à-vis de l'analyse polyédrique. La figure 15 montre le graphe de flot de contrôle et l'IR LLVM simplifiée générés par le compilateur pour le noyau **trisolv** (liste 13).

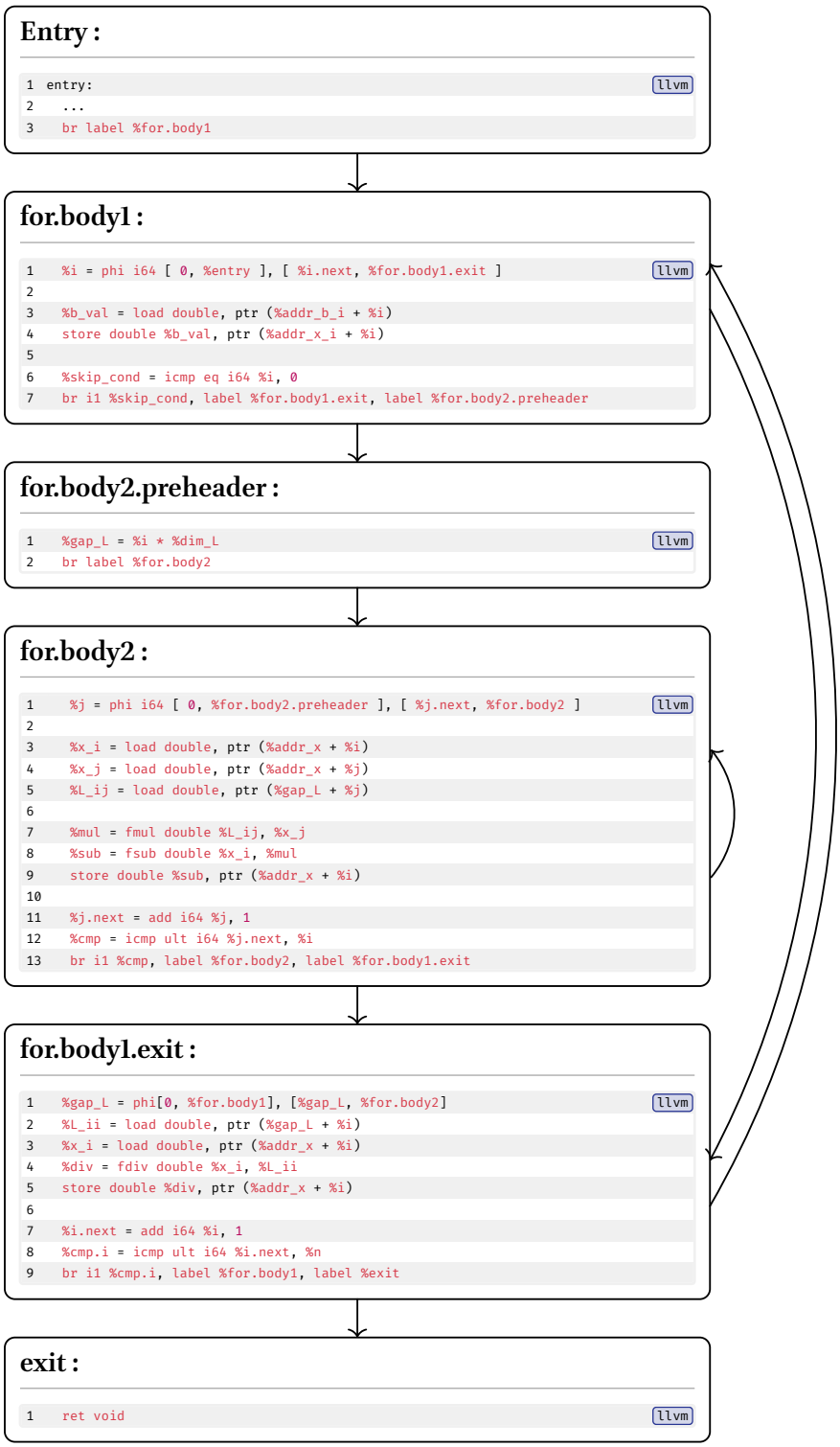


Figure 15. – Représentation simplifiée en IR LLVM du noyau **trisolv** (liste 13).

Lors du processus de compilation, un branchement conditionnel est inséré dans le bloc de base `for.body1` afin d'ignorer la boucle interne lors de la première itération de la boucle externe (lorsque $i = 0$), l'espace d'itération de l'itérateur j étant alors vide. L'insertion de ce branchement autorise le compilateur à optimiser les accès au tableau L en extrayant

un décalage invariant (`%gap_L`) pour la boucle j. Par conséquent, le bloc de convergence `for.body1.exit` doit traiter deux cas de figure distincts : si l'itérateur `%i` est égal à 0, l'accès à L s'effectue avec un décalage statique nul et dans le cas contraire, il s'opère avec le décalage constant `%gap_L`.

Lorsque la phase de détection de Polly examine ces accès mémoire, elle s'appuie sur l'analyse d'évolution scalaire (SCEV) de LLVM. Pour l'accès `L(i, i)` situé dans le bloc `for.body1.exit`, l'analyse produit l'évolution scalaire de LLVM suivante :

$$((8 * \%\text{gap_L}) + \{0, +, 8\}<\%\text{for.body1}>) \tag{14}$$

où `%gap_L` représente la distance en mémoire entre deux lignes consécutives du tableau L.

L'apparition de la variable `%gap_L` dans l'expression de l'accès mémoire est perçue comme un scalaire opaque par la passe d'analyse SCEV. À ce stade de la compilation, Polly est dans l'incapacité de déduire que `%gap_L` constitue en réalité le produit de la taille de la dimension interne du tableau L, qui est une constante (une propriété qui ne sera démontrée que lors de l'étape ultérieure `ScopInfo`) et de l'itérateur `%i`. Afin de garantir la validité et la sûreté de ses transformations, Polly adopte une approche conservatrice et rejette cet accès qu'il classifie comme non affine. Cette décision entraîne inéluctablement le rejet de la région entière, empêchant de fait l'application de toute optimisation polyédrique.

Afin de pallier cette limitation inhérente sans nécessiter de modification profonde de l'architecture de détection de Polly, et pour neutraliser d'autres structures d'IR susceptibles de corrompre l'analyse, nous avons introduit une passe d'épluchage de boucle en amont du processus de détection des SCoPs.

5.1.2. Épluchage de boucles

L'épluchage de boucle (ou **loop peeling**) est une transformation consistant à extraire une ou plusieurs itérations spécifiques en début ou en fin d'exécution, hors de la boucle principale. L'objectif est d'isoler un comportement marginal pour uniformiser l'espace d'itérations principal restant.

Afin de pallier les irrégularités induites par les boucles triangulaires, cette technique est mise à profit au travers d'une passe de transformation dédiée. Son rôle est d'analyser

la représentation intermédiaire pour cibler et éliminer systématiquement les branchements « parasites » du flot de contrôle. Le processus s'articule autour de trois étapes principales :

1. la détection des boucles triangulaires,
2. le calcul du nombre d'itérations à éplucher pour chaque boucle,
3. la simplification des conditions résiduelles.

5.1.2.1. Détection des nids de boucles triangulaires

L'identification d'une boucle triangulaire s'appuie sur l'analyse de l'évolution scalaire de LLVM (SCEV). Le compilateur évalue le nombre de passages par l'arc retour (nombre d'itérations - 1), appelé *Backedge Taken Count* (BETC). Dans le cas d'une boucle rectangulaire classique, cette valeur est invariante par rapport à la boucle externe. À l'inverse, si le compilateur détecte que l'expression SCEV du BETC dépend de la variable d'induction de la boucle englobante (exprimée par exemple sous la forme d'une récurrence `AddRec`), la boucle est formellement identifiée comme triangulaire.

De manière générale, on distingue deux grands types de boucles triangulaires en fonction de l'évolution de l'espace d'itération interne. L'analyse SCEV du BETC permet d'identifier formellement ces deux profils et d'en déduire à la fois la stratégie d'épluchage adéquate et le nombre exact d'itérations à extraire :

- **Triangulaire croissante** (épluchage par l'avant ou **front peeling**) : La taille de la boucle interne augmente avec l'itérateur externe. L'expression SCEV du BETC prend la forme d'une récurrence à pas positif `{Start, +, Step}<%for.i>`. Si l'évaluation de cette expression pour la première itération (la valeur `Start`) est strictement négative, le corps de la boucle interne ne s'exécute pas initialement. Un épluchage par l'avant est alors requis pour extraire l'ensemble des premières itérations. Le nombre exact d'itérations à éplucher correspond au nombre de pas nécessaires pour atteindre un décompte positif ou nul.
- **Triangulaire décroissante** (épluchage par l'arrière ou **back peeling**) : À l'inverse, la taille de la boucle interne diminue à mesure que l'itérateur externe avance. Le BETC se traduit par une expression avec un pas négatif `{Start, +, -Step}<%for.i>`. Le nombre d'itérations internes décroît jusqu'à s'annuler puis devenir négatif vers la fin de l'espace d'itération englobant, ce qui nécessite un épluchage par l'arrière. Le nombre d'itérations à éplucher

est déterminé en calculant d'abord le nombre d'itérations où la boucle interne s'exécute effectivement. Ce nombre d'itérations effectives est ensuite soustrait du nombre total d'itérations de la boucle englobante pour obtenir exactement le nombre d'itérations à éplucher.

Cette méthode analytique permet de caractériser complètement les boucles à traiter et d'éliminer de façon ciblée les itérations non exécutées du corps principal.

5.1.2.2. Application de l'épluchage de boucle

L'étape d'analyse précédente permet de recenser toutes les boucles triangulaires et de déterminer le nombre d'itérations à éplucher requis pour chacune d'entre elles. Néanmoins, un noyau de calcul peut encapsuler plusieurs boucles internes triangulaires distinctes. Il est alors fréquent que plusieurs d'entre elles requièrent l'épluchage de la même boucle englobante, avec des nombres d'itérations potentiellement différents en amont ou en aval de la boucle.

Pour consolider ces requêtes, la passe génère un plan d'épluchage. Pour chaque boucle externe ciblée et chaque direction (en amont ou en aval), le compilateur calcule la borne supérieure des requêtes en retenant le nombre maximum d'itérations à éplucher. Cette approche garantit que la boucle englobante restante sera totalement purgée de ses itérations mortes, satisfaisant ainsi les contraintes de toutes les boucles internes qu'elle contient.

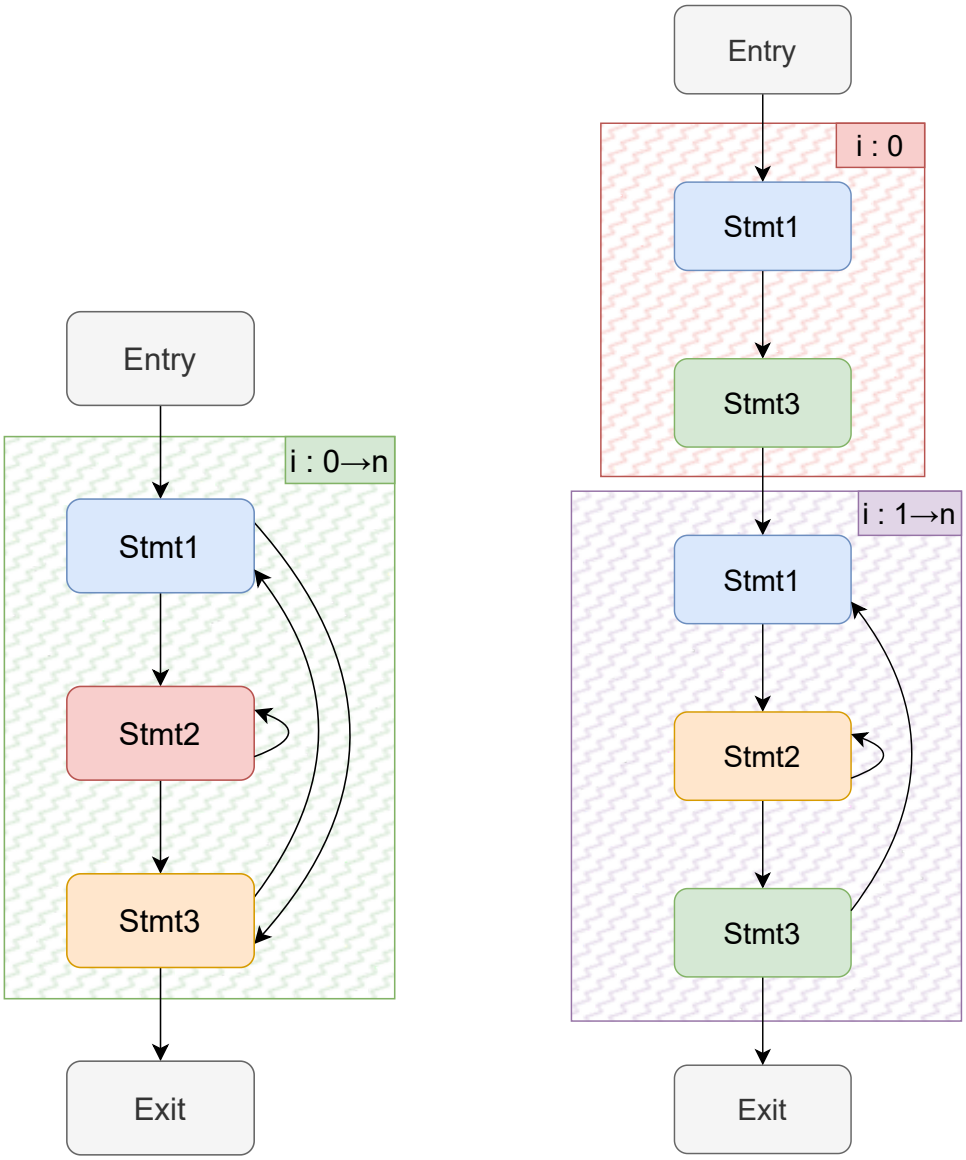
Une fois ce plan d'épluchage fusionné, la modification de l'IR est appliquée via la fonction utilitaire `peelLoop` native de LLVM. Une étape supplémentaire est toutefois nécessaire pour l'épluchage en fin de boucle. En effet, lors de l'extraction par LLVM, les instructions du bloc épluché conservent leurs liens avec la variable d'induction d'origine (par exemple via un nœud `%i.peeled = phi [%i, %for.end]` où l'on utilise `%i` de la boucle originale comme valeur de départ de la nouvelle boucle épluchée). Pour isoler totalement ce bloc, notre passe remplace toutes les utilisations de l'ancien itérateur de la boucle originale par une constante dépendante de la borne de la boucle (par exemple `N-2`), puisque l'on connaît statiquement la valeur exacte d'initialisation du nouvel itérateur, pour supprimer toute fausse dépendance entre les deux boucles.

5.1.2.3. Régularisation du flot de contrôle

La simple extraction matérielle des itérations rend théoriquement certaines conditions de garde obsolètes. Bien que les passes de simplification standards de LLVM, exécutées en aval, puissent en résoudre une partie, elles ne parviennent pas systématiquement à détruire toutes les comparaisons générées par l'abaissement en IR. Afin de garantir formellement que la boucle résultante sera parfaitement analysable par les outils polyédriques, une étape de nettoyage est appliquée.

Cette intervention anticipée élimine définitivement les gardes conditionnelles parasites inhérentes aux boucles triangulaires. Le graphe de flot de contrôle redevient ainsi parfaitement régulier, résolvant de fait les problèmes précédents liés aux accès mémoire non linéaires. Cette régularisation autorise enfin la modélisation complète du noyau Kokkos par Polly.

L'impact de cette transformation est directement visible sur l'évolution du graphe de flot de contrôle : la structure initiale, représentée à la figure 16a, est ainsi convertie en une forme parfaitement régulière sans optimisation locale cassant la détection, comme le montre la figure 16b, la rendant compatible avec les analyses et transformations polyédriques de Polly.



(a) Avant l'épluchage de boucle.

(b) Après l'épluchage de boucle.

Figure 16. – Évolution du graphe de flot de contrôle du noyau **trisolv** suite à la passe d'épluchage statique.

5.2. Vision globale des boucles

Pour optimiser les performances des noyaux de calcul comportant plusieurs nids de boucles, le modèle polyédrique peut appliquer des transformations inter-noyaux. L'une d'elles est la fusion de boucles, généralement appliquée lorsque des noyaux successifs accèdent aux mêmes

données avec des dépendances de données inter-noyaux. Pour y parvenir, l'optimiseur doit être en mesure de capturer l'intégralité du contexte de calcul au sein d'un SCoP.

Cependant, l'application du modèle polyédrique sur des codes Kokkos se heurte à une incompatibilité fondamentale de conception. Alors que le modèle polyédrique nécessite une vue globale et unifiée du flux d'instructions pour appliquer des transformations efficaces, l'approche standard de Kokkos favorise l'isolation. En effet, la représentation de noyaux complexes contraint chaque nid de boucles à être encapsulé dans un appel distinct à un `parallel_for`. Cette fragmentation structurelle limite Polly à une vision locale de chaque noyau, empêchant de fait toute optimisation inter-noyaux.

La liste 14 illustre ce problème de conception à travers l'implémentation du noyau `gemver` de la suite PolyBench réécrit avec Kokkos. On constate que la logique algorithmique globale est nécessairement fragmentée en quatre sous-noyaux distincts.

```cpp
const auto policy_1D = Kokkos::RangePolicy<Kokkos::OpenMP>(0, n);
const auto policy_2D = Kokkos::MDRangePolicy<Kokkos::OpenMP,
                                             Kokkos::Rank<2>>({0, 0}, {n, n});

Kokkos::parallel_for(
    policy_2D, KOKKOS_LAMBDA(const INT_TYPE i, const INT_TYPE j) {
      A(i, j) += u1(i) * v1(j) + u2(i) * v2(j);
    });

Kokkos::parallel_for(
    policy_1D, KOKKOS_LAMBDA(const INT_TYPE i) {
      for (INT_TYPE j = 0; j < n; j++)
        x(i) += beta * A(j, i) * y(j);
    });

Kokkos::parallel_for(
    policy_1D, KOKKOS_LAMBDA(const INT_TYPE i) {
      x(i) += z(i);
    });

Kokkos::parallel_for(
    policy_1D, KOKKOS_LAMBDA(const INT_TYPE i) {
      for (INT_TYPE j = 0; j < n; j++)
        w(i) += alpha * A(i, j) * x(j);
    });
```

Liste 14. – Implémentation du noyau `gemver` en Kokkos, illustrant la fragmentation en de multiples appels à `parallel_for`.

Pour pallier cette limitation et fournir à Polly une vision globale d'un noyau de calcul fragmenté, nous avons conçu un nouveau motif `parallel_for` permettant de donner plusieurs noyaux ainsi qu'un pipeline spécifique de traitement de l'IR pour reconstruire un seul et unique SCoP.

5.2.1. Nouveau `parallel_for` variadique

Dans l'intégration de Polly avec Kokkos, la granularité de détection des SCoPs se limite au périmètre d'un appel unique à la fonction `parallel_for`. Pour étendre cette analyse à une séquence de noyaux, il est nécessaire que l'optimiseur puisse englober plusieurs de ces appels sans rencontrer d'instructions susceptibles d'interrompre la détection de la région affine. Afin de garantir cette détection et de générer un code intermédiaire structuré et prévisible, nous avons conçu une version variadique de la fonction `parallel_for`.

En exploitant les capacités de métaprogrammation du C++, cette implémentation variadique (illustrée par la liste 15) permet d'accepter une séquence de paires `(Policy, Functor)`. Lors de la compilation, une fonction lambda est instanciée pour chaque paire, et ces fonctions lambda sont ensuite invoquées séquentiellement.

```cpp
template <class... FunctorTypes>
__attribute__((noinline, annotate("multi_parallel_for")))
void call_for_parallel_for(FunctorTypes... funcs) {
  (funcs(), ...);
}

template <bool usePolyOpt,
          class... Args,
          std::size_t... Is>
inline void parallel_for_impl(const std::string& str,
                              const std::tuple<Args...>& args_tuple,
                              std::index_sequence<Is...>) {
  call_for_parallel_for(
      (Kokkos::Impl::construct_with_shared_allocation_tracking_disabled<
           Impl::ParallelFor<
               std::tuple_element_t<2 * Is + 1, std::tuple<Args...>>,
               std::tuple_element_t<2 * Is, std::tuple<Args...>>>>(
           std::get<2 * Is + 1>(args_tuple), std::get<2 * Is>(args_tuple))
           .template getExecute<usePolyOpt>())...);
}

template <bool usePolyOpt = false,
          class... Args,
          class Enable = std::enable_if_t<(sizeof...(Args) > 0 &&
                                           sizeof...(Args) % 2 == 0)>
                                         >
inline void parallel_for(const std::string& str, const Args&... args) {
  static_assert(sizeof...(Args) > 0,
                "parallel_for requires at least one policy/functor pair.");
  static_assert(sizeof...(Args) % 2 == 0,
                "parallel_for requires an even number of arguments "
                "(policy/functor pairs).");

  Impl::parallel_for_impl<usePolyOpt>(
      str, std::make_tuple(args...),
      std::make_index_sequence<sizeof...(Args) / 2>{});
}
```

Liste 15. – `parallel_for` variadique.

Statiquement, le compilateur génère pour chaque noyau de calcul une lambda annotée (grâce à `getExecute` implémenté dans chaque backend) avec les annotations nécessaires au modèle polyédrique (section 4.2.3), selon sa politique d'exécution. L'ensemble de ces lambdas, qui représente la globalité du calcul, est appelé séquentiellement grâce à `call_for_parallel_for` en s'affranchissant de structures de contrôle complexes susceptibles de perturber l'analyse de Polly grâce aux **fold expressions** du C++. Cette conception offre un point d'entrée unique et transparent pour l'utilisateur. Elle permet en outre de délimiter l'intégralité du calcul au sein

d'une seule portion d'IR, bornée par un appel de fonction LLVM spécifique annoté par un attribut de fonction.

Grâce à cette nouvelle implémentation, un noyau complexe tel que `gemver` (liste 14) peut être réécrit de manière unifiée, comme le montre la liste 16. Un unique appel au `parallel_for` variadique suffit désormais pour écrire l'ensemble du noyau.

```cpp
const auto policy_1D = Kokkos::RangePolicy<Kokkos::OpenMP>(0, n);
const auto policy_2D = Kokkos::MDRangePolicy<Kokkos::OpenMP,
                                             Kokkos::Rank<2>>({0, 0}, {n, n});

Kokkos::parallel_for<Kokkos::usePolyOpt>("gemver",
  policy_2D, KOKKOS_LAMBDA(const INT_TYPE i, const INT_TYPE j) {
    A(i, j) += u1(i) * v1(j) + u2(i) * v2(j);
  },
  policy_2D, KOKKOS_LAMBDA(const INT_TYPE i, const INT_TYPE j) {
    x(i) += beta * A(j, i) * y(j);
  },
  policy_1D, KOKKOS_LAMBDA(const INT_TYPE i) {
    x(i) += z(i);
  },
  policy_2D, KOKKOS_LAMBDA(const INT_TYPE i, const INT_TYPE j) {
    w(i) += alpha * A(i, j) * x(j);
  });
```

Liste 16. – Implémentation du noyau `gemver` en Kokkos, illustrant le regroupement en un unique appel au `parallel_for` variadique.

Cependant, le simple regroupement des noyaux au niveau du code source ne suffit pas. L'IR LLVM générée initialement demeure fragmentée en raison des multiples lambdas. Il est donc indispensable d'appliquer une série de transformations spécifiques sur cette IR afin que Polly puisse l'interpréter comme un unique SCoP.

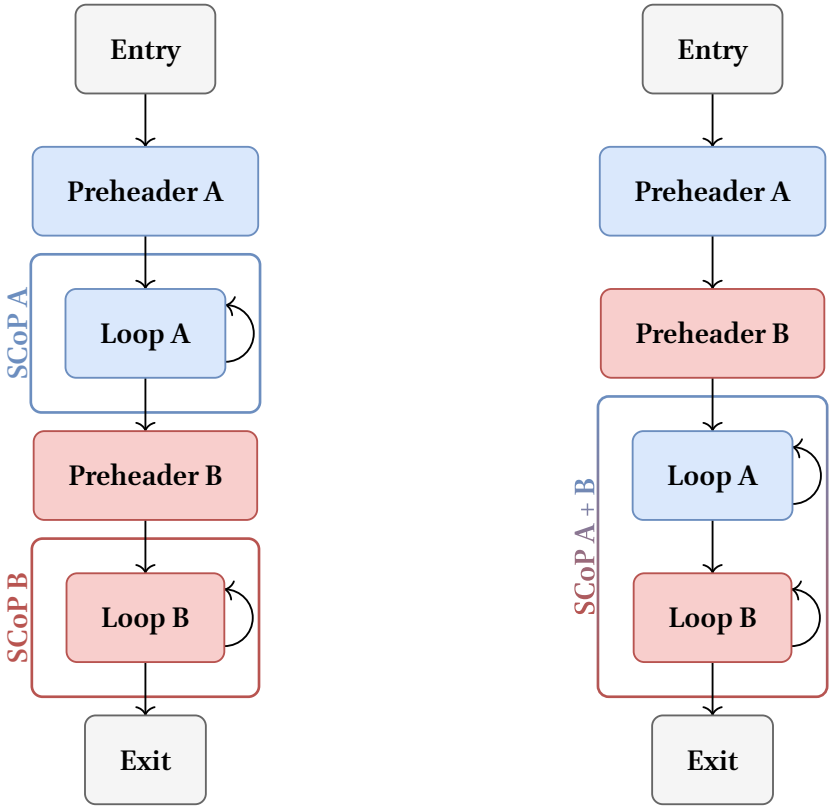
5.2.1.1. Linéarisation du flot de contrôle

La nouvelle implémentation présentée dans la section précédente permet à l'utilisateur de regrouper plusieurs noyaux en un seul appel à la fonction Kokkos `parallel_for`. Cette approche permet de regrouper physiquement les noyaux dans l'IR, mais ne suffit pas à elle seule pour garantir la reconnaissance de l'ensemble de ces noyaux comme un unique SCoP. En effet, lors de la compilation, LLVM traite chaque noyau individuellement et construit l'IR associée. Chaque noyau s'accompagne alors de ses propres chargements mémoire (bornes de boucles,

pointeurs de tableaux, paramètres, etc.), des instructions qui s'intercalent entre les régions de calcul et brisent l'analyse polyédrique de Polly. La figure 17a expose le graphe de flot de contrôle de l'IR pour deux noyaux de calcul exécutés via le `parallel_for` variadique. On y observe que les boucles sont séparées par le bloc `Preheader B`.

Dans un flot de compilation standard, le regroupement des noyaux n'apporte pas de gain de performance direct, par conséquent, le compilateur ne réalise aucune transformation spécifique visant à les fusionner. Cependant, notre implémentation du `parallel_for` variadique offre de fortes garanties structurelles. Grâce aux annotations, à la régularité imposée par l'appel variadique, et à l'impossibilité pour l'utilisateur d'insérer des instructions entre les noyaux, nous sommes en mesure d'opérer une véritable fusion des noyaux Kokkos directement au niveau de la représentation intermédiaire en remontant les blocs d'instructions problématiques au-dessus du SCoP global.

La figure 17b présente l'IR après l'application de cette passe de transformation. Le bloc `Preheader B` a été déplacé pour être exécuté juste après le `Preheader A` et avant la boucle `Loop A`. Le bloc remonté ne charge que des captures et des champs de politique que le noyau A n'écrit pas. Ces blocs peuvent ainsi être déplacés sans risquer de corrompre la sémantique du programme.



(a) Flot de contrôle initial fragmenté en multiples SCoPs.

(b) Flot de contrôle linéarisé formant un unique SCoP.

Figure 17. – Transformation de l'IR illustrant la linéarisation du flot de contrôle et la fusion des noyaux.

Grâce à cette transformation, Polly est en mesure de reconnaître l'ensemble des noyaux fusionnés comme un seul et unique SCoP. Toutefois, le modèle d'abstraction de Kokkos pose une dernière difficulté : il empêche Polly d'identifier correctement les paramètres du modèle (bornes de boucles, tableaux, etc.). En effet, chaque valeur du code source fait l'objet d'un chargement mémoire distinct pour chaque noyau du fait de la capture par lambda des noyaux, conduisant à la création de multiples variables SSA différentes pour une même variable logique. Les sections suivantes exposent les méthodes déployées pour résoudre ce problème.

5.2.2. Nommage statique des tableaux

L'encapsulation d'un noyau de calcul au sein de plusieurs lambdas C++ engendre un problème majeur pour l'analyse globale de la mémoire : la duplication des pointeurs capturés. Lorsqu'un même tableau logique (représenté par une `View` Kokkos) est accédé par plusieurs noyaux successifs, chaque lambda capture ses propres variables. Au niveau de la

représentation intermédiaire, le compilateur traduit ces captures par la création de variables SSA distinctes pour chaque noyau. Par conséquent, bien que deux noyaux manipulent la même zone mémoire, l'IR présente des variables de pointeurs différentes. Cette divergence empêche Polly de construire un modèle cohérent des dépendances de données, compromet l'ordonnancement efficace des noyaux et introduit de faux problèmes d'aliasing à l'exécution, invalidant ainsi les vérifications dynamiques d'exécution.

Pour pallier ce problème, nous avons étendu l'API `View` de Kokkos en introduisant un mécanisme de nommage statique. Contrairement aux chaînes de caractères utilisées dynamiquement pour le débogage (les `labels` standards de Kokkos), ce nommage statique est intégré directement dans la signature du type de la vue par l'intermédiaire d'un paramètre template. L'implémentation de cette fonctionnalité s'appuie sur la structure `ConstExprLabel`, présentée par la liste 17.

```cpp
template <std::size_t N>
struct ConstExprLabel {
  char value[N];

  constexpr ConstExprLabel(const char (&str)[N]) {
    for (std::size_t i = 0; i < N; ++i)
      value[i] = str[i];
  }

  constexpr std::string_view view() const {
    return std::string_view(value, N - 1);
  }

  constexpr auto operator<=>(const ConstExprLabel&) const = default;
};

template <ConstExprLabel Label, class DataType, class... Properties>
class View;
```

Liste 17. – Définition de la structure `ConstExprLabel` permettant l'intégration de chaînes de caractères statiques dans les paramètres de *template* d'une `View`.

La structure `ConstExprLabel` exploite les fonctionnalités introduites par la norme C++20, autorisant l'instanciation de chaînes de caractères littérales dès la phase de compilation. Ainsi, chaque vue instanciée acquiert un type unique, défini non seulement par le type de ses éléments (`DataType`), mais également par l'identifiant statique qui lui est associé. Par exemple,

la déclaration suivante illustre la création d'une vue bidimensionnelle nommée statiquement `"A"` :

```C++
1  Kokkos::View<"A", double **> A(n, n);
```

Grâce à cette intégration au sein même de la signature du type de la donnée, le nom d'un tableau peut être déterminé statiquement en inspectant son type. Ainsi, lors de chaque accès mémoire effectué via Kokkos, une annotation est générée pour exposer non seulement le pointeur et les dimensions du tableau, mais également ce nom statique. L'analyse de ces annotations permet dès lors d'identifier les tableaux partageant un identifiant commun au sein de l'IR. L'égalité de ces noms statiques est une garantie de l'utilisateur que les vues partagent à la fois le même pointeur de données et les mêmes dimensions.

Une fois ces équivalences établies, nous appliquons une transformation globale sur l'IR en tirant parti de la méthode de remplacement de LLVM : *Replace All Uses With* (RAUW). Les références aux pointeurs dupliqués dans les noyaux ultérieurs sont systématiquement remplacées par le pointeur originel capturé lors de la première occurrence du tableau. Cette unification des variables SSA permet non seulement d'éliminer les redondances dans l'IR, mais aussi d'éradiquer les faux positifs liés à l'aliasing. Polly dispose ainsi d'une vision globale et parfaitement cohérente des différents accès mémoire, pour une analyse correcte du SCoP fusionné.

5.2.3. Injection de contraintes

Bien que la représentation intermédiaire soit dorénavant unifiée au sein d'un seul SCoP, les politiques d'exécution indépendantes conservent leurs propres copies locales des variables définissant les bornes de boucle. Statiquement, il est donc impossible de déduire depuis l'IR que ces différentes variables sont identiques dans le code source, ce qui empêche leur fusion dans l'IR. Afin que l'optimiseur puisse identifier que les bornes de différentes boucles sont identiques ou strictement liées (pour permettre la fusion, par exemple), il est nécessaire que l'utilisateur fournisse des contraintes explicites à Polly pour permettre d'interpréter correctement les relations entre les bornes des différentes boucles. Ce mécanisme, que nous appelons contraintes d'assomption, permet de compenser la perte d'informations sémantiques induite par la structure interne de Kokkos.

Pour injecter ces informations au niveau du modèle, le système de contraintes repose sur un argument template optionnel de la fonction `parallel_for`, positionné à la suite du paramètre `usePolyOpt`. La liste 18 illustre l'implémentation permettant de construire et de transmettre statiquement la chaîne de caractères contenant ces contraintes, en se basant sur la structure `StringAssumption`.

```C++
template <std::size_t N>
struct StringAssumption {
  constexpr StringAssumption(const char (&str)[N]) {
    std::copy_n(str, N, value);
  }
  char value[N];
};

template <bool usePolyOpt = false,
          StringAssumption StrAssumption = "",
          class ExecPolicy,
          class FunctorType,
          class Enable = std::enable_if_t<is_execution_policy<ExecPolicy>::value>
         >
inline void parallel_for(const std::string& str, const ExecPolicy& policy,
                         const FunctorType& functor);

template <bool usePolyOpt = false,
          StringAssumption StrAssumption = "",
          class... Args,
          class Enable = std::enable_if_t<(sizeof...(Args) > 0 &&
                                           sizeof...(Args) % 2 == 0)>
                                         >
inline void parallel_for(const std::string& str, const Args&... args);
```

Liste 18. – Implémentation de l'injection statique des chaînes de contraintes (**assumptions**) dans les fonctions de Kokkos.

Cette structure, à l'instar de la méthode employée pour le nommage des tableaux (section 5.2.2), permet de construire une chaîne de caractères, exploitable dès la phase de compilation. Par exemple, l'utilisation de ce paramètre s'illustre de la façon suivante :

```C++
Kokkos::parallel_for<Kokkos::usePolyOpt, "p0.l0 == 0">(...);
```

Afin de transmettre l'information contenue dans cette chaîne de caractères à l'optimiseur Polly, nous nous appuyons sur le système d'annotation présenté dans la section 4.3.1. Grâce à ce

mécanisme, le noyau est annoté dans l'IR. Cette annotation textuelle est ensuite directement extraite et traduite en contraintes réelles par notre passe de transformation.

Cette manipulation de contraintes est largement utilisée dans le domaine polyédrique car elle permet de définir simplement des polyèdres. La section suivante présente la grammaire de contraintes que nous utilisons pour contraindre les SCoPs à interpréter différentes variables comme une variable unique, et ainsi favoriser l'analyse et l'optimisation polyédriques de Polly.

5.2.3.1. Grammaire des contraintes

Afin de formaliser ces relations, nous définissons une grammaire de contraintes sous la forme suivante :

$$\text{Contrainte} \rightarrow \text{Opérande}_G \quad \text{Opérateur} \quad \text{Opérande}_D \tag{15}$$

Plusieurs contraintes peuvent ainsi être combinées en les séparant par des virgules.

Un opérande peut appartenir à l'une des trois catégories distinctes : une borne de boucle issue d'une politique (**Policy Loop Bound**), une borne de boucle interne (**Inner Loop Variable**), ou une valeur littérale (un entier constant).

La liste 19 illustre de manière concrète l'application de cette syntaxe au sein d'un code source. Dans cet exemple, la contrainte `"p0.u0 == n"` est injectée directement en argument du noyau `parallel_for` afin d'indiquer que la borne supérieure de la politique du premier et unique noyau est égale à la variable `n`. De plus, la macro `KOKKOS_LOOP_BOUND(n)` est utilisée pour exposer explicitement la borne de la boucle interne à l'analyse.

```cpp
void function(Kokkos::View<"A", double **> &A,
              Kokkos::View<"B", double **> &B,
              Kokkos::View<"x", double *> &x,
              Kokkos::View<"y", double *> &y,
              Kokkos::View<"tmp", double *> &tmp,
              double alpha, double beta, size_t n) {
  auto policy = Kokkos::RangePolicy<OpenMP>(0, n);

  Kokkos::parallel_for<usePolyOpt, "p0.u0 == n">(
    policy, KOKKOS_LAMBDA(const size_t i) {
      tmp(i) = SCALAR_VAL(0.0);
      y(i) = SCALAR_VAL(0.0);
      for (size_t j = 0; j < KOKKOS_LOOP_BOUND(n); j++) {
        tmp(i) = A(i, j) * x(j) + tmp(i);
        y(i) = B(i, j) * x(j) + y(i);
      }
      y(i) = alpha * tmp(i) + beta * y(i);
    });
}
```

Liste 19. – Exemple de code avec des contraintes injectées.

Référence aux bornes de politique. Pour faire référence à une borne de boucle spécifique définie par une politique d'exécution, nous utilisons la syntaxe `pN.TM`. Dans cette notation, `N` représente l'indice de la politique dans la séquence d'exécution $(0, 1, \ldots)$, `T` définit le type de la borne (`l` pour la borne inférieure ou **lower**, `u` pour la borne supérieure ou **upper**), et `M` correspond à la dimension de la boucle $(0, 1, \ldots)$. En pratique, cette approche s'appuie sur le système d'annotation des bornes de boucle (section 4.3.1), ce qui permet de repérer de manière fiable quelle borne correspond à quelle dimension directement dans l'IR.

Référence aux variables de boucles internes. Afin de cibler des variables utilisées au sein même du corps du noyau d'exécution (`parallel_for`), nous introduisons la macro `KOKKOS_LOOP_BOUND`. En arrière-plan, cette macro annote les instructions avec le nom de la variable tel qu'il apparaît dans le code source. Cette approche permet de manipuler un nom de variable depuis le code source et de le repérer de façon fiable dans l'IR, alors que cette information est habituellement perdue lors de la compilation. L'utilisation de cette macro apporte un avantage supplémentaire : si deux variables partagent l'annotation avec un nom identique dans deux noyaux du même `parallel_for`, elles désignent en général la même variable capturée par les lambdas, et donc la même valeur. Elles sont alors considérées comme

identiques par l'analyse, même si l'utilisateur n'a pas défini de contrainte explicite. Cette identification repose toutefois sur une hypothèse de confiance discutée à la section 5.2.4.

Opérateurs et utilisation. Les contraintes peuvent être définies à l'aide des opérateurs de comparaison standards (`==`, `<`, `>`, `<=`, `>=`). Afin d'appliquer ces contraintes, nous utilisons la fonction intrinsèque `llvm.assume` de LLVM. Celle-ci permet de transmettre des informations sémantiques directement aux passes d'optimisation de LLVM dans le but de simplifier le code généré. Ces indications sont par ailleurs interprétées nativement au sein du flux d'analyse de Polly.

Réduction du temps de compilation. Un avantage de cette approche réside dans la réduction du temps de compilation. Les algorithmes polyédriques présentent une complexité exponentielle vis-à-vis du nombre de paramètres présents dans le SCoP. En fusionnant les paramètres représentant les bornes de boucle, la dimensionnalité du système de contraintes transmis au solveur est mathématiquement réduite, ce qui diminue considérablement le coût et la durée de l'analyse lors de la compilation.

5.2.4. Limitations de la vision globale des noyaux

Bien que ces transformations permettent d'obtenir une vision globale et unifiée des noyaux de calcul, elles présentent actuellement certaines limites inhérentes à leur dépendance envers les informations fournies par l'utilisateur. En effet, que ce soit pour le nommage statique des tableaux, l'annotation des bornes de boucles internes ou l'injection de contraintes, le processus de compilation accorde une confiance absolue aux annotations sans pouvoir les vérifier formellement de manière statique.

Risques liés au nommage des tableaux. Le mécanisme de nommage statique des tableaux repose entièrement sur la justesse des déclarations du développeur. Il n'existe actuellement aucune vérification lors de l'analyse polyédrique pour s'assurer que deux vues partageant le même identifiant possèdent le même pointeur de base et partagent les mêmes dimensions. L'utilisation d'un nom erroné entraîne alors une fusion silencieuse de tableaux distincts au sein de l'IR, ce qui corrompt inévitablement l'analyse des dépendances et fausse l'exécution du programme.

Risques liés à l'annotation des bornes internes. L'identification de deux bornes annotées par `KOKKOS_LOOP_BOUND` repose uniquement sur le nom de la variable dans le code source. Or, deux annotations de même nom ne désignent pas nécessairement la même valeur : si une lambda déclare une variable locale qui masque la variable capturée (par exemple une variable locale `n` redéfinie dans le corps du noyau), les deux bornes portent le même nom mais des valeurs différentes. L'analyse les fusionne pourtant en un unique paramètre, ce qui conduit à des transformations invalides. Cette identification constitue donc une hypothèse de confiance supplémentaire envers le développeur.

Risques liés à l'injection de contraintes. De manière similaire, l'injection de contraintes s'appuie inconditionnellement sur les affirmations de l'utilisateur. Si ce dernier fournit une hypothèse erronée concernant les relations entre les bornes de boucle, le domaine d'itération s'en trouvera restreint de façon incorrecte lors de la modélisation mathématique, conduisant à des transformations invalides.

Vérification dynamique. Pour pallier ces vulnérabilités, une perspective d'évolution consisterait à introduire des vérifications à l'exécution. Des assertions dynamiques valideraient que les pointeurs des tableaux fusionnés sont bien identiques, que les bornes annotées sous un même nom ont la même valeur et que les bornes de boucles respectent les contraintes définies.

Ces extensions permettent ainsi à Polly de modéliser une part bien plus large des noyaux Kokkos. Toutefois, leur optimisation reste dépendante du flot standard de Polly, limité à l'ordonnanceur d'ISL et aux architectures CPU. Le chapitre 6 présente l'intégration de l'ordonnanceur Pluto ainsi que celle de PPCG pour cibler les accélérateurs GPU.

Chapitre 6

Ordonnancement et génération hétérogènes

L'extraction de la représentation polyédrique et la génération de code optimisé constituent les deux phases fondamentales du flot de compilation polyédrique. Dans son fonctionnement par défaut, Polly orchestre ces étapes en s'appuyant sur l'ordonnanceur de la bibliothèque ISL (section 2.5.1.4) avant la reconstruction de la représentation intermédiaire LLVM optimisée (section 2.5.1.6).

Cependant, cette infrastructure standard présente des limitations vis-à-vis des objectifs de notre chaîne de compilation, tant en matière de support matériel que de diversité des optimisations. D'une part, l'abstraction de programmation Kokkos tire une part essentielle de ses performances d'exécution sur accélérateurs graphiques (GPU) et machines multiprocesseurs généralistes à mémoire partagée, tandis que le flot standard de Polly cible exclusivement les processeurs multicœurs (CPU). D'autre part, Polly se limite aux heuristiques d'un unique ordonnanceur, celui d'ISL, restreignant l'espace des transformations de code explorées.

Afin de pallier ces limitations, nous avons étendu le compilateur en y intégrant un nouveau moteur d'ordonnancement. Ce chapitre expose cette démarche visant à diversifier les approches d'optimisation et les architectures cibles. La section 6.1 détaille l'intégration de l'ordonnanceur de Pluto pour étendre les stratégies d'ordonnancement sur architectures CPU. Ensuite, la section 6.2 présente l'intégration de PPCG, un outil d'optimisation polyédrique

permettant la génération de code pour accélérateurs matériels (GPU), répondant ainsi aux exigences de portabilité des performances inhérentes à Kokkos.

6.1. Intégration de Pluto

Dans le modèle polyédrique, l'ordonnancement associe chaque instance d'instruction à des dimensions temporelles et spatiales afin d'établir un ordre d'exécution global. La détermination d'un ordonnancement optimal sous contraintes étant un problème NP-complet, les compilateurs recourent nécessairement à des heuristiques. L'ordonnanceur d'ISL, utilisé par défaut dans Polly, est une évolution de l'approche de Pluto. Alors que l'algorithme classique de Pluto opère globalement pour extraire des bandes de boucles permutables favorisant intrinsèquement le tuilage et le parallélisme par front d'onde, ISL adopte une approche incrémentale. Tout en cherchant également à minimiser les distances de dépendance, ISL compose l'ordonnancement par sous-graphes et intègre l'algorithme de Feautrier [Fea92a, Fea92b] comme solution de repli.

De plus, Pluto supporte nativement des transformations géométriques avancées, à l'instar du tuilage en diamant (*diamond tiling*) [BBP17], non réalisables directement via ISL. Ces dernières permettent de maximiser la localité et l'exécution concurrente, particulièrement au sein des schémas de calcul de type *stencil*. Comme le souligne l'étude dans [TLG24], Pluto démontre régulièrement une capacité supérieure à extraire des opportunités de parallélisation sur des codes aux accès réguliers. Son intégration constitue ainsi une extension particulièrement pertinente pour notre flot de compilation.

Afin d'interfacer Pluto avec l'infrastructure de Polly, nous avons développé une passerelle d'exportation et d'importation reposant sur le format **OpenScop** [Bas]. Ce choix de conception est motivé par deux raisons fondamentales. Premièrement, Pluto étant conçu comme un compilateur source-à-source opérant sur du code C, il est agnostique à la représentation intermédiaire de LLVM. L'utilisation d'OpenScop permet de franchir cette barrière d'abstraction en fournissant un format d'échange formel et standardisé. Deuxièmement, cette standardisation induit un découplage fort entre l'infrastructure interne de Polly et le moteur d'ordonnancement externe. Une telle architecture garantit non seulement une interopérabilité robuste avec Pluto, mais assure également l'extensibilité du flot de compilation

en vue de l'intégration future d'autres ordonnanceurs polyédriques. En pratique, une fois le SCoP extrait de l'IR LLVM, Polly sérialise ce dernier au format OpenScop et délègue l'étape d'ordonnancement à Pluto. Ce dernier produit un nouvel ordonnancement optimisé qui est réimporté dans Polly, se substituant à l'ordonnancement classiquement généré par ISL.

6.1.1. Représentation OpenScop

Bien que Polly et Pluto modélisent tous deux les programmes selon le formalisme polyédrique, leurs représentations internes en mémoire divergent structurellement. La bibliothèque ISL, noyau mathématique de Polly, encode les domaines d'itération et les fonctions d'ordonnancement sous forme d'ensembles et de relations paramétrés, décrits par des formules algébriques. À l'inverse, OpenScop repose sur une représentation matricielle de ces contraintes linéaires.

Afin de concevoir une passerelle fiable entre les structures ISL et le format OpenScop, nous nous appuyons sur la bibliothèque `libopenscop`. Durant la phase d'exportation, notre interface traverse récursivement les structures de données d'ISL afin d'en extraire les contraintes géométriques, reformulant ainsi un SCoP sémantiquement équivalent sous la forme de matrices OpenScop.

À titre d'illustration, considérons le domaine d'itération et l'ordonnancement canonique d'une boucle parfaitement imbriquée parcourant un espace bidimensionnel. Au sein d'ISL, cette modélisation s'exprime selon une syntaxe quasi-littérale, comme l'illustre la liste 20.

```ISL
Domain: { Stmt[i, j] : 0 <= i < N and 0 <= j < N }
Schedule: { Stmt[i, j] -> [i, j] }
```

Liste 20. – Représentation ensembliste et relationnelle d'un domaine et de son ordonnancement sous ISL.

Lors de la conversion, cette représentation géométrique est abaissée sous forme de matrices d'entiers, dont le résultat est détaillé dans la liste 21. Concernant le domaine d'itération, chaque ligne de la matrice correspond à une contrainte affine. Les colonnes décrivent successivement : le type de contrainte (1 pour une inégalité ≥ 0, 0 pour une égalité), les coefficients associés aux variables d'itération (i, j), les coefficients rattachés aux paramètres globaux (N), et enfin la constante de la fonction affine. L'ordonnancement fait l'objet d'une transformation analogue,

traduite au sein d'une matrice d'ordonnancement. Les colonnes additionnelles y représentent les nouvelles dimensions temporelles ou spatiales.

```
<DOMAIN>                                                    OpenScop
4 5
# e/i   i    j    N    1
   1     1    0    0    0    # i >= 0
   1    -1    0    1   -1    # N - i - 1 >= 0
   1     0    1    0    0    # j >= 0
   1     0   -1    1   -1    # N - j - 1 >= 0

<SCATTERING>
2 7
# e/i  t1   t2    i    j    N    1
   0     1    0   -1    0    0    0    # t1 - i == 0
   0     0    1    0   -1    0    0    # t2 - j == 0
```

Liste 21. – Traduction matricielle du domaine et de l'ordonnancement au format OpenScop.

6.1.1.1. Conversion du tuilage

Un verrou technique majeur lors de la réimportation de l'ordonnancement optimisé réside dans l'incompatibilité sémantique de la gestion des domaines d'itération entre les deux outils à la suite du tuilage de boucles. En effet, pour exprimer mathématiquement les tuiles, Pluto augmente explicitement la dimensionnalité du domaine d'itération en y introduisant de nouvelles variables représentant les tuiles. L'importation naïve de ces dimensions supplémentaires au sein de Polly entraînerait une corruption de la représentation interne, ces itérateurs fantômes n'ayant aucune correspondance dans la représentation intermédiaire LLVM d'origine. À l'opposé de cette approche, le flot natif de Polly (basé sur les arbres d'ordonnancement d'ISL) maintient une séparation stricte : le domaine d'itération demeure invariant aux transformations. Les transformations telles que le tuilage y sont encodées de manière purement relationnelle au sein de la fonction d'ordonnancement. Les nouvelles dimensions sont instanciées localement dans la structure arborescente, par l'intermédiaire de nœuds spécifiques de type **band**.

Afin de résoudre cette incompatibilité structurelle, nous avons conçu une passe de conversion intermédiaire destinée à combler ce fossé sémantique. L'algorithme de conversion procède à une rétro-ingénierie de la fonction de dispersion de Pluto afin d'isoler les dimensions relatives aux tuiles, pour ensuite les réinjecter sous forme de variables locales au sein de

la relation d'ordonnancement d'ISL. Cette méthodologie permet de garantir l'immuabilité du domaine d'itération originel, tout en préservant scrupuleusement l'ordre d'exécution lexicographique défini par Pluto.

La séquence de figures suivante illustre les différentes étapes de cette transformation géométrique pour un *statement* extrait du noyau `syr2k` de la suite PolyBench :

- La liste 22 expose le domaine d'itération et l'ordonnancement identité extraits initialement par Polly.
- La liste 23 détaille l'ordonnancement optimisé restitué par Pluto. On y observe l'altération du domaine d'itération par l'ajout explicite des dimensions de tuiles (variables `fk0`, `fk1`, `fk2`).
- La liste 24 présente l'ordonnancement final après l'application de notre passe de conversion. Le domaine d'itération d'origine est rigoureusement restauré. Les contraintes algébriques définissant les tuiles (identifiées par les variables `o1`, `o2`, `o3`) sont désormais confinées à la relation d'ordonnancement.

```
Domaine original :                                                    ISL
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[i0, i1, i2] : i0 < p_1 and 0 <= i1 < p_0 and 0 <= i2 <= i0
}

Ordonnancement original :
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[i0, i1, i2] -> [i0, 1, i1, i2]
}
```

Liste 22. – Domaine et ordonnancement originaux générés par Polly avant optimisation.

```
Domaine Pluto :                                                    OpenScop
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[fk0, fk1, fk2, i0, i1, i2] :
    32fk0 <= i0 <= 31 + 32fk0 and
    i0 < p_1 and
    i1 >= 32fk2 and
    0 <= i1 <= 31 + 32fk2 and
    i1 < p_0 and
    i2 >= 32fk1 and
    0 <= i2 <= i0 and
    i2 <= 31 + 32fk1
}

Ordonnancement Pluto :
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[fk0, fk1, fk2, i0, i1, i2] -> [1, fk0, fk1, fk2, i0, i2, i1]
}
```

Liste 23. – Représentation après optimisation par Pluto, avec ajout de dimensions de tuilage (`fk0`, `fk1`, `fk2`) dans le domaine.

```
Domaine final :                                                      ISL
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[i0, i1, i2] : i0 < p_1 and 0 <= i1 < p_0 and 0 <= i2 <= i0
}

Ordonnancement final :
[p_0, p_1, p_2, p_3, p_4, p_5] -> {
  Stmt[i0, i1, i2] -> [1, o1, o2, o3, i0, i2, i1] :
    -31 + i0 <= 32o1 <= i0 and
    -31 + i2 <= 32o2 <= i2 and
    -31 + i1 <= 32o3 <= i1
}
```

Liste 24. – Ordonnancement converti pour Polly : le domaine initial est restauré et les contraintes de tuilage sont déléguées à la relation d'ordonnancement.

6.2. Intégration de PPCG et réhabilitation de Polly-ACC

L'existence d'un support permettant la génération de code pour accélérateurs matériels GPU constitue un prérequis incontournable de nos travaux, initialement absent de notre flot de compilation. En effet, une divergence fondamentale subsistait entre les modèles

d'exécution de Kokkos et les capacités de Polly en matière d'architectures hétérogènes. Tandis que l'abstraction de Kokkos s'appuie structurellement sur une compilation CPU/GPU visant la portabilité des performances, Polly s'est historiquement restreint à l'optimisation pour architectures multicœurs et multiprocesseurs CPU. Or, un ordonnancement polyédrique taillé pour un processeur classique s'avère profondément inadapté au modèle d'exécution massivement parallèle d'un GPU. Ce dernier requiert une projection spatiale très contrainte des itérations sur une hiérarchie matérielle stricte (grilles, blocs), doublée d'une gestion explicite de la hiérarchie mémoire (mémoire globale, mémoire partagée, transferts d'interface).

Afin de combler cette lacune architecturale sans avoir à concevoir un moteur de génération de code GPU polyédrique, notre approche s'est portée sur la réhabilitation du projet **Polly-ACC** [GH16]. Cet outil exploitait historiquement PPCG, un compilateur source-à-source polyédrique dédié à l'optimisation et à la génération de code pour accélérateurs. La délégation de cette étape à PPCG, en substitution de la génération de code intrinsèque au backend Kokkos, offre des perspectives d'optimisations supérieures. En effet, la mécanique de Kokkos repose essentiellement sur un abaissement direct de l'expression algorithmique écrite par l'utilisateur vers le modèle de programmation cible (CUDA ou HIP), sans possibilité d'optimisation de la structure de contrôle (pas de fusion de boucles ou d'échanges complexes). À l'inverse, PPCG modélise le noyau de calcul et dérive algorithmiquement un espace de transformations complexes. Il automatise notamment l'exploitation de la mémoire partagée et la privatisation de registres, des mécanismes indispensables pour masquer la latence d'accès mémoire et saturer la bande passante sur GPU.

Cependant, le développement de Polly-ACC ayant été abandonné, le projet souffrait d'un important retard de développement. En effet, l'évolution rapide et continue des interfaces de programmation de LLVM et d'ISL avait rendu son code source incompatible avec les versions modernes du compilateur.

Dans le cadre de cette thèse, un travail d'ingénierie conséquent a été mené pour moderniser et réintégrer l'architecture conceptuelle de Polly-ACC au sein d'une version récente de LLVM. L'un des principaux défis d'intégration inhérents à PPCG réside dans sa conception originelle. PPCG ayant été pensé comme un outil opérant exclusivement sur du code source

C, ses structures de représentation interne faites pour gérer du code source ne prévoient nativement aucun mécanisme pour sauvegarder la sémantique et les métadonnées requises par un IR bas niveau comme celui de LLVM.

Pour surmonter ces obstacles structurels, une démarche minutieuse de rétro-ingénierie a été entreprise : identifier, isoler puis transposer les modifications effectuées à l'époque dans le PPCG historique de Polly-ACC vers la base de code maintenue de PPCG. L'objectif est de rétablir une chaîne de compilation hétérogène complète, structurée selon trois axes principaux :

- **Adaptation de PPCG** : Enrichissement des structures de données de PPCG afin d'assurer la traçabilité des instructions de l'IR LLVM tout au long du processus, et mise en conformité avec les API récentes d'ISL.
- **Interface de communication Polly-PPCG** : Rétablissement des mécanismes permettant à Polly de déléguer l'analyse des SCoPs à PPCG. Cette interface permet à PPCG de produire un ordonnancement GPU, d'établir une stratégie de placement mémoire (promotion en mémoire partagée, privatisation) et de formuler la projection des itérations sur la topologie d'exécution du GPU (grilles et blocs).
- **Génération de code hybride** : Restauration des passes LLVM responsables de la synthèse du code final. Cette étape englobe à la fois le code exécuté sur l'hôte (orchestration des transferts mémoire et appels aux noyaux de calcul via l'API CUDA) et la génération du code PTX des noyaux spécifiques au périphérique matériel.

Dans l'architecture de notre pipeline, lorsqu'un noyau de calcul est identifié (via les annotations du backend Kokkos) comme ciblant une architecture matérielle GPU, le pipeline classique de Polly est dynamiquement reconfiguré. Cette bifurcation intervient lors des phases de transformation et de synthèse de code. Les passes classiques s'appuyant sur l'ordonnanceur natif ISL et le générateur de code CPU sont désactivées. Le code est redirigé vers PPCG pour le calcul de l'ordonnancement et la génération du code GPU.

Cette implémentation logicielle comble l'écart structurel entre le compilateur et le modèle de programmation, dotant à nouveau Polly de la capacité de générer du code pour les architectures hétérogènes. Le rétablissement de ce pipeline GPU polyédrique dans Polly est indispensable pour permettre l'optimisation performante de toutes les applications écrites en Kokkos compatibles avec le modèle polyédrique.

L'intégration de Pluto et la réhabilitation du flot PPCG finalisent ainsi notre chaîne de compilation polyédrique adaptée à Kokkos. Le chapitre suivant (chapitre 7) est ainsi consacré à la présentation des résultats expérimentaux sur architectures CPU et GPU, en quantifiant les accélérations obtenues face aux implémentations de référence de Kokkos.

Chapitre 7

Évaluations

Ce chapitre quantifie et analyse l'impact des contributions présentées. Le but principal est de démontrer que la restauration de la visibilité sémantique et structurelle au niveau du compilateur permet de débloquer des optimisations de boucles agressives grâce au modèle polyédrique. À travers une évaluation sur la suite de benchmarks PolyBench [Poul1], nous mettons à l'épreuve notre chaîne de compilation sur des architectures hétérogènes (CPU multicœurs et accélérateurs GPU). Nous détaillons les accélérations obtenues par rapport à une implémentation Kokkos idiomatique et étudions l'impact avec différentes architectures et ordonnanceurs.

7.1. Protocole expérimental

Version de référence. Les performances sont mesurées par rapport à une implémentation idiomatique (parallèle lorsque cela est possible) des noyaux de la suite de benchmarks PolyBench en C++ avec Kokkos, telle que l'écrirait un utilisateur standard de la bibliothèque. Les nids de boucles sont exprimés à l'aide de `parallel_for` et utilisent les structures Kokkos comme les vues `Kokkos::View`, sans optimisation spécifique, comme par exemple la fusion ou la fission de boucles, la réécriture du calcul pour exposer davantage de parallélisme ou l'usage explicite de la mémoire rapide (`Kokkos::ScratchSpace`). Cette version est compilée avec la même chaîne de compilation Clang, mais en activant ou en désactivant l'utilisation de Polly, sur les backends standards de Kokkos : OpenMP pour les CPU et CUDA pour les GPU.

Version optimisée. La version évaluée exprime les mêmes calculs, mais sous la forme attendue par notre chaîne de compilation. Chaque noyau est écrit en entier dans un

`parallel_for` (ou `parallel_for` variadique, voir section 5.2.1) annoté par l'option `usePolyOpt`, y compris ses boucles externes non parallèles. La politique d'exécution ne décrit plus le parallélisme à exploiter mais elle délimite l'espace d'itération du noyau et désigne le backend ciblé. Polly analyse l'ensemble comme un seul noyau puis décide lui-même des boucles à paralléliser. Le même code sert ainsi pour les deux cibles (CPU, GPU), seul change l'espace d'exécution de la politique.

Plateformes. Les expérimentations sont menées sur deux machines, décrites dans le tableau 3 : une machine bi-socket équipée de deux processeurs AMD EPYC 7502, et une machine combinant un processeur Intel Core Ultra 7 265 et une carte graphique NVIDIA RTX 5090.

	AMD EPYC 7502	**Intel Core Ultra 7 265**
Micro-architecture	Zen 2	Arrow Lake
Fréquence de base	2.5 GHz	2.4 GHz (cœurs P)
Fréquence maximale	3.35 GHz	5.3 GHz
Cœurs	64 (2 sockets × 32)	20 (8 P + 12 E)
Cache L1 par cœur (données + instructions)	32 Ko + 32 Ko	48 Ko + 64 Ko (P), 32 Ko + 64 Ko (E)
Cache L2	512 Ko par cœur	3 Mo par cœur P, 4 Mo par groupe de 4 cœurs E
Cache L3	8×16 Mo par socket, un par groupe de 4 cœurs	30 Mo, partagé
Mémoire	756 Go DDR4	32 Go DDR5
Nœuds NUMA	2, un par socket	1
Unités vectorielles	256 bits	256 bits
Noyau Linux	5.15.0-142-generic	6.17.0-1030-oem
GPU	–	NVIDIA RTX 5090 (Blackwell)
SM / cœurs CUDA	–	170 / 21 760
Mémoire GPU	–	32 Go GDDR7
Mémoire partagée par SM	–	128 Ko
Registres par SM	–	256 Ko
Pilote NVIDIA	–	595.99

Tableau 3. – Caractéristiques des deux plateformes d'évaluation.

Chaîne de compilation. Les deux machines utilisent une version de développement de LLVM/Clang 23.0.0 intégrant nos contributions, la version 4.6.99 de Kokkos, modifiée comme décrit

dans les chapitres précédents, et PPCG 0.09.3 pour la génération de code GPU. Tous les codes sont compilés en mode `Release` (`CMAKE_BUILD_TYPE=Release`).

Mesures. Chaque temps est la moyenne de 5 exécutions consécutives, dont la dispersion relative reste inférieure à 5 %. Sur CPU, chaque thread OpenMP est fixé sur un cœur et les threads sont répartis uniformément sur l'ensemble des cœurs, et donc sur les deux sockets de la machine AMD (`OMP_PROC_BIND=spread`, `OMP_PLACES=threads`). Chaque exécution est validée en comparant les résultats des calculs de la version optimisée élément par élément à ceux de la version de référence, avec une tolérance absolue de 10^{-8}.

Noyaux évalués. L'évaluation porte sur 27 des 30 noyaux de la suite PolyBench. Les trois noyaux écartés, `correlation`, `nussinov` et `floyd-warshall`, sortent du cadre du modèle polyédrique statique, car leur flot de contrôle dépend de valeurs calculées pendant l'exécution par le biais d'opérations ternaires, ce qui produit dans l'IR LLVM des accès non affines que Polly ne peut pas représenter. Tous les calculs sont effectués en double précision (`double`), à l'exception de `deriche` qui, comme dans la version originale de PolyBench, utilise la simple précision (`float`). Les jeux de données ont été agrandis afin d'obtenir des temps d'exécution significatifs. Le tableau 4 donne, pour chaque noyau de la suite PolyBench, la taille utilisée.

Catégorie	**Noyau**	**Paramètres du problème**
Datamining	`covariance`	$M = 4700, N = 5000$
BLAS	`gemm`	$NI = 5000, NJ = 5500, NK = 6000$
	`gemver`	$N = 20000$
	`gesummv`	$N = 35800$
	`symm`	$M = 1000, N = 1200$
	`syr2k`	$M = 5000, N = 5200$
	`syrk`	$M = 5000, N = 5200$
	`trmm`	$M = 2000, N = 2600$
Kernels	`2mm`	$NI = 5024, NJ = 5048, NK = 5072, NL = 5096$
	`3mm`	$NI = 5024, NJ = 5048, NK = 5072, NL = 5096, NM = 5120$
	`atax`	$M = 10024, N = 10048$
	`bicg`	$M = 22400, N = 24800$
	`doitgen`	$NQ = 656, NR = 664, NP = 672$
	`mvt`	$N = 20028$
Solvers	`cholesky`	$N = 2048$

Catégorie	Noyau	Paramètres du problème
	`durbin`	$N = 15048$
	`gramschmidt`	$M = 2000, N = 2600$
	`lu`	$N = 2048$
	`ludcmp`	$N = 2048$
	`trisolv`	$N = 35048$
Medley	`deriche`	$W = 1024, H = 880$
Stencils	`adi`	$N = 1000, T = 250$
	`fdtd-2d`	$NX = 900, NY = 1100, T = 250$
	`heat-3d`	$N = 256, T = 250$
	`jacobi-1d`	$N = 50000, T = 10000$
	`jacobi-2d`	$N = 10000, T = 250$
	`seidel-2d`	$N = 10000, T = 250$

Tableau 4. – Catégories et tailles des 27 noyaux PolyBench évalués.

7.2. Évaluation sur CPU

Cette section évalue l'efficacité de notre chaîne de compilation sur processeurs multicœurs. Pour chacun des 27 noyaux retenus, la version de référence Kokkos est comparée à sa version optimisée (section 7.1). Les deux ordonnanceurs polyédriques sont confrontés : l'ordonnanceur natif de Polly, reposant sur ISL, et l'ordonnanceur Pluto (section 6.1). Dans les deux cas, le tuilage est appliqué avec une taille de tuile fixe de 32 sur chaque dimension, et Polly parallélise le code généré au moyen du support d'exécution OpenMP. Notons que l'utilisateur peut choisir d'autres tailles de tuiles en utilisant le flag de compilation `-mllvm -polly-tile-sizes` ou `-mllvm -polly-default-tile-size` (section 8.3).

7.2.1. Étude des résultats CPU

La figure 18 et la figure 19 présentent les accélérations obtenues par rapport à la version de référence sur chacune des deux machines. Le tableau 5 en extrait des statistiques par famille de noyaux et globalement.

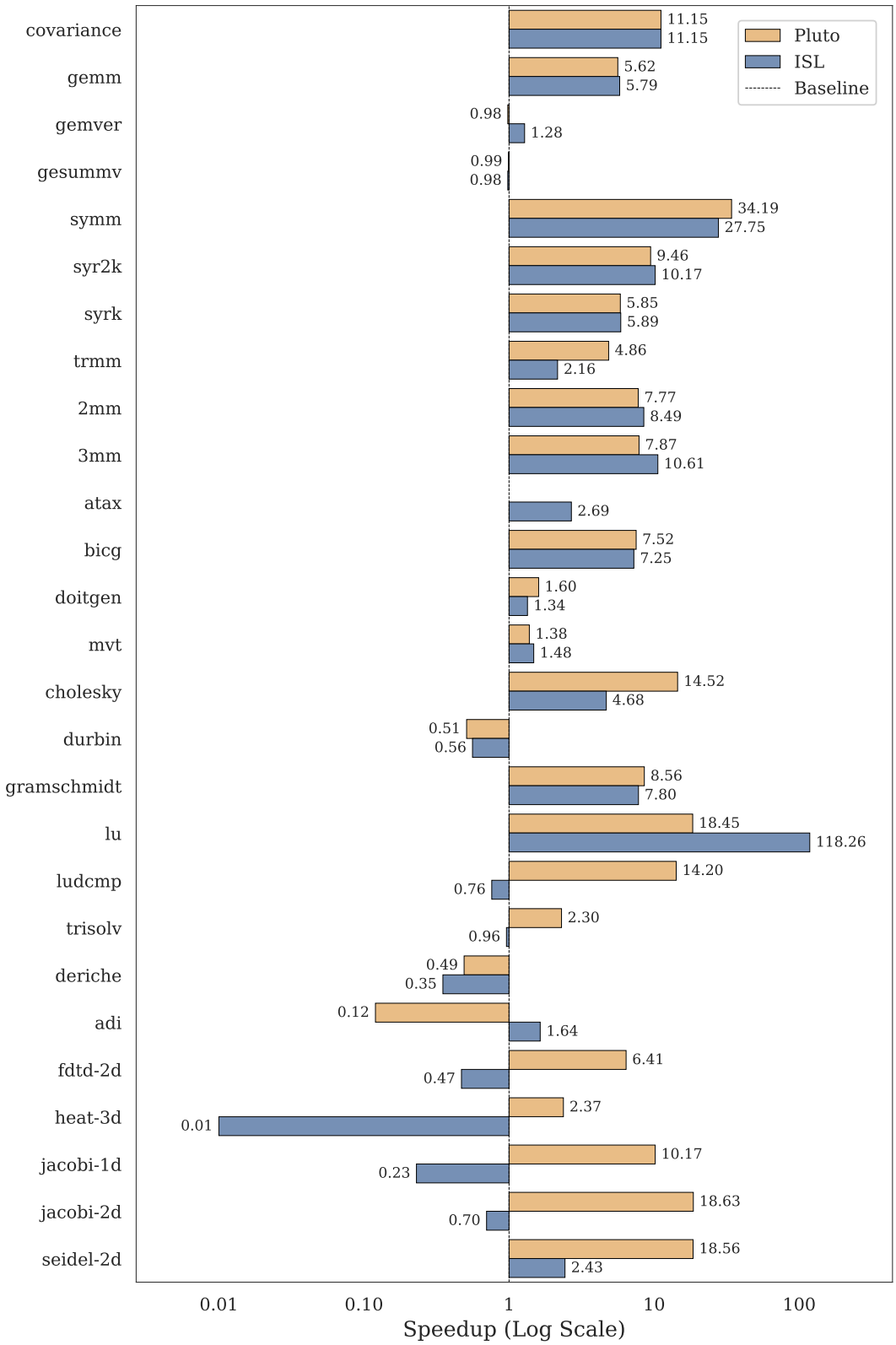


Figure 18. – Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend OpenMP) avec les ordonnanceurs ISL et Pluto sur processeur Dual AMD EPYC 7502 (2×32 cœurs, 64 threads sans hyperthreading, taille de tuile : 32).

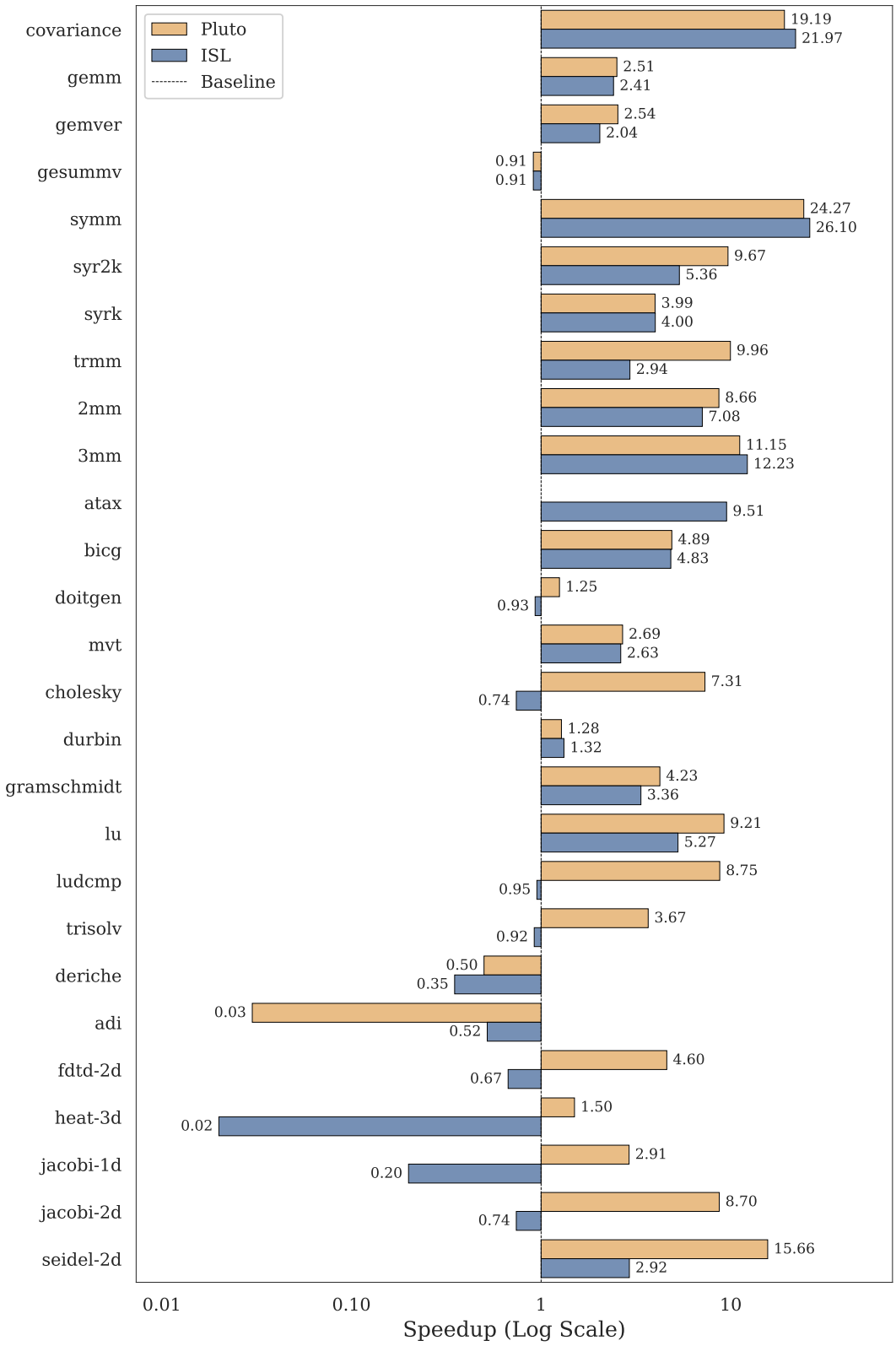


Figure 19. – Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend OpenMP) avec les ordonnanceurs ISL et Pluto sur processeur Intel Core Ultra 7 265 (20 threads sans hyperthreading, taille de tuile : 32).

Catégorie	Noyaux	AMD EPYC 7502		Intel Core Ultra 7 265	
		Pluto	ISL	Pluto	ISL
Datamining	1	11.15× (1/1)	11.15× (1/1)	19.19× (1/1)	21.97× (1/1)
BLAS	7	4.69× (5/7)	4.27× (6/7)	4.74× (6/7)	3.57× (6/7)
Kernels	6	3.99× (5/5)	3.89× (6/6)	4.37× (5/5)	4.62× (5/6)
Solvers	6	5.80× (5/6)	3.48× (3/6)	4.77× (6/6)	1.57× (3/6)
Medley	1	0.49× (0/1)	0.35× (0/1)	0.50× (0/1)	0.35× (0/1)
Stencils	6	4.31× (5/6)	0.38× (2/6)	2.08× (5/6)	0.38× (1/6)
Ensemble	27	4.44× (21/26)	2.21× (18/27)	3.74× (23/26)	1.88× (16/27)

Tableau 5. – Moyennes géométriques des accélérations sur CPU par catégorie de benchmark et par ordonnanceur. Entre parenthèses figure le nombre de noyaux accélérés sur le nombre de noyaux évalués.

Le premier constat est que les transformations permises par nos contributions profitent à la grande majorité des noyaux. Sur chacune des deux machines, 24 des 27 noyaux sont accélérés par au moins l'un des deux ordonnanceurs. En retenant pour chaque noyau le meilleur des deux ordonnanceurs, la moyenne géométrique des accélérations atteint 5.32× sur la machine AMD et 4.36× sur la machine Intel. L'accélération maximale, 118.26×, est obtenue sur `lu` avec l'ordonnanceur ISL sur la machine AMD. Ces résultats confirment qu'une fois la structure des boucles rendue visible à Polly, les transformations polyédriques exploitent efficacement la localité et le parallélisme laissés inexploités par une écriture Kokkos idiomatique des noyaux.

Ces gains dépendent avant tout de la manière dont la version de référence est écrite, et proviennent de deux sources : la localité, lorsque la référence est déjà parallèle mais réutilise mal ses données, et le parallélisme, lorsqu'elle exécute une partie du calcul séquentiellement. Le choix de l'ordonnanceur compte surtout dans ce second cas, car seul Pluto expose un parallélisme en front d'onde, ce qui explique son avantage moyen (4.44× contre 2.21× sur AMD, 3.74× contre 1.88× sur Intel), concentré sur les *stencils* et les *solvers*. Les sections suivantes examinent ces deux sources de gain, puis les noyaux pour lesquels le code généré reste moins performant que la référence.

Sur les 54 variantes compilées (27 noyaux × 2 ordonnanceurs), 53 produisent des résultats conformes à la référence. Seul le noyau `atax` avec l'ordonnanceur Pluto produit un résultat erroné, en raison d'un ordonnancement invalide. L'erreur provient de l'outil Pluto lui-même.

En reproduisant le noyau sans la première boucle (remplacée par un `memset` par LLVM lors de la compilation), on obtient le même résultat incorrect.

7.2.2. Gains de localité

Plusieurs noyaux bénéficient d'accélérations importantes avec les deux ordonnanceurs et sur les deux machines. Le cas le plus marquant est `symm`, qui atteint 34.19× (Pluto) et 27.75× (ISL) sur la machine AMD, et 24.27× et 26.10× sur la machine Intel. Les produits matriciels `2mm` et `3mm` (de 7.08× à 12.23×), `covariance` (de 11.15× à 21.97×), `syr2k`, `gramschmidt` et `bicg` présentent un profil similaire. Ces noyaux partagent une forte réutilisation des données, avec un volume de calcul en $O(n^3)$ pour des données en $O(n^2)$, que la version de référence n'exploite pas. Dans ces versions de référence, la boucle la plus interne, écrite dans le corps de la lambda, parcourt une colonne d'au moins un tableau stocké par lignes, avec un grand pas mémoire entre deux accès successifs. Or Kokkos ne tuile que les dimensions d'une `MDRangePolicy`, jamais les boucles écrites dans la lambda. La plupart de ces versions reposent sur une `RangePolicy` et ne sont pas tuilées du tout, et seules celles de `2mm` et `3mm` découpent l'espace `(i, j)` en tuiles de 32×32, sans toucher à la boucle de réduction. Le tuilage polyédrique porte au contraire sur l'ensemble du nid, boucle de réduction comprise, comme l'illustre `symm` dans le paragraphe suivant.

Le cas de `symm` illustre la combinaison de transformations à l'origine de ces gains. Pour chaque élément `(i, j)`, ce noyau accumule une somme partielle `tmp(i, j)` tout en mettant à jour les éléments `C(k, j)` des lignes précédentes (liste 25). Ces mises à jour relient entre elles toutes les itérations de `i`, si bien que seule la boucle `j` est parallèle dans le nid d'origine. ISL sépare le corps du nid en plusieurs nids de boucles distincts, chacun parallèle et tuilé par blocs de 32 (liste 26). L'accumulation de `tmp` et la mise à jour finale de `C(i, j)` sont parallélisées sur `i`, et les mises à jour de `C(k, j)` sur `j`.

L'ampleur du gain tient à la version de référence. Celle-ci exécute la boucle `i` séquentiellement sur l'hôte et lance, à chacune des itérations, un `parallel_for` sur `j`.

```cpp
// Version de référence
const auto policy = Kokkos::RangePolicy<Kokkos::OpenMP>(0, n);
for (INT_TYPE i = 0; i < m; i++) {
  Kokkos::parallel_for(
      policy, KOKKOS_LAMBDA(const INT_TYPE j) {
        tmp(i, j) = 0;
        for (INT_TYPE k = 0; k < i; k++) {
          C(k, j) += alpha * B(i, j) * A(i, k);
          tmp(i, j) += B(k, j) * A(i, k);
        }
        C(i, j) = beta * C(i, j) + alpha * B(i, j) * A(i, i) + alpha * tmp(i, j);
      });
}

// Version analysée par Polly
Kokkos::parallel_for<Kokkos::usePolyOpt>(
    policy, KOKKOS_LAMBDA(const INT_TYPE i, const INT_TYPE j) {
      tmp(i, j) = 0;
      for (INT_TYPE k = 0; k < i; k++) {
        C(k, j) += alpha * B(i, j) * A(i, k);
        tmp(i, j) += B(k, j) * A(i, k);
      }
      C(i, j) = beta * C(i, j) + alpha * B(i, j) * A(i, i) + alpha * tmp(i, j);
    });
```

Liste 25. – Implémentations Kokkos du noyau `symm` : version de référence, parallélisée sur `j` seul, et version analysée par Polly (`usePolyOpt`).

```cpp
// Accumulation de tmp : parallèle sur i
parallel for (tuiles de i)
  for (tuiles de j)
    for (tuiles de k)
      tmp(i, j) += B(k, j) * A(i, k);
// Mise à jour finale de C(i, j) : parallèle sur i
parallel for (tuiles de i)
  for (tuiles de j)
    C(i, j) = beta * C(i, j) + alpha * B(i, j) * A(i, i) + alpha * tmp(i, j);
// Mises à jour de C(k, j) : parallèle sur j
parallel for (tuiles de j)
  for (tuiles de k)
    for (tuiles de i)
      C(k, j) += alpha * B(i, j) * A(i, k);
```

Liste 26. – Structure simplifiée du code généré par Polly avec l'ordonnanceur ISL pour `symm` : les trois nids principaux, parcourus par tuiles de 32. Les boucles internes aux tuiles sont omises.

7.2.3. Gains de parallélisme

Sur plusieurs noyaux, le gain provient du parallélisme que la version de référence n'exploite pas : `seidel-2d` (18.56× et 15.66× avec Pluto), `cholesky` (14.52× et 7.31×), `ludcmp` (14.20× et 8.75×) et `trisolv` (2.30× et 3.67×), ainsi que `lu`, discuté dans la section 7.2.5. Ces noyaux présentent des dépendances portées par toutes les dimensions du nid de boucles, de sorte qu'aucune boucle n'est directement parallélisable sans transformation préalable. Leur version de référence exécute donc tout ou partie du calcul séquentiellement. En appliquant des transformations de torsion de boucle, Pluto rend le nid tuilable et expose un parallélisme en front d'onde entre les tuiles.

Le code généré par Pluto pour `seidel-2d` illustre ce mécanisme (liste 27). Chaque instance d'instruction (t, i, j) est ordonnancée en $(t, t + i, 2t + i + j)$. Cette torsion rend les trois dimensions tuilables. Les tuiles sont ensuite parcourues par fronts d'onde. Les fronts sont traités l'un après l'autre, et les tuiles d'un même front, indépendantes, s'exécutent en parallèle. Pluto applique le même mécanisme à `ludcmp`.

L'écriture des versions de référence explique l'ampleur de ces gains. La version de `seidel-2d` est entièrement séquentielle, la mise à jour en place du tableau interdisant toute parallélisation directe. La version de `cholesky` est également entièrement séquentielle. Les accélérations de Pluto sur ces deux noyaux (18.56× et 15.66× pour `seidel-2d`, 14.52× et 7.31 × pour `cholesky`) mesurent donc à la fois le parallélisme par front d'onde et le gain de localité apporté par le tuilage des boucles. La version de référence de `ludcmp` n'est que partiellement parallèle. La boucle externe de la factorisation s'exécute séquentiellement, et à chacune de ses itérations, seule la partie supérieure de la ligne est parallélisée, la partie inférieure restant séquentielle. En transformant l'ensemble de la factorisation par fronts d'onde, Pluto lève cette limitation de parallélisation.

```C++
// Fronts d'onde de tuiles : séquentiels
#pragma minimal dependence distance: 1
for (int c0 = 0; c0 <= floord(N + 2 * T - 5, 32); c0 += 1)
  // Tuiles d'un même front : parallèles
  #pragma omp parallel for
  for (int c1 = ...; c1 <= ...; c1 += 1)
    #pragma minimal dependence distance: 1
    for (int c2 = ...; c2 <= ...; c2 += 1)
      // Points de la tuile
      for (int c3 = ...; c3 <= ...; c3 += 1)          // t
        for (int c4 = ...; c4 <= ...; c4 += 1)        // t + i
          for (int c5 = ...; c5 <= ...; c5 += 1)      // 2t + i + j
            Stmt2(c3, -c3 + c4, -c3 - c4 + c5);
```

Liste 27. – Structure de l'AST généré par Polly avec l'ordonnanceur Pluto pour `seidel-2d` sur CPU.

ISL n'applique pas cette transformation en front d'onde. Sur `seidel-2d`, il produit un code tuilé mais toujours séquentiel, qui ne gagne que par la localité des données (2.43× et 2.92×). Sur les autres noyaux de ce groupe, il ne fait au mieux qu'égaler la référence, comme le détaille la section suivante.

7.2.4. Régressions

À l'inverse, plusieurs noyaux ne gagnent rien ou régressent, de manière identique sur les deux machines. Dans la plupart des cas, le code généré exploite moins de parallélisme que la version de référence.

Les noyaux sans réutilisation de données. `gesummv` reste neutre quels que soient l'ordonnanceur et la machine (entre 0.91× et 0.99×), et `doitgen` comme `mvt` n'obtiennent que des gains modestes. Ces noyaux de type matrice-vecteur lisent chaque élément de matrice une seule fois. Leur performance est bornée par la bande passante mémoire et le tuilage n'a aucune localité à exploiter.

Les fausses dépendances dues aux scalaires. Le noyau `deriche` régresse sur les deux machines avec les deux ordonnanceurs (de 0.35× à 0.50×). Ce noyau parcourt une image ligne par ligne, puis colonne par colonne. Le long d'une ligne, chaque pixel dépend des pixels déjà traités, dont les valeurs sont transmises d'une itération à l'autre par des variables

scalaires remises à zéro au début de chaque ligne. Les lignes, comme les colonnes, restent donc indépendantes les unes des autres, et la version de référence les traite en parallèle.

Le modèle polyédrique ne voit pas cette indépendance. Polly représente chaque scalaire par un emplacement mémoire unique, partagé par toutes les lignes. Du point de vue du modèle, chaque ligne réécrit les valeurs laissées par la précédente, ce qui crée de fausses dépendances entre les lignes. Les deux ordonnanceurs exécutent alors séquentiellement les quatre passes récursives (liste 28), et le parallélisme se réduit pour l'essentiel aux deux passes qui combinent leurs résultats, dépourvues de scalaires. Transformer ces scalaires en tableaux indexés par la ligne, ou les privatiser, suffirait à lever ces fausses dépendances.

```C++
// Passe récursive sur les lignes : séquentielle
#pragma minimal dependence distance: 1
for (int c0 = 0; c0 < w; c0 += 1) {      // lignes
  Stmt0(c0);                             // remise à zéro des scalaires
  #pragma minimal dependence distance: 1
  for (int c1 = 0; c1 < h; c1 += 1)      // pixels de la ligne
    Stmt1(c0, c1);
}
// Passe de combinaison : tuilée et parallèle
#pragma omp parallel for
for (int c0 = 0; c0 <= (w - 1) / 32; c0 += 1)
  for (int c1 = 0; c1 <= (h - 1) / 32; c1 += 1)
    for (int c2 = ...; c2 <= ...; c2 += 1)
      #pragma simd
      for (int c3 = ...; c3 <= ...; c3 += 1)
        Stmt8(32 * c0 + c2, 32 * c1 + c3);
```

Liste 28. – Extrait de l'AST généré par Polly avec l'ordonnanceur ISL pour `deriche` sur CPU : une passe récursive et une passe de combinaison.

Le parallélisme limité à la boucle la plus interne. Avec ISL, le code généré s'effondre sur `heat-3d` (0.01× et 0.02×) et dégrade `jacobi-1d` (0.23× et 0.20×), `jacobi-2d` (0.70× et 0.74×) et `fdtd-2d` (0.47× et 0.67×). Sur ces noyaux, chaque pas de temps dépend du précédent, ce qui interdit de paralléliser directement les boucles externes. ISL parvient pourtant à tuiler le nid, y compris la dimension temporelle, grâce à une torsion des boucles (liste 29). Mais il n'exécute pas en parallèle les tuiles indépendantes. La seule boucle qu'il identifie comme parallèle est la plus interne mais Polly ne génère pas de parallélisation de threads pour une boucle interne. Le code généré s'exécute sur un seul cœur, alors que la version de référence parallélise chaque pas de temps. Le même phénomène se produit sur `jacobi-1d`, `jacobi-2d` et `fdtd-2d`. Sur `jacobi-1d`,

cette explication n'est toutefois que partielle, car l'essentiel de la régression vient d'une vectorisation défavorable du code généré, détaillée dans la section 7.5. Sans cette vectorisation, le code séquentiel produit par ISL atteint 0.99× de la version de référence parallèle sur la machine Intel.

```cpp
// Boucles de tuiles (t, i, j, k) : séquentielles
#pragma minimal dependence distance: 1
for (int c0 = 0; c0 <= (T - 1) / 32; c0 += 1)
  #pragma minimal dependence distance: 1
  for (int c1 = ...; c1 <= ...; c1 += 1)
    #pragma minimal dependence distance: 1
    for (int c2 = ...; c2 <= ...; c2 += 1)
      #pragma minimal dependence distance: 1
      for (int c3 = ...; c3 <= ...; c3 += 1)
        // Boucles de points : seule la plus interne est parallèle
        #pragma minimal dependence distance: 1
        for (int c4 = ...; c4 <= ...; c4 += 1)
          #pragma minimal dependence distance: 1
          for (int c5 = ...; c5 <= ...; c5 += 1)
            #pragma minimal dependence distance: 1
            for (int c6 = ...; c6 <= ...; c6 += 1)
              #pragma simd
              #pragma known-parallel
              for (int c7 = ...; c7 <= ...; c7 += 1) {
                Stmt3(...);  // calcul de B à partir de A
                Stmt9(...);  // calcul de A à partir de B
              }
```

Liste 29. – Structure de l'AST généré par Polly avec l'ordonnanceur ISL pour `heat-3d` sur CPU. La boucle la plus interne est marquée comme pouvant être parallélisée avec l'annotation `#pragma known-parallel`.

Pluto, au contraire, exécute en parallèle les tuiles indépendantes parcourues par fronts d'onde, et accélère ces mêmes noyaux, par exemple `jacobi-2d` (18.63× et 8.70×).

ISL échoue également sur les solveurs triangulaires `ludcmp` (0.76× et 0.95×) et `trisolv` (0.96× et 0.92×). Sur `trisolv`, il échange les deux boucles du nid, mais la seule boucle parallèle obtenue est à nouveau la plus interne. Comme pour les stencils, le code généré est entièrement séquentiel. La version de référence n'est pas efficace, elle lance une réduction parallèle pour chacune des lignes de la matrice, dont le coût de lancement pèse lourdement face au peu de travail confié à chaque thread. Ces deux inefficacités se compensent, d'où des performances proches de la référence.

Sur `ludcmp`, ISL conserve la structure d'origine du nid, sans tuilage, et reproduit la parallélisation de la version de référence, là où Pluto parallélise l'ensemble de la factorisation par fronts d'onde. Faute de gain de localité, le code généré ne peut au mieux qu'égaler la référence et les surcoûts des calculs d'indices polyédriques créent un léger ralentissement.

Le noyau `durbin` régresse sur la machine AMD ($0.51\times$ et $0.56\times$) mais accélère légèrement sur la machine Intel ($1.28\times$ et $1.32\times$). Ce noyau comporte une boucle externe intrinsèquement séquentielle, dont chaque itération ne contient que de petites boucles. La version de référence parallélise ces petites boucles, au prix de trois synchronisations par itération, tandis que le code généré par ISL comme par Pluto est entièrement séquentiel. Le résultat dépend donc du matériel. Le code séquentiel profite de la fréquence élevée d'un cœur Intel, mais ne rivalise pas, sur la machine AMD et ses cœurs plus lents, avec une version parallèle répartie sur 64 cœurs.

Une fusion défavorable. Avec Pluto, le code généré régresse fortement sur `adi` sur les deux machines ($0.12\times$ et $0.03\times$), alors qu'ISL l'accélère sur la machine AMD ($1.64\times$). Chaque pas de temps de ce noyau enchaîne deux balayages de la grille, l'un selon les colonnes et l'autre selon les lignes. Ces colonnes et ces lignes étant indépendantes les unes des autres, la version de référence parallélise chacune des quatre phases, et ISL préserve ce parallélisme.

Pluto fusionne deux de ces phases, comme l'une parcourt la grille par colonnes et l'autre par lignes, le nid fusionné est parcouru diagonale par diagonale (liste 30). Seule une partie du calcul reste alors parallèle, le long de chaque diagonale et au prix d'une synchronisation à chacune d'elles. Ces deux phases perdent ainsi l'essentiel de la parallélisation de la version de référence.

```C++
// Remontée en colonnes et descente en lignes, fusionnées
#pragma minimal dependence distance: 1
for (int c3 = 0; c3 < 2 * N - 5; c3 += 1) {  // anti-diagonales
  if (c3 < N - 2)
    #pragma minimal dependence distance: 1
    for (int c8 = 0; c8 < N - 2; c8 += 1)
      Stmt4(...);   // remontée en colonnes : séquentielle
  #pragma omp parallel for
  for (int c8 = ...; c8 <= ...; c8 += 1)
    #pragma simd
    for (int c9 = ...; c9 <= ...; c9 += 1)
      Stmt8b(...);  // descente en lignes, 1re partie : parallèle
  if (c3 < N - 2)
    #pragma minimal dependence distance: 1
    for (int c8 = 0; c8 < N - 2; c8 += 1)
      Stmt8(...);   // descente en lignes, 2e partie : séquentielle
}
```

Liste 30. – Extrait simplifié de l'AST généré par Polly avec l'ordonnanceur Pluto pour `adi` sur CPU : le nid fusionné, à l'intérieur de la boucle temporelle. Les bornes et les indices sont omis, et le paramètre `p_0` a été renommé `N`.

Ces régressions se reproduisent à l'identique sur les deux machines. Aucun des deux ordonnanceurs ne dispose d'un modèle de coût capable de détecter qu'une transformation réduit le parallélisme ou la localité par rapport au code d'origine.

7.2.5. Sensibilité au matériel

Au-delà de ces comportements stables, plusieurs noyaux présentent des écarts importants entre les deux machines, qui révèlent la sensibilité des transformations aux caractéristiques matérielles.

7.2.5.1. Le cas de la factorisation `lu`

Le noyau `lu` présente un écart important d'exécution sur la machine AMD : ISL atteint 118.26× et Pluto 18.45× contre 5.27× et 9.21× sur la machine Intel. Deux effets se combinent.

Un premier effet provient de la version de référence. Comme celle de `ludcmp`, elle n'est parallèle que pour moitié. La boucle externe s'exécute séquentiellement et à chacune de ses itérations seule la partie supérieure de la ligne est calculée en parallèle. Sur la machine AMD, un profilage avec `perf` montre que cette moitié séquentielle détermine à elle seule la durée de

l'exécution. Les 63 threads secondaires passent l'essentiel de leur temps à attendre le thread principal, la référence s'exécute donc pratiquement à la vitesse d'un seul cœur. Or ce cœur est nettement plus lent que celui de la machine Intel. La référence est donc nettement moins efficace sur la machine AMD que sur la machine Intel.

Le second effet tient à la manière dont chaque code généré exploite la machine. Le code ISL parallélise en une succession d'étapes propice au grand nombre de cœurs de la machine AMD. Sur la machine Intel, les étapes sont réparties en blocs égaux entre les cœurs P (performance) et E (efficacité énergétique). Les cœurs E, moins rapides, terminent leur bloc en dernier et déterminent ainsi la durée du calcul. Pluto parcourt au contraire le calcul par fronts d'onde. Il réutilise mieux les données, mais chaque front de charges inégales compte un nombre limité de tuiles qui ne suffisent pas à occuper les 64 cœurs de la machine AMD. Le code tire davantage parti de la fréquence élevée des cœurs Intel que du nombre de cœurs de la machine AMD.

7.2.5.2. Régressions liées au tuilage fixe

Pour `cholesky` et `adi`, ISL obtient 4.68× et 1.64× sur la machine AMD, mais régresse à 0.74× et 0.52× sur la machine Intel. L'accélération de Pluto sur `cholesky` est elle aussi divisée par deux (de 14.52× à 7.31×). Les deux ordonnanceurs utilisent une taille de tuile fixe de 32. Or le cache L2 de la machine Intel contient déjà une grande partie des données manipulées par la version de référence. Le tuilage n'apporte pas de gain de localité et son surcoût le rend contre-productif. Sur la machine AMD le tuilage reste bénéfique.

7.2.6. Bilan de l'évaluation sur CPU

Cette évaluation montre que la visibilité restaurée par nos contributions permet à Polly d'optimiser des codes Kokkos avec des gains en performance non négligeables, puisque 24 des 27 noyaux sont accélérés sur chaque machine. Ces gains ont deux origines : une amélioration de la localité mémoire et des transformations créant des boucles sans dépendance parallèles comme le front d'onde de Pluto.

Les régressions surviennent lorsque le code généré exploite moins de parallélisme que la référence (fausses dépendances dues aux scalaires, parallélisme limité à la boucle la plus interne ou fusion défavorable). S'y ajoute une sensibilité au matériel avec la taille de tuile fixe

et l'absence de modèle de coût prenant en compte la hiérarchie de caches provoquant des régressions sur certaines architectures.

La section suivante étend cette évaluation aux accélérateurs GPU, où la génération de code repose sur PPCG.

7.3. Évaluation sur GPU

Cette section évalue la génération de code GPU rendue possible par la réintégration de PPCG dans Polly (section 6.2). La version de référence suit la même démarche que sur CPU : une implémentation Kokkos idiomatique, compilée avec le backend CUDA de Kokkos, qui parallélise directement les boucles parallèles de l'algorithme d'origine. La version optimisée est celle décrite dans la section 7.1, dont la politique d'exécution désigne cette fois le backend CUDA.

Les mesures sont réalisées sur la carte NVIDIA RTX 5090 de la machine Intel (tableau 3) et incluent, dans les deux versions, les transferts de données entre l'hôte et le GPU. Kokkos choisit dynamiquement la taille des blocs de threads de la version de référence. PPCG découpe au contraire chaque noyau en tuiles de 32 itérations fixes par dimension, confiées à des blocs de 32 threads pour un noyau à une dimension et de 4×32 threads pour un noyau à deux dimensions. La promotion des données en mémoire partagée (limitée à 16 Ko par bloc) et la privatisation en registres des données sont activées.

7.3.1. Étude des résultats GPU

Sur les 27 noyaux évalués, 24 sont compilés et produisent des résultats corrects avec PPCG. Les trois noyaux restants, `adi`, `fdtd-2d` et `heat-3d`, n'ont pas pu être compilés, car l'ordonnancement calculé par PPCG ne termine pas en un temps raisonnable, en raison du nombre considérable de contraintes générées pour ces nids de boucles.

La figure 20 présente les accélérations obtenues sur les 24 noyaux restants, et le tableau 6 en résume les moyennes géométriques par catégorie.

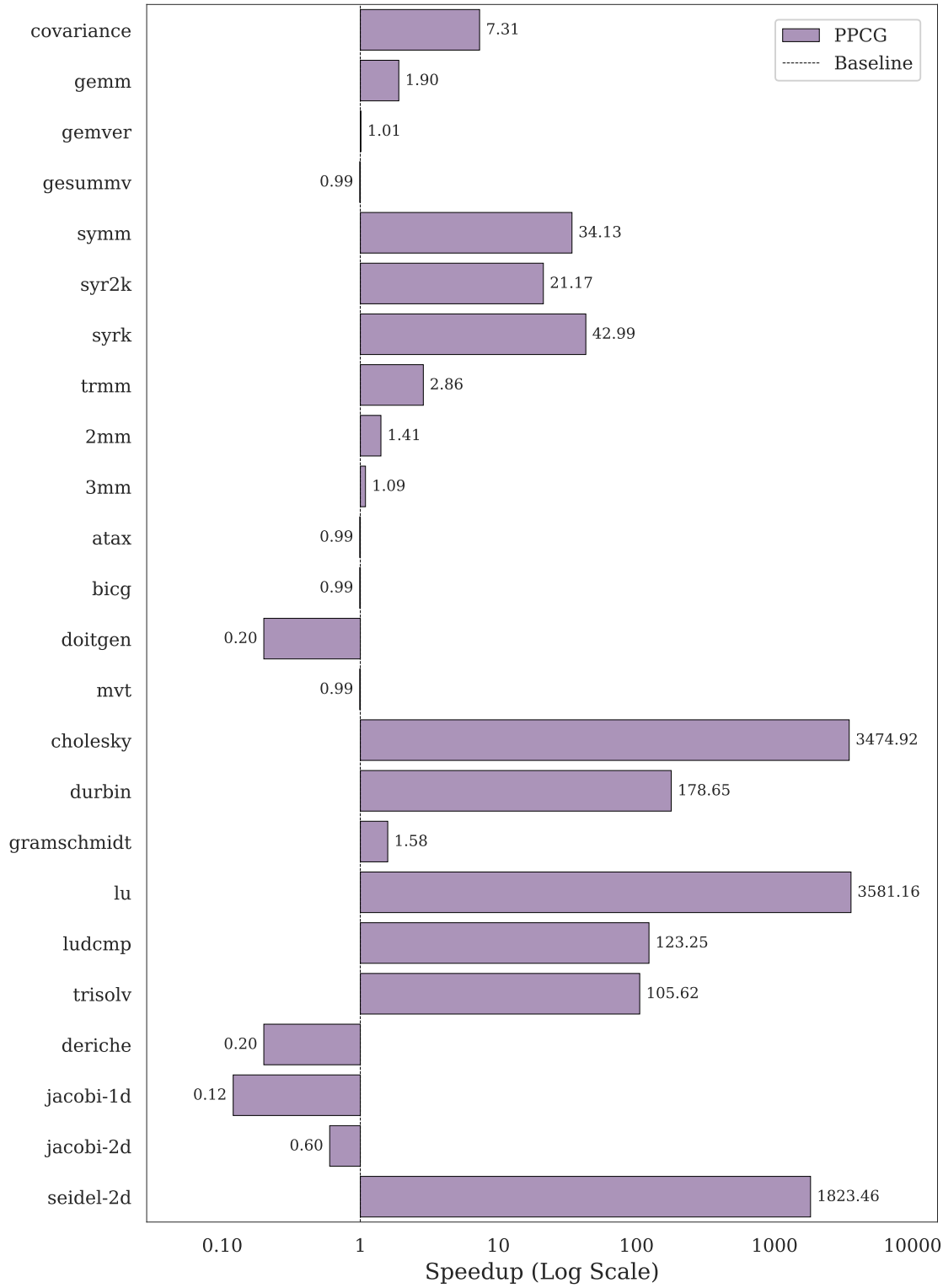


Figure 20. – Accélération de Kokkos + Polly (usePolyOpt) par rapport à Kokkos (backend CUDA) avec l'ordonnanceur PPCG sur accélérateur NVIDIA RTX 5090 (32 Go VRAM, hôte Intel Core Ultra 7 265, 20 threads sans hyperthreading).

Catégorie	Noyaux	NVIDIA RTX 5090 PPCG
Datamining	1	7.31× (1/1)
BLAS	7	5.58× (6/7)
Kernels	6	0.82× (2/6)
Solvers	6	189.10× (6/6)
Medley	1	0.20× (0/1)
Stencils	3	5.08× (1/3)
Ensemble	24	7.25× (16/24)

Tableau 6. – Moyennes géométriques des accélérations sur GPU par catégorie de benchmark. Entre parenthèses figure le nombre de noyaux accélérés sur le nombre de noyaux évalués.

Les accélérations s'étendent sur plus de quatre ordres de grandeur, de 0.12× sur `jacobi-1d` à 3581.16× sur `lu`. Seize noyaux sont accélérés, dont `gemver` de façon négligeable (1.01×), quatre restent pratiquement neutres (`gesummv`, `atax`, `bicg` et `mvt` à 0.99×) et quatre régressent (`doitgen`, `deriche`, `jacobi-1d` et `jacobi-2d`). La moyenne géométrique globale de 7.25× est tirée vers le haut par les six noyaux dont la version de référence est limitée par une boucle externe séquentielle. Sans eux, l'accélération moyenne est de 1.65×. Les résultats se répartissent ainsi en trois groupes, détaillés ci-dessous.

7.3.2. Parallélisation des noyaux à dépendances

Les accélérations les plus élevées concernent les solveurs et les factorisations : `lu` (3581.16×), `cholesky` (3474.92×), `durbin` (178.65×), `ludcmp` (123.25×) et `trisolv` (105.62×), ainsi que le stencil `seidel-2d` (1823.46×). Ces noyaux ont en commun une boucle externe porteuse de dépendances, qui interdit une parallélisation directe de l'ensemble du nid sur la grille de threads. Dans la version de référence, cette boucle externe s'exécute séquentiellement sur l'hôte, et seules les boucles internes parallèles de l'algorithme d'origine sont parallélisées. Les références de `cholesky` et `seidel-2d`, dont aucune boucle n'est parallèle sans transformation préalable, sont même entièrement séquentielles.

PPCG réorganise au contraire ces calculs pour en extraire le parallélisme. Pour les factorisations `lu`, `ludcmp` et `cholesky`, il découpe le calcul en étapes successives, lancées depuis l'hôte (liste 31). À chaque étape, la mise à jour de la partie restante de la matrice est confiée à des noyaux mobilisant des centaines de milliers de threads et chargeant en mémoire partagée les

données réutilisées au sein d'un bloc. Pour `seidel-2d`, il applique un front d'onde qui, à chaque lancement successif de blocs, traite en parallèle tous les points d'un même front. Pour `trisolv` et `durbin`, le calcul reste organisé en étapes, mais chaque étape enchaîne plusieurs noyaux sur le GPU sans retour vers l'hôte, même lorsque, comme pour `durbin`, une réduction reste calculée par un seul thread. La version de référence effectue au contraire, à chaque étape, une réduction parallèle dont le résultat est rapatrié sur l'hôte, engendrant énormément de synchronisations, ce qui explique l'écart de la performance. Enfin, l'accélération plus faible de `ludcmp` que de `lu`, dont la factorisation est identique, tient à la dernière substitution triangulaire, que le code généré exécute sur l'hôte au moyen d'une boucle inefficace.

```cpp
cudaMemcpy(dev_A, A, N * N * sizeof(double), cudaMemcpyHostToDevice);
for (int c0 = 0; c0 < N - 1; c0 += 1) {
  kernel2 <<<dim3(64, 64), dim3(4, 32)>>> (dev_A, N, c0);
  kernel3 <<<dim3(64, 64), dim3(4, 32)>>> (dev_A, N, c0);
  kernel4 <<<dim3(64), dim3(32)>>> (dev_A, N, c0);
}
cudaMemcpy(A, dev_A, N * N * sizeof(double), cudaMemcpyDeviceToHost);
```

Liste 31. – Extrait simplifié du code hôte généré par PPCG pour `lu`.

Le noyau `gramschmidt` n'obtient qu'un gain modeste (1.58×). Son calcul se déroule en étapes successives, et sa version de référence parallélise déjà l'essentiel du travail. PPCG parallélise plus largement l'opération principale de la mise à jour de la matrice, mais il calcule sur un seul thread la norme dont chaque étape a besoin, là où la référence l'obtient par une réduction parallèle, ce qui limite le gain.

7.3.3. Exploitation de la hiérarchie mémoire

Sur les noyaux d'algèbre linéaire, les gains dépendent surtout du parallélisme exploité par la version de référence. Celles de `syrk` (42.99×), `syr2k` (21.17×) et `covariance` (7.31×) ne parallélisent qu'une seule dimension, avec un thread par ligne du résultat. Celle de `symm` (34.13×) enchaîne des lancements successifs d'un petit nombre de threads. C'est bien trop peu pour occuper tous les cœurs de la carte. PPCG génère au contraire un noyau, découpé en tuiles de 32×32 éléments. Chaque tuile est chargée en mémoire partagée par les threads d'un bloc, puis réutilisée depuis cette mémoire rapide. Pour `trmm` (2.86×), les dépendances du calcul limitent le code généré à la même parallélisation que la référence, avec un thread par

colonne, et le gain provient essentiellement de la mémoire partagée. Enfin, `gemm` (1.90×), `2mm` (1.41×) et `3mm` (1.09×) n'obtiennent que des gains modérés grâce à la mémoire partagée et à la privatisation des variables, les versions de référence utilisant déjà un parallélisme efficace.

À l'inverse, `gemver` (1.01×), `gesummv`, `atax`, `bicg` et `mvt` (0.99×) restent neutres. Comme sur CPU, ces produits matrice-vecteur lisent chaque élément de matrice une seule fois. Leur performance est bornée par la bande passante de la mémoire globale. PPCG reproduit la parallélisation faite dans la version de référence.

7.3.4. Contre-performances

Quatre noyaux régressent : `jacobi-1d` (0.12×), `doitgen` (0.20×), `deriche` (0.20×) et `jacobi-2d` (0.60×). Dans les quatre cas, PPCG n'expose aucun parallélisme supplémentaire et reproduit la parallélisation de la version de référence. PPCG génère des gardes conditionnelles et des boucles supplémentaires pour garantir la correction aux bords du domaine, dont le coût devient important sur des noyaux aussi légers. Ces surcoûts, multipliés par le nombre de lancements, expliquent l'ampleur des régressions.

7.3.5. Bilan de l'évaluation sur GPU

La réintégration de PPCG dans Polly permet de générer automatiquement, à partir de code Kokkos, du code GPU qui accélère 16 des 24 noyaux compilés. Les gains les plus importants, atteignant trois ordres de grandeur, sont obtenus sur les noyaux à dépendances, dont PPCG réorganise le calcul en étapes ou en fronts d'onde massivement parallèles, là où la version de référence exécute une partie du calcul sur un seul thread. Sur les noyaux d'algèbre linéaire dont la référence ne parallélise qu'une dimension, la projection du calcul sur l'architecture GPU et l'utilisation de la mémoire partagée apportent des gains, jusqu'à 42.99× sur `syrk`.

Cette évaluation met également en évidence les limites actuelles de la génération de code GPU. Lorsque l'ordonnanceur ne parvient pas à trouver davantage de parallélisme, le surcoût de contrôle rend le code généré moins performant que celui du backend CUDA de Kokkos.

7.4. Temps de compilation

Les algorithmes polyédriques ont une complexité qui croît de manière exponentielle avec le nombre de paramètres et de contraintes du SCoP. Le gain obtenu à l'exécution doit donc être mis en regard du surcoût qu'impose l'analyse polyédrique lors de la compilation. Le tableau 7 rapporte, pour chaque noyau, le temps de compilation de Clang avec Kokkos et Polly (option `usePolyOpt`) relativement à celui de Clang avec Kokkos seul, mesuré sur la machine Intel avec les ordonnanceurs ISL et Pluto pour la cible CPU et PPCG pour la cible GPU.

Catégorie	**Noyau**	**CPU**		**GPU**
		ISL	**Pluto**	**PPCG**
Datamining	`covariance`	1.07	1.70	0.96
BLAS	`gemm`	**2.55**	1.06	0.97
	`gemver`	1.03	1.05	0.81
	`gesummv`	1.02	**10.40**	0.89
	`symm`	1.06	1.09	1.17
	`syr2k`	1.02	1.05	0.99
	`syrk`	1.02	**10.40**	0.99
	`trmm`	1.02	1.06	1.01
Kernels	`2mm`	1.07	1.17	1.74
	`3mm`	**28.82**	1.38	**17.36**
	`atax`	1.01	1.04	0.91
	`bicg`	1.01	1.03	0.90
	`doitgen`	1.04	1.09	1.37
	`mvt`	1.03	1.04	0.86
Solvers	`cholesky`	1.04	1.13	1.05
	`durbin`	1.00	1.02	1.01
	`gramschmidt`	1.06	1.14	0.92
	`lu`	1.05	1.16	1.05
	`ludcmp`	1.09	1.26	1.03
	`trisolv`	1.02	1.03	0.94
Medley	`deriche`	1.10	1.24	**2.13**
Stencils	`adi`	1.14	1.38	∞
	`fdtd-2d`	1.04	1.51	∞
	`heat-3d`	1.14	**24.04**	∞
	`jacobi-1d`	1.13	1.17	0.93
	`jacobi-2d`	1.05	**2.60**	**88.28**
	`seidel-2d`	1.07	1.92	1.00

Catégorie	Noyau	CPU		GPU
		ISL	Pluto	PPCG
Moyenne géométrique		1.23	1.60	1.41
Médiane		1.05	1.16	1.00

Tableau 7. – Temps de compilation relatif de Clang avec Kokkos + Polly (`usePolyOpt`) par rapport à Clang avec Kokkos, mesuré sur la machine Intel.

Pour la grande majorité des noyaux, le surcoût reste faible. Les médianes s'établissent à 1.05× pour ISL, 1.16× pour Pluto et 1.00× pour PPCG, et 25 des 27 noyaux se compilent en moins de 1.2× le temps de référence avec ISL. Ce résultat s'explique en partie par l'isolation des noyaux (Chapitre 4), Polly n'analysant que les fonctions marquées par l'option `usePolyOpt` et non l'ensemble du module, mais aussi par la réduction de l'usage intensif de templates dans les backends modifiés. Le surcoût légèrement plus élevé de Pluto provient de l'étape d'ordonnancement et des étapes d'export et d'import au format OpenScop (section 6.1).

Quelques noyaux font exception. Le plus marquant est `3mm`, dont la compilation est 28.82× plus longue avec ISL et 17.36× avec PPCG. Pluto est quant à lui pénalisé sur `gesummv`, `syrk` (10.40×) et `heat-3d` (24.04×), et PPCG sur `jacobi-2d` (88.28×). Enfin, la compilation de `adi`, `fdtd-2d` et `heat-3d` avec PPCG ne termine pas en un temps raisonnable. Ce point est discuté dans l'évaluation sur GPU (section 7.3.1).

7.5. Comparaison avec Polly sur PolyBench

Les sections précédentes comparent notre chaîne de compilation à une implémentation Kokkos idiomatique. Cette section la compare à l'usage habituel de Polly, appliqué aux noyaux de la suite PolyBench écrits en C séquentiel. Elle vérifie que l'abstraction Kokkos, une fois rendue visible au compilateur, n'empêche pas Polly d'obtenir des performances comparables à celles qu'il atteint sur le code C d'origine. La version de Polly utilisée est issue de la même version de LLVM que notre chaîne de compilation, sans nos modifications. Les mesures sont réalisées sur la machine Intel avec l'ordonnanceur ISL, une taille de tuile de 32, 20 threads et les mêmes tailles de problème que dans le reste du chapitre (tableau 4). Une accélération supérieure à 1 signifie que notre version est plus rapide que Polly appliqué au code C. Le noyau

`deriche` est absent de la comparaison, car Polly s'arrête sur une erreur lors de sa compilation. Les deux versions sont compilées en `-O3` avec les mêmes options de Polly.

Sur les 26 noyaux mesurés, la moyenne géométrique des accélérations est de 1.07×, et 14 noyaux restent à moins de 10 % de Polly appliqué au code C. Pour plus de la moitié des noyaux, notre chaîne de compilation obtient donc des performances proches de celles de Polly sur le code d'origine. Les écarts restent toutefois importants sur quelques noyaux, dans les deux sens.

Noyaux plus rapides avec notre version. `symm` (9.52×), `adi` (3.86×), `seidel-2d` (2.93×) et, avec une accélération moins importante, `lu` (1.17×) et `bicg` (1.15×). Sur les trois premiers, les optimisations que LLVM applique avant Polly remplacent, dans le code C, certaines lectures de tableaux par des valeurs scalaires, transmises d'une itération ou d'une instruction à l'autre. Comme pour `deriche` (section 7.2.4), ces scalaires créent de fausses dépendances, qui empêchent Polly de tuiler et de paralléliser le nid principal de `symm` et deux des quatre phases d'`adi`, et de tuiler celui de `seidel-2d`. Dans notre chaîne de compilation, la passe `ArrayReg2MemPass` (section 4.3.3), absente de Polly d'origine, rétablit ces accès aux tableaux. Polly tuile et parallélise alors `symm` et `adi`, et tuile `seidel-2d`, qui ne gagne que par la localité des données.

Noyaux plus lents avec notre version. `jacobi-1d` (0.24×), `2mm` (0.73×), `syrk` (0.79×), `jacobi-2d` (0.80×), `doitgen` (0.83×), `gemm` (0.85×) et `ludcmp` (0.89×). Sur tous ces noyaux, comme sur `lu` et `bicg`, les deux versions appliquent les mêmes transformations et parallélisent les mêmes boucles. Les tailles du problème sont toutefois connues à la compilation dans le code C et passées en paramètres dans notre version. L'écart ne provient donc pas des transformations polyédriques elles-mêmes, mais du code généré ensuite par Polly et LLVM.

Sur `jacobi-1d`, l'écart vient d'une vectorisation défavorable. Dans notre version, le vectoriseur SLP de LLVM lit en une seule fois deux éléments, `B(i-2)` et `B(i-1)`, écrits séparément aux deux itérations précédentes. Le processeur ne sait pas transmettre directement à une telle lecture des valeurs venant de deux écritures distinctes. Chaque itération doit donc attendre que ces écritures atteignent le cache. En désactivant ce vectoriseur (`-fno-slp-vectorize`), notre version devient 1.10× plus rapide que Polly sur le code C, et atteint 0.99× de la référence Kokkos, contre 0.20× avec la vectorisation (section 7.2.4).

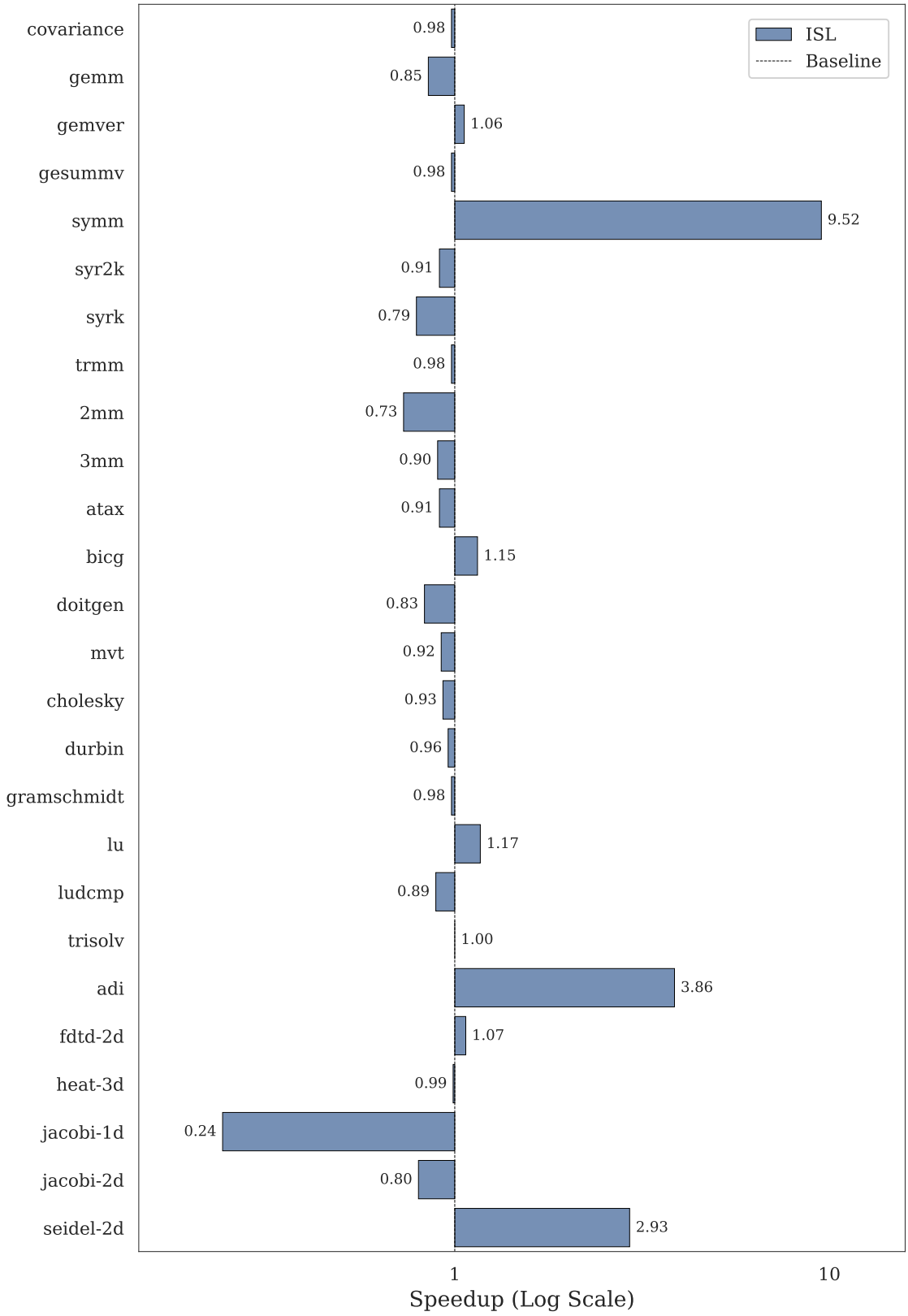


Figure 21. – Accélération de Kokkos + Polly (usePolyOpt) par rapport à Polly appliqué à PolyBench/C, avec l'ordonnanceur ISL sur processeur Intel Core Ultra 7 265 (20 threads sans hyperthreading, taille de tuile : 32).

Sur `gemm`, `syrk` et le premier produit de `2mm`, la boucle la plus interne accumule un produit multiplié par `alpha` dans un élément du tableau résultat. Dans la version C, LLVM garde `alpha` et cet accumulateur dans des registres. Dans notre version, `alpha` est relu en mémoire et l'accumulateur y est écrit à chaque itération. Le second produit de `2mm`, qui ne fait pas intervenir `alpha`, garde au contraire son accumulateur dans un registre dans les deux versions. Dans le code Kokkos, `alpha` est un membre de la lambda, lu au travers d'un pointeur. LLVM ne parvient pas à prouver l'absence d'aliasing, ce qui l'empêche à la fois de sortir sa lecture de la boucle et de garder l'accumulateur dans un registre.

Sur `doitgen` et `jacobi-2d`, l'écart tient aux tailles du problème, connues à la compilation dans le code C. Sur `doitgen`, LLVM peut alors dérouler et vectoriser la boucle la plus interne, qui reste scalaire dans notre version. Sur `jacobi-2d`, les adresses des lignes voisines se calculent par des décalages constants dans le code C, alors que notre version doit les garder dans des registres supplémentaires. Sa boucle la plus interne manque alors de registres et exécute environ 50 % d'instructions en plus.

Enfin, sur `ludcmp`, la boucle la plus coûteuse est identique dans les deux versions. L'écart vient de la taille $N = 2048$, une puissance de deux. En parcourant une colonne de `A`, la boucle provoque alors de nombreux conflits dans le cache. Les deux versions sont environ deux fois plus lentes qu'avec $N = 2040$ ou $N = 2056$, tailles pour lesquelles l'écart entre elles disparaît ($0.96\times$ et $1.00\times$). Ces conflits dépendent de la position des tableaux en mémoire, car l'adresse d'un élément détermine l'emplacement du cache où il est rangé. Or cette position diffère entre les deux versions. Dans le code C, PolyBench aligne chaque tableau sur le début d'une page mémoire. Dans notre version, les données d'une vue Kokkos sont précédées d'un en-tête d'allocation, ce qui les décale de quelques lignes de cache. Les éléments lus par la boucle ne se répartissent donc pas de la même façon dans le cache, ce qui explique l'écart restant pour $N = 2048$.

En résumé, l'abstraction Kokkos n'empêche pas Polly d'optimiser les noyaux. Notre chaîne de compilation obtient en moyenne des performances légèrement supérieures à celles de Polly sur le code C d'origine. Sur les noyaux où elle l'emporte nettement, la passe `ArrayReg2MemPass` permet à Polly d'appliquer des transformations qu'il ne peut pas appliquer au code C. Sur les noyaux où elle est plus lente, les deux versions appliquent les mêmes transformations, et l'écart

vient du code que LLVM génère ensuite ou du placement des données en mémoire, et non de la visibilité des noyaux Kokkos.

Chapitre 8

Conclusion et perspectives

L'optimisation des programmes constitue un enjeu majeur pour le calcul scientifique à haute performance. Le développement de codes de calcul est en effet confronté à la complexité et à l'hétérogénéité des architectures matérielles modernes. Si les bibliothèques de portabilité des performances, telles que Kokkos, permettent de s'abstraire de l'architecture cible pour le déploiement des noyaux de calcul, elles confient largement la charge de l'optimisation fine aux compilateurs. Or, ces derniers peinent souvent à appliquer des transformations agressives face à la complexité des nids de boucles rencontrés dans les applications réelles.

À l'inverse, le modèle polyédrique offre un cadre d'optimisation mathématique puissant, mais son application directe sur des abstractions de haut niveau reste difficile. L'objectif principal de cette thèse a été de combler ce fossé sémantique en proposant une approche hybride, unissant la flexibilité et la portabilité de Kokkos à la puissance d'analyse et d'optimisation du modèle polyédrique.

8.1. Contributions

8.1.1. Visibilité des noyaux Kokkos

L'optimisation polyédrique nécessite une représentation mathématique précise du code source, une information qui est souvent masquée par l'utilisation d'abstractions de haut niveau et de constructions logicielles complexes. Les noyaux Kokkos, bien que portables et expressifs, demeurent opaques pour les outils d'analyse statique polyédrique classiques.

Pour lever ce verrou technologique, nous avons modifié la chaîne de compilation et étendu l'outil Polly au sein de LLVM. Cette intégration permet d'extraire la sémantique et la structure des noyaux Kokkos pour les rendre pleinement visibles et compréhensibles par les outils polyédriques.

Cela constitue la fondation de notre approche hybride, en permettant l'extraction transparente de modèles mathématiques rigoureux à partir de codes sources portables.

8.1.2. Motifs d'exécution complexes

Si l'extraction et l'optimisation de noyaux isolés et réguliers constituent une première étape cruciale, les applications scientifiques reposent fréquemment sur des schémas d'exécution plus complexes. L'application du modèle polyédrique à ces structures non triviales nécessite des mécanismes d'analyse pour garantir la validité des transformations.

Nous avons ainsi étendu notre approche hybride à la gestion et à la modélisation des nids de boucles triangulaires, ainsi qu'à la modélisation fidèle des nids de boucles fragmentés en différents appels à `parallel_for`.

Nous avons démontré que notre approche pouvait s'appliquer efficacement à ces structures algorithmiques, augmentant significativement la portée des codes pouvant être optimisés par le modèle polyédrique.

8.1.3. Ordonnancement et génération de code hétérogène

La dernière étape de notre démarche consiste à introduire de nouveaux ordonnanceurs, à la fois pour profiter des capacités d'optimisation de ces derniers, mais aussi pour permettre d'exploiter des architectures hétérogènes.

Pour répondre à ce défi, nous avons traduit la représentation polyédrique en un format standard, permettant ainsi à d'autres ordonnanceurs, tels que Pluto, d'optimiser le code. L'intégration de l'outil PPCG a également été réalisée plus en profondeur dans le pipeline de compilation afin de permettre l'application de transformations polyédriques ciblant des noyaux GPU.

Nos évaluations expérimentales ont validé l'efficacité de cette chaîne d'outils. Nos résultats démontrent des accélérations significatives par rapport aux implémentations Kokkos standards, tant sur CPU que sur GPU. Ces résultats prouvent que l'approche hybride Kokkos, couplée aux optimisations polyédriques, permet non seulement de surpasser les performances de l'approche Kokkos standard, mais aussi d'exploiter pleinement le potentiel des architectures hétérogènes, tout en conservant la portabilité du code. Comparée à Polly appliqué aux versions C d'origine des noyaux, notre chaîne de compilation ne dégrade pas les performances, et les améliore même en moyenne, ce qui montre que l'abstraction Kokkos ne fait pas obstacle à l'optimisation polyédrique.

8.2. Discussion et limites

Bien que notre approche hybride démontre des gains de performance prometteurs pour les codes Kokkos, elle reste soumise à certaines limites conceptuelles et techniques.

Premièrement, notre outil ne prend pas en charge la politique d'exécution hiérarchique `TeamPolicy`. Cette limitation découle d'une incompatibilité inhérente avec le modèle polyédrique classique. La sémantique de `TeamPolicy` repose sur une hiérarchie d'exécution parallèle incompatible avec le modèle polyédrique, limitant ainsi notre périmètre d'action actuel aux politiques `RangePolicy` et `MDRangePolicy`.

Deuxièmement, la complexité des algorithmes polyédriques peut entraîner des problèmes de scalabilité lors de la phase de compilation. Si le surcoût reste modéré pour la grande majorité des noyaux, le calcul de l'ordonnancement par PPCG ne termine pas en un temps raisonnable sur les stencils `adi`, `fdtd-2d` et `heat-3d`, en raison du nombre de contraintes générées pour ces nids de boucles.

Enfin, l'outil accorde actuellement une confiance totale aux informations sémantiques fournies par le développeur (noms statiques, contraintes d'assomption), ce qui constitue un point de fragilité pouvant mener à des erreurs silencieuses en cas de spécification erronée.

8.3. Perspectives

Les travaux présentés dans ce manuscrit ouvrent plusieurs pistes, allant de l'amélioration de l'implémentation actuelle à des extensions plus fondamentales de notre approche hybride.

Premièrement, l'implémentation logicielle pourrait être améliorée. À l'heure actuelle, notre prototype prend en charge les espaces d'exécution fondamentaux de Kokkos, à savoir `Serial`, `OpenMP` et `Cuda`. Une évolution naturelle consisterait à intégrer d'autres backends d'exécution, tels que HIP pour cibler les accélérateurs matériels d'AMD, ou SYCL pour les architectures Intel. Cette extension garantirait une portabilité des performances sur un spectre matériel encore plus vaste.

Deuxièmement, la robustesse de l'outil face aux spécifications de l'utilisateur pourrait être considérablement renforcée. Actuellement, notre approche permet aux développeurs d'injecter des informations sémantiques cruciales pour l'analyse polyédrique, notamment via le nommage statique des vues et l'ajout de suppositions mathématiques. Une perspective de développement majeure consisterait à implémenter des mécanismes de vérification statique capables de valider la cohérence de ces informations. Cela permettrait de fournir une gestion des erreurs robuste, avec des retours pertinents lors de la compilation.

Dans cette même dynamique d'intégration à l'écosystème Kokkos, une autre évolution viserait à contrôler les paramètres d'optimisation, comme la taille des tuiles, directement au travers des abstractions de la bibliothèque. Cela offrirait au développeur la possibilité de piloter finement l'application du modèle polyédrique depuis son code source, en respectant la philosophie de conception de Kokkos.

Enfin, notre chaîne d'outils s'appuie encore sur des heuristiques de tuilage et de partitionnement de blocs qui manquent d'adaptabilité. En l'absence de modèles de coût dynamiques guidés par la structure des noyaux, les tailles de tuiles générées ne garantissent pas toujours l'atteinte des performances maximales théoriques. Le développement d'heuristiques adaptatives, capables d'ajuster automatiquement ces paramètres en fonction de la structure des noyaux et de l'architecture ciblée, constitue ainsi une piste de recherche.

Références

[Bon+08] U. Bondhugula, M. Baskaran, S. Krishnamoorthy, J. Ramanujam, A. Rountev, et P. Sadayappan, « Automatic Transformations for Communication-Minimized Parallelization and Locality Optimization in the Polyhedral Model », in *Proceedings of the International Conference on Compiler Construction* (*ETAPS CC*), Budapest, Hungary, avr. 2008a.

[Gro+11] T. Grosser, H. Zheng, R. Aloor, A. Simbürger, A. Größlinger, et L.-N. Pouchet, « Polly - Polyhedral Optimization in LLVM », in *1st International Workshop on Polyhedral Compilation Techniques* (*IMPACT*), C. Alias et C. Bastoul, Éd., Chamonix, France, 2011. [En ligne]. Disponible sur: http://perso.ens-lyon.fr/christophe.alias/impact2011/impact-07.pdf

[Ver+13] S. Verdoolaege, J. Juega, A. Cohen, J. Gomez, C. Tenllado, et F. Catthoor, « Polyhedral Parallel Code Generation for CUDA », *ACM Transactions on Architecture and Code Optimization* (*TACO*), vol. 9, p. 54, 2013, doi: 10.1145/2400682.2400713.

[ET13] H. C. Edwards et C. R. Trott, « Kokkos: Enabling Performance Portability Across Manycore Architectures », in *20k3 Extreme Scaling Workshop* (*xsw 2013*), 2013, p. 18-24. doi: 10.1109/XSW.2013.7.

[Tro+22] C. R. Trott et al., « Kokkos 3: Programming Model Extensions for the Exascale Era », *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, nº 4, p. 805-817, 2022, doi: 10.1109/TPDS.2021.3097283.

[FL11] P. Feautrier et C. Lengauer, « Polyhedron Model », *Encyclopedia of Parallel Computing*. Springer, p. 1581-1592, 2011. doi: 10.1007/978-0-387-09766-4_502.

[Bas12] C. Bastoul, « Contributions to High-Level Program Optimization », Habilitation to Supervise Research, Paris, France, 2012. [En ligne]. Disponible sur: https://icps.u-strasbg.fr/~bastoul/research/papers/Bastoul_HDR.pdf

[Fea91] P. Feautrier, « Dataflow Analysis of Array and Scalar References », *International Journal of Parallel Programming*, vol. 20, nº 1, p. 23-53, 1991, doi: 10.1007/BF01407931.

[Fea92] P. Feautrier, « Some Efficient Solutions to the Affine Scheduling Problem. Part I. One-Dimensional Time », *International Journal of Parallel Programming*, vol. 21, nº 5, p. 313-347, 1992a, doi: 10.1007/BF01407835.

[Fea92] P. Feautrier, « Some Efficient Solutions to the Affine Scheduling Problem. Part II. Multidimensional Time », *International Journal of Parallel Programming*, vol. 21, nº 6, p. 389-420, 1992b, doi: 10.1007/BF01379404.

[Ver10] S. Verdoolaege, « ISL: An Integer Set Library for the Polyhedral Model », in *Proc. of the Third Int. Conf. on Math. Software*, in LNCS 6327. 2010, p. 299-302.

[Bon+08] U. Bondhugula, A. Hartono, J. Ramanujam, et P. Sadayappan, « A Practical Automatic Polyhedral Parallelizer and Locality Optimizer », in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation* (*PLDI*), Tucson, Arizona, USA, juin 2008b.

[Bas04] C. Bastoul, « Code Generation in the Polyhedral Model Is Easier Than You Think », in *Proceedings of the 13th International Conference on Parallel Architectures and Compilation Techniques* (*PACT*), 2004, p. 7-16. doi: 10.1109/PACT.2004.1342537.

[Bas] C. Bastoul, « OpenScop: A Specification and a Library for Data Exchange in Polyhedral Compilation Tools ». [En ligne]. Disponible sur: https://github.com/periscop/openscop

[Caa+17] J. M. M. Caamano, A. Sukumaran-Rajam, A. Baloian, M. Selva, et P. Clauss, « APOLLO: Automatic speculative POLyhedral Loop Optimizer », *7th International Workshop on Polyhedral Compilation Techniques* (*IMPACT*), janv. 2017.

[Mos21] W. S. Moses, « Polygeist: Affine C in MLIR », 2021. [En ligne]. Disponible sur: https://api.semanticscholar.org/CorpusID:231771924

[Gro11] T. Grosser, « Enabling Polyhedral Optimizations in LLVM », Diploma Thesis, 2011. [En ligne]. Disponible sur: https://polly.llvm.org/publications/grosser-diploma-thesis.pdf

[GGL12] T. Grosser, A. Groesslinger, et C. Lengauer, « POLLY — Performing Polyhedral Optimizations on a Low-Level Intermediate Representation », *Parallel Processing Letters*, vol. 22, nº 4, 2012, doi: 10.1142/S0129626412500107.

[LA04] C. Lattner et V. Adve, « LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation », in *International Symposium on Code Generation and Optimization* (*CGO*), 2004, p. 75-86. doi: 10.1109/CGO.2004.1281665.

[Gro+15] T. Grosser, J. Ramanujam, L.-N. Pouchet, P. Sadayappan, et S. Pop, « Optimistic Delinearization of Parametrically Sized Arrays », in *Proceedings of the 29th ACM on International Conference on Supercomputing*, in ICS '15. Newport Beach, California, USA: Association for Computing Machinery, 2015, p. 351-360. doi: 10.1145/2751205.2751248.

[GVC15] T. Grosser, S. Verdoolaege, et A. Cohen, « Polyhedral AST Generation Is More Than Scanning Polyhedra », *ACM Transactions on Programming Languages and Systems*, vol. 37, nº 4, p. 12:1-12:50, 2015, doi: 10.1145/2743016.

[Rag11] Raghesh A, « A Framework for Automatic OpenMP Code Generation », Master's Thesis, 2011. [En ligne]. Disponible sur: https://polly.llvm.org/publications/raghesh-a-masters-thesis.pdf

[GH16] T. Grosser et T. Hoefler, « Polly-ACC: Transparent compilation to heterogeneous hardware », in *Proceedings of the the 30th International Conference on Supercomputing* (*ICS'16*), juin 2016.

[HK14] R. D. Hornung et J. A. Keasler, « The RAJA Portability Layer: Overview and Status », technical report, 2014. doi: 10.2172/1169830.

[Mar26] A. Marowka, « Evaluating SYCL as a Unified Programming Model for Heterogeneous Systems ». [En ligne]. Disponible sur: https://arxiv.org/abs/2604.16043

[ISO17] ISO/IEC, *ISO/IEC 14882:2017 — Programming Languages — C++*, 5e éd. Genève, Suisse: International Organization for Standardization, 2017. [En ligne]. Disponible sur: https://www.iso.org/standard/68564.html

[Ham+18] S. D. Hammond, C. R. Trott, D. Ibanez, et D. Sunderland, « Profiling and Debugging Support for the Kokkos Programming Model », in *High Performance Computing*, Cham: Springer International Publishing, 2018, p. 743-754.

[Huc22] K. A. Huck, « Broad Performance Measurement Support for Asynchronous Multi-Tasking with APEX », in *2022 IEEE/ACM 7th International Workshop on Extreme Scale Programming Models and Middleware* (*ESPM2*), 2022, p. 20-29. doi: 10.1109/ESPM256814.2022.00008.

[Bec+17] D. Beckingsale, O. Pearce, I. Laguna, et T. Gamblin, « Apollo: Reusable Models for Fast, Dynamic Tuning of Input-Dependent Code », in *2017 IEEE International Parallel and Distributed Processing Symposium* (*IPDPS*), 2017, p. 307-316. doi: 10.1109/IPDPS.2017.38.

[Bon+08] U. Bondhugula, A. Hartono, J. Ramanujam, et P. Sadayappan, « A practical automatic polyhedral parallelizer and locality optimizer », in *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*, in PLDI '08. Tucson, AZ, USA: Association for Computing Machinery, 2008c, p. 101-113. doi: 10.1145/1375581.1375595.

[Sjö+09] J. Sjödin, S. Pop, H. Jagasia, T. Grosser, et A. Pop, « Design of graphite and the Polyhedral Compilation Package », in *GCC Developers' Summit*, Montréal, Canada, juin 2009. [En ligne]. Disponible sur: https://minesparis-psl.hal.science/hal-00408759

[Mer03] J. Merrill, « Generic and gimple: A new tree represen-tation for entire functions », in *Proceedings of the 2003 GCC Developers' Summit*, 2003. [En ligne]. Disponible sur: https://api.semanticscholar.org/CorpusID:58211542

[Lat+21] C. Lattner et al., « MLIR: Scaling Compiler Infrastructure for Domain Specific Computation », in *2021 IEEE/ACM International Symposium on Code Generation and Optimization* (*CGO*), 2021, p. 2-14. doi: 10.1109/CGO51591.2021.9370308.

[Bag+18] R. Baghdadi et al., « Tiramisu: A Polyhedral Compiler for Expressing Fast and Portable Code ». [En ligne]. Disponible sur: https://arxiv.org/abs/1804.10694

[Ram+18] C. Ramon-Cortes, R. Amela, J. Ejarque, P. Clauss, et R. M. Badia, « AutoParallel: A Python module for automatic parallelization and distributed execution of affine loop nests », *arXiv preprint arXiv:1810.11268*, 2018.

[Har+20] C. R. Harris et al., « Array programming with NumPy », *Nature*, vol. 585, nº 7825, p. 357-362, sept. 2020, doi: 10.1038/s41586-020-2649-2.

[Al +21] N. Al Awar, S. Zhu, G. Biros, et M. Gligoric, « A Performance Portability Framework for Python », in *International Conference on Supercomputing*, 2021, p. 467-478.

[Bas+20] M. Baskaran, C. Jin, B. Meister, et J. Springer, « Automatic Mapping and Optimization to Kokkos with Polyhedral Compilation », in *2020 IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, p. 1-7. doi: 10.1109/HPEC43674.2020.9286233.

[BBP17] U. Bondhugula, V. Bandishti, et I. Pananilath, « Diamond Tiling: Tiling Techniques to Maximize Parallelism for Stencil Computations », *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, nº 5, p. 1285-1298, 2017, doi: 10.1109/TPDS.2016.2615094.

[TLG24] A. Thangamani, V. Loechner, et S. Genaud, « A Survey of General-purpose Polyhedral Compilers », *ACM Trans. Archit. Code Optim.*, vol. 21, nº 4, nov. 2024, doi: 10.1145/3674735.

[Pou11] L.-N. Pouchet, « PolyBench: The Polyhedral Benchmark Suite », *URL: http://www.cs.colostate.edu/~pouchet/software/polybench*, 2011.

École doctorale **Mathématiques, sciences** de l'**information** et de l'**ingénieur** | ED 269
Université de Strasbourg

Ugo BATTISTON

Université de Strasbourg

Optimisation polyédrique automatique des noyaux Kokkos : un co-design bibliothèque–compilateur

Résumé

Dans cette thèse, nous présentons une approche de co-design visant à faire bénéficier Kokkos, une bibliothèque de portabilité des performances, des optimisations offertes par le modèle polyédrique. Si ce type d'environnement permet aux développeurs d'écrire un code unique pour CPU et GPU tout en garantissant des performances indépendantes de l'architecture, il ne permet pas de bénéficier des optimisations offertes par le modèle polyédrique. En effet, les optimiseurs polyédriques, tels que Polly de LLVM, échouent à appliquer leurs modèles d'optimisation. Pour surmonter cet obstacle, nous proposons une intégration étroite entre Kokkos et Polly. En annotant la représentation intermédiaire et en appliquant des transformations spécifiques au code généré par Kokkos, nous restituons à l'optimiseur les informations nécessaires pour appliquer des transformations agressives. Le couplage avec un ordonnanceur et un générateur de code dédiés permet d'étendre ces optimisations aux architectures CPU et GPU. Notre évaluation sur la suite PolyBench démontre des accélérations atteignant 118,26× sur CPU et trois ordres de grandeur d'accélération sur GPU par rapport à l'implémentation Kokkos idiomatique standard.

Mots-clés : Calcul haute performance, Modèle polyédrique, Compilation, Optimisation de boucles, Représentation intermédiaire, Kokkos

Automatic Polyhedral Optimization of Kokkos Kernels: A Library-Compiler Co-Design

Abstract

In this thesis, we present a co-design approach aimed at bringing the optimizations offered by the polyhedral model to Kokkos, a performance portability framework. Although this type of framework allows developers to write a single codebase for both CPUs and GPUs while guaranteeing architecture-independent performance, it does not allow taking advantage of the optimizations offered by the polyhedral model. Indeed, polyhedral optimizers, such as LLVM's Polly, fail to apply their optimization models. To overcome this obstacle, we propose a tight integration between Kokkos and Polly. By annotating the intermediate representation and applying transformations specific to the code generated by Kokkos, we restore the necessary information for the optimizer to apply aggressive transformations. Coupling this with a dedicated scheduler and code generator allows extending these optimizations to CPU and GPU architectures. Our evaluation on the PolyBench suite demonstrates speedups reaching 118.26× on CPU and three orders of magnitude on GPU compared to the standard idiomatic Kokkos implementation.

Keywords: High-performance computing, Polyhedral model, Compilation, Loop optimization, Intermediate representation, Kokkos